VibeStore — Guia de Implementação do Frontend

Contrato de referência Vue 3 · TypeScript · Vuetify 3 · Pinia · Vue Router sobre o backend reiniciado (módulos 001–007).

Fonte efetiva: `vibestore_backend_manifest.json` + migrations 001–007 + Edge `storefront-api`. Versão 1.0 · 06/10/2026.

25 tabelas/views · 40 RPCs (35 client + 5 service-only) · 36 permissões · 33 policies · 3 buckets · 4 ações públicas HTTP · 34 rotas propostas · Testado só localmente

Sumário

1. 1. Escopo, fontes e premissas
2. 2. Arquitetura e convenções do frontend
3. 3. Mapa completo de rotas propostas
4. 4. Matriz de permissões
5. 5. Camada de services e stores
6. 6. Autenticação (Supabase Auth SDK)
7. 7. Acesso, sessão e perfil
8. 8. Onboarding (plano + loja)
9. 9. Configurações da loja, branding e publicação
10. 10. Catálogo: categorias
11. 11. Catálogo: produtos, SKU, atributos e imagens
12. 12. Estoque: saldos, movimentos, contagem e lote
13. 13. Pedidos (operação manual)
14. 14. Clientes (somente leitura)
15. 15. Despesas pagas
16. 16. Relatórios ERP
17. 17. Dashboard (/app)
18. 18. Painel SUDO
19. 19. Vitrine pública via Edge storefront-api
20. 20. Catálogo completo de RPCs (40 = 35 client + 5 service-only)
21. 21. Tabelas: leitura, filtros e operações permitidas
22. 22. Catálogo de erros e mensagens
23. 23. Fluxos ponta a ponta
24. 24. Ordem de implementação e critérios de aceitação
25. 25. Ambiguidades, lacunas e checagens de deploy
26. Apêndice A — Tipos TypeScript completos
27. Apêndice B — Dicionário das 25 tabelas/views
28. Apêndice C — Permissões, buckets e policies de Storage

1. Escopo, fontes e premissas

Este guia é o **contrato de implementação do frontend** da VibeStore reiniciada (Vue 3 + TypeScript + Vuetify 3 + Pinia + Vue Router 4) sobre o backend dos **módulos 001 a 007**. Ele foi escrito para que o desenvolvedor implemente todas as telas, rotas, services, stores e ações *sem precisar abrir o Supabase*. Nenhuma linha de frontend real foi criada; rotas, nomes de arquivo, nomes de store e de service são **propostas** deste guia.

1.1 Fontes e precedência

Fonte	Papel	Uso neste guia
`vibestore_backend_manifest.json` (extraído do PostgreSQL local após as 7 migrations)	**Fonte principal e efetiva**	25 tabelas/views, 40 RPCs (assinaturas, defaults, corpo final, ACL), 33 policies, 207 constraints, 341 grants de coluna, 36 permissões, 3 buckets
Migrations 001…007 (SQL)	Complementar	Triggers, helpers `private.*` (mensagens de erro, regras de lock), seeds de role_permissions, comentários
ZIP módulo 05: `storefront-api/index.ts`, `handler.ts`, `handler_test.ts`	Contrato HTTP público	Ações catalog/product/checkout/order-status, validação, CORS, rate limit, códigos HTTP

Versões finais

- **Módulo 06 substitui** `get_public_order_status` (agora com revision, frete, timeline, pagamento, rastreio) e corrige o guard de reservas (`guard_stock_against_reservations` não bloqueia o INSERT ON CONFLICT de uma projeção existente).
- **Módulo 07** separa despesas `OPERATING` de compras `INVENTORY_PURCHASE` (caixa ≠ resultado) e trata o reembolso de pedido como fato financeiro na data do REFUND (não pelo status atual do pedido).
- Quando a migration e o manifesto divergirem, vale o manifesto (estado efetivo do banco).

Status de verificação

Os testes citados pelas fontes foram executados **somente localmente** (PostgreSQL 16 local, testes SQL de regressão/concorrência, 11 testes Deno do handler + type-check, integração handler↔PostgreSQL). **Não há deploy remoto nem homologação no Supabase do projeto.** Ver seção 25 (checagens de deploy).

1.2 O que NÃO existe no backend atual (não implementar chamadas)

Recurso ausente	O que o frontend deve fazer
Planos pagos / assinatura / cobrança	Somente plano `FREE` (CHECK `plans_current_code`). Tela de plano é informativa e seleciona o FREE.
Convites / equipe / múltiplos membros	Não existe. Acesso à loja é do OWNER (dono) ou SUDO (store.admin).
Bucket de avatar	`profiles.avatar_path` existe, mas não há bucket nem policy. Exibir iniciais; não enviar arquivo nem gravar caminho.
CRUD de clientes	`customers` é somente leitura (criado pelo checkout público). Lista e histórico de pedidos.
Exportação CSV/relatório	Não há endpoint. Opcional: gerar CSV no navegador a partir das linhas já carregadas (sem nova API).
Alterar endereço/itens de pedido	Snapshots imutáveis (trigger `ORDER_IDENTITY_AND_SNAPSHOTS_IMMUTABLE`). Para corrigir: cancelar (PENDING/PAID) e o cliente refaz o pedido.
Cupons, gateways (Mercado Pago/Stripe), webhooks, PIX automático	Pagamento é **declaração manual** em `confirm_order_payment`.
Páginas de integrações / métodos de pagamento	Não implementar. Métodos manuais são fixos: PIX, CASH, BANK_TRANSFER, CARD, OTHER.
Páginas públicas dedicadas de categoria / busca	Usar filtros `category_id` / `search` da ação `catalog` na própria vitrine (`/s/:storeSlug?categoria=…&q=…`).
Cotação de frete	Frete negociado no WhatsApp e confirmado pelo lojista em `update_order_terms` antes do pagamento.
Edição de e-mail/senha em profiles	E-mail e senha pertencem ao Supabase Auth (SDK). `profiles` não guarda e-mail.

2. Arquitetura e convenções do frontend

2.1 Pilha e estrutura de pastas proposta

TEXT

```
src/
├── main.ts                     # createApp + Pinia + Router + Vuetify
├── lib/
│   ├── supabase.ts             # createClient (anon/publishable key, PKCE)
│   ├── rpc.ts                  # wrapper tipado + parse de erro
│   ├── errors.ts               # mapa código → mensagem pt-BR
│   ├── ids.ts                  # newRequestId(), newClientSecret()
│   ├── pending.ts              # persistência de operações para retry idempotente
│   ├── money.ts                # normalização numeric/bigint
│   ├── media.ts                # caminhos de Storage + URLs públicas
│   └── safeRedirect.ts         # validação de redirect interno
├── types/{backend,rpc,storefront}.ts   # ver Apêndice A
├── services/                   # 1 arquivo por domínio (seção 5.1)
├── stores/                     # auth, access, store, cart, storefront (Pinia)
├── router/{index.ts,guards.ts}
├── layouts/{AuthLayout,AppLayout,OnboardingLayout,SudoLayout,StorefrontLayout}.vue
└── pages/{auth,app,sudo,storefront}/…
```

2.2 Variáveis de ambiente do frontend

Variável	Exemplo	Uso
`VITE_SUPABASE_URL`	https://xyz.supabase.co	Cliente Supabase (Auth, PostgREST, Storage)
`VITE_SUPABASE_ANON_KEY`	chave anon/publishable	Única chave permitida no navegador. **Nunca** service_role/secret.
`VITE_APP_URL`	https://app.vibestore.com.br	Base de `redirectTo` / `emailRedirectTo`; deve ser igual ao `APP_URL` da Edge
`VITE_STOREFRONT_API_URL`	${VITE_SUPABASE_URL}/functions/v1/storefront-api	Endpoint da Edge (pode ser derivado)

TS

```ts
// src/lib/supabase.ts
import { createClient } from '@supabase/supabase-js'

export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY,
  {
    auth: {
      flowType: 'pkce',          // links de e-mail retornam ?code=... para /auth/callback
      persistSession: true,
      autoRefreshToken: true,
      detectSessionInUrl: false, // a troca do code é EXPLÍCITA em /auth/callback (seção 6.4)
    },
  },
)
```

2.3 Regras transversais (valem para todas as telas)

Regra	Detalhe
Escrita só por RPC	Catálogo, estoque, pedidos, despesas, loja (criar/publicar/arquivar) escrevem **somente** por RPC. Tabelas não têm GRANT INSERT/UPDATE/DELETE para `authenticated`, exceto: `stores` (UPDATE de 13 colunas), `profiles` (UPDATE full_name/phone/avatar_path), `plans` (UPDATE name/description/is_active, só SUDO) e `roles/permissions/role_permissions` (CRUD, só SUDO).

Regra	Detalhe
request_id	UUID gerado no cliente *uma vez por intenção do usuário* (abrir o diálogo / clicar “Salvar”) e reutilizado em todos os retries da MESMA intenção. Mudar qualquer campo = nova intenção = novo UUID; reaproveitar com payload diferente gera `IDEMPOTENCY_KEY_REUSED`.
Replay	Resposta com `replayed: true` (estoque, pedidos, despesas) ou `created: false` (loja, categoria, produto) é **sucesso**: mostrar o mesmo toast de sucesso e recarregar dados.
Concorrência otimista	`expected_revision` obrigatório em contagem de estoque (`inventory_balances.revision`) e em todas as operações de pedido (`orders.revision`). `STALE_*_REVISION` → recarregar, mostrar valores atuais e pedir nova confirmação (com NOVO request_id).
RLS transparente	As policies filtram por loja; ainda assim, sempre filtre `.eq('store_id', storeId)` nas listas para usar índices e evitar misturar lojas quando o usuário é SUDO.
UPDATE direto silencioso	UPDATE negado por RLS retorna 0 linhas *sem erro*. Sempre usar `.select()` e tratar `data.length === 0` como “sem permissão/registro indisponível”.
Numeric/bigint	PostgREST e jsonb devolvem `numeric` e `bigint` como número JSON. Ver 2.6.
Datas	Todas as colunas são `timestamptz` (ISO). Exibir em `America/Sao_Paulo` por padrão com `Intl.DateTimeFormat('pt-BR', { timeZone })`.
PII	Telefone/endereço de clientes só em telas autenticadas com `customers.read` / `orders.read`. Nunca enviar a analytics, logs ou URLs.

2.4 Wrapper de RPC e parse de erros

TS

```ts
// src/lib/rpc.ts
import type { PostgrestError } from '@supabase/supabase-js'
import { supabase } from './supabase'

export class BackendError extends Error {
  constructor(
    public code: string,            // código de negócio (RAISE EXCEPTION) ou SQLSTATE/PGRST
    public sqlState: string,        // ex.: '22023', '42501', '23505'
    public raw?: PostgrestError,
  ) { super(code) }
  get isForbidden() { return this.sqlState === '42501' }
  get isStale() { return this.code === 'STALE_ORDER_REVISION' || this.code === 'STALE_INVENTORY_REVISION' }
  get isNetwork() { return this.code === 'NETWORK_ERROR' }
}

const BUSINESS_CODE = /^[A-Z][A-Z0-9_]{2,80}$/

export function toBackendError(e: PostgrestError | null | undefined): BackendError {
  if (!e) return new BackendError('UNKNOWN', '')
  const msg = (e.message ?? '').trim()
  if (BUSINESS_CODE.test(msg)) return new BackendError(msg, e.code ?? '', e)     // RAISE EXCEPTION 'CODIGO'
  const constraint = /constraint "([^"]+)"/.exec(msg)?.[1]                       // 23505 / 23514 / 23503
  if (constraint) return new BackendError(`${e.code}:${constraint}`, e.code ?? '', e)
  if (e.code === '42501') return new BackendError('FORBIDDEN', '42501', e)       // RLS/GRANT
  if (!e.code && /fetch|network/i.test(msg)) return new BackendError('NETWORK_ERROR', '', e)
  return new BackendError(e.code || 'UNKNOWN', e.code ?? '', e)
}

/** Chamada tipada: rpc<CreateProductArgs, CreateProductResult>('create_product', args) */
export async function rpc<A extends object, R>(fn: string, args?: A): Promise<R> {
  const { data, error } = await supabase.rpc(fn, args as never)
  if (error) throw toBackendError(error)
  return data as R
}
```

2.5 request_id, client_secret e operações pendentes

TS

```ts
// src/lib/ids.ts
export const newRequestId = (): string => crypto.randomUUID()   // v4, minúsculo
```

```
/** 32 bytes aleatórios → base64url sem padding = exatamente 43 caracteres */
export function newClientSecret(): string {
  const bytes = crypto.getRandomValues(new Uint8Array(32))
  let bin = ''
  bytes.forEach(b => { bin += String.fromCharCode(b) })
  return btoa(bin).replace(/\+/g, '-').replace(/\//g, '_').replace(/=+$/, '')
}

// src/lib/pending.ts — mantém a MESMA chave entre retries (inclusive após recarregar a aba)
interface Pending<T> { requestId: string; fingerprint: string; payload: T; createdAt: number }
export function stablePending<T>(key: string, payload: T): Pending<T> {
  const fingerprint = JSON.stringify(payload)
  const saved = sessionStorage.getItem(key)
  if (saved) {
    const p = JSON.parse(saved) as Pending<T>
    if (p.fingerprint === fingerprint) return p          // mesma intenção → mesma chave
  }
  const fresh = { requestId: crypto.randomUUID(), fingerprint, payload, createdAt: Date.now() }
  sessionStorage.setItem(key, JSON.stringify(fresh))
  return fresh
}
export const clearPending = (key: string) => sessionStorage.removeItem(key)
```

Uso: `const op = stablePending(`order:${orderId}:pay`, form)`; chamar a RPC com `op.requestId`; em sucesso (inclusive replay) `clearPending`; em erro de rede/timeout manter e oferecer “Tentar novamente”; em erro de negócio definitivo (ex.: `ORDER_NOT_PENDING`) limpar.

2.6 Normalização de numeric / bigint

Tipo SQL	Colunas	Tratamento
numeric(12,2)	preços, totais, frete, despesas, pagamentos	Número JSON com ≤ 2 casas. Converter para centavos inteiros para somar/comparar; exibir com `Intl.NumberFormat('pt-BR',{style:'currency',currency:'BRL'})`. Enviar sempre com no máximo 2 casas (o banco rejeita mais: `INVALID_MONEY_VALUE`, `INVALID_EXPENSE_AMOUNT`, `INVALID_SHIPPING_AMOUNT`).
numeric(18,6) / numeric(26,6)	unit_cost, average_cost, stock_value, value_delta, allocated_value	Até 6 casas. Somente exibição (arredondar para 2 na UI) — nunca recalcular custo no cliente. Para telas de auditoria com valores grandes, pedir como texto: `.select('stock_value::text')`.
numeric de agregação (ERP)	todos os campos de `get_erp_*`	Somas sem escala fixa (ex.: 123.456789). Arredondar só na exibição.
bigint	revision, security_audit_logs.id	Cabe em `number` (≤ 2^53). Reenviar o valor exatamente como lido.
integer	quantidades	Sempre inteiros. Validar com `Number.isInteger` antes de enviar.

TS

```
// src/lib/money.ts
export const toCents = (v: number | string): number => Math.round(Number(v) * 100)
export const fromCents = (c: number): number => c / 100
export const brl = (v: number | string | null | undefined) =>
  Number(v ?? 0).toLocaleString('pt-BR', { style: 'currency', currency: 'BRL' })
/** Converte entrada digitada "1.234,56" → 1234.56 (2 casas) ou null */
export function parseBrlInput(s: string): number | null {
  const n = Number(s.replace(/\./g, '').replace(',', '.').trim())
  return Number.isFinite(n) && n >= 0 ? fromCents(toCents(n)) : null
}
```

2.7 Estados de tela padrão (loading / empty / error / retry)

Estado	Quando	Componente/UX
loading	Primeira carga	`v-skeleton-loader` com o formato final; nunca spinner de página inteira após a 1ª carga

Estado	Quando	Componente/UX
refreshing	Recarga com dados já exibidos	Barra `v-progress-linear` no topo; manter dados
empty	Consulta OK com 0 linhas	Ilustração + texto + CTA permitido pela permissão (ex.: “Criar produto”)
empty-filtered	0 linhas com filtro ativo	“Nenhum resultado para os filtros” + “Limpar filtros”
forbidden	`42501`, `*_FORBIDDEN`, `*_UNAVAILABLE` de acesso	Mensagem fixa “Você não tem acesso a este recurso” e link para /app
not-found	Registro inexistente/arquivado (`maybeSingle()` → null)	Tela 404 interna com voltar à lista
error-retryable	Rede, 503, timeout, PGRST3xx	`v-alert type="error"` + botão “Tentar novamente” (mesmo request_id)
error-validation	Códigos 22023/23514 de negócio	Mensagem no campo correspondente (mapa da seção 22)
stale	`STALE_*_REVISION`	Diálogo: “Os dados mudaram. Recarregamos os valores atuais.” → recarregar e reabrir confirmação
rate-limited	HTTP 429 (Edge)	Desabilitar ação por `Retry-After` segundos com contagem regressiva

3. Mapa completo de rotas propostas

Todas as rotas abaixo são **decisões deste guia** (não existem ainda). *AppLayout**: o perfil também deve funcionar para usuário sem loja (guard não redireciona para onboarding).

Path · componente (proposto) · layout	route.name	Acesso	Permissão	Dependências de dados
`/auth/login` pages/auth/LoginPage.vue · AuthLayout	`auth-login`	Pública (guest-only)	—	supabase.auth.signInWithPassword
`/auth/signup` pages/auth/SignupPage.vue · AuthLayout	`auth-signup`	Pública (guest-only)	—	supabase.auth.signUp
`/auth/forgot-password` pages/auth/ForgotPasswordPage.vue · AuthLayout	`auth-forgot-password`	Pública	—	supabase.auth.resetPasswordForEmail
`/auth/reset-password` pages/auth/ResetPasswordPage.vue · AuthLayout	`auth-reset-password`	Sessão de recuperação	sessão ativa	supabase.auth.updateUser({password})
`/auth/callback` pages/auth/AuthCallbackPage.vue · AuthLayout	`auth-callback`	Pública (processa ?code)	—	exchangeCodeForSession, safeRedirect
`/app` pages/app/DashboardPage.vue · AppLayout	`app-dashboard`	Autenticada + loja	store.read	get_store_access, product_stock_balances, orders, get_erp_summary (se reports.read)
`/app/profile` pages/app/ProfilePage.vue · AppLayout*	`app-profile`	Autenticada	profile.update (global)	profiles SELECT/UPDATE, auth.updateUser
`/app/onboarding/plan` pages/app/onboarding/PlanStepPage.vue · OnboardingLayout	`app-onboarding-plan`	Autenticada sem loja	plans.read + store.create	plans SELECT
`/app/onboarding/store` pages/app/onboarding/StoreStepPage.vue · OnboardingLayout	`app-onboarding-store`	Autenticada sem loja	store.create	check_store_slug, create_store
`/app/store/settings` pages/app/store/StoreSettingsPage.vue · AppLayout	`app-store-settings`	Autenticada + loja	store.read (editar: store.update / branding: store.branding.manage / publicar: store.publish / arquivar: store.archive)	stores SELECT/UPDATE, Storage store-logos/store-banners, set_store_publication, archive_store
`/app/catalog/categories` pages/app/catalog/CategoriesPage.vue · AppLayout	`app-categories`	Autenticada + loja	catalog.read (+ catalog.categories.manage)	categories SELECT, create/update/archive_category
`/app/catalog/products` pages/app/catalog/ProductsPage.vue · AppLayout	`app-products`	Autenticada + loja	catalog.read	products + categories + product_images + product_stock_balances
`/app/catalog/products/new` pages/app/catalog/ProductFormPage.vue · AppLayout	`app-product-new`	Autenticada + loja	catalog.products.manage	create_product → depois mídia
`/app/catalog/products/:productId` pages/app/catalog/ProductFormPage.vue · AppLayout	`app-product-edit`	Autenticada + loja	catalog.read (editar: catalog.products.manage / mídia: catalog.products.media)	update_product, replace_product_attributes, attach/detach/reorder_product_images, archive_product
`/app/inventory` pages/app/inventory/InventoryPage.vue · AppLayout	`app-inventory`	Autenticada + loja	inventory.read (+ catalog.read)	product_stock_balances, register_inventory_movement, adjust_inventory_count, adjust_inventory_batch
`/app/inventory/movements` pages/app/inventory/MovementsPage.vue · AppLayout	`app-inventory-movements`	Autenticada + loja	inventory.read	inventory_movements, reverse_inventory_movement
`/app/orders` pages/app/orders/OrdersPage.vue · AppLayout	`app-orders`	Autenticada + loja	orders.read	orders (+ customers)
`/app/orders/:orderId` pages/app/orders/OrderDetailPage.vue · AppLayout	`app-order-detail`	Autenticada + loja	orders.read (ações: orders.terms/pay/ship/deliver/cancel/return)	orders, order_items, order_status_history, order_payments, inventory_reservations, order_return_items + 6 RPCs
`/app/customers` pages/app/customers/CustomersPage.vue · AppLayout	`app-customers`	Autenticada + loja	customers.read	customers (+ orders se orders.read)
`/app/expenses` pages/app/expenses/ExpensesPage.vue · AppLayout	`app-expenses`	Autenticada + loja	expenses.read (+ record/reverse)	expense_entries, record_expense, reverse_expense
`/app/expenses/categories` pages/app/expenses/ExpenseCategoriesPage.vue · AppLayout	`app-expense-categories`	Autenticada + loja	expenses.read (+ expenses.categories.manage)	expense_categories, create/update_expense_category

Path · componente (proposto) · layout	route.name	Acesso	Permissão	Dependências de dados
`/app/reports` pages/app/reports/ReportsPage.vue · AppLayout	`app-reports`	Autenticada + loja	reports.read	get_erp_summary, get_erp_daily, get_erp_product_performance, get_erp_expense_breakdown
`/sudo` pages/sudo/SudoHomePage.vue · SudoLayout	`sudo-home`	SUDO	can_manage_security	get_my_access, security_audit_logs
`/sudo/users` pages/sudo/SudoUsersPage.vue · SudoLayout	`sudo-users`	SUDO	can_manage_security	profiles, user_platform_roles (leitura), sudo_set_user_active
`/sudo/roles` pages/sudo/SudoRolesPage.vue · SudoLayout	`sudo-roles`	SUDO	can_manage_security	roles CRUD
`/sudo/permissions` pages/sudo/SudoPermissionsPage.vue · SudoLayout	`sudo-permissions`	SUDO	can_manage_security	permissions CRUD
`/sudo/role-permissions` pages/sudo/SudoRolePermissionsPage.vue · SudoLayout	`sudo-role-permissions`	SUDO	can_manage_security	role_permissions CRUD
`/s/:storeSlug` pages/storefront/StorefrontHomePage.vue · StorefrontLayout	`storefront-home`	Pública	— (Edge)	Edge action=catalog
`/s/:storeSlug/produto/:productSlug` pages/storefront/StorefrontProductPage.vue · StorefrontLayout	`storefront-product`	Pública	— (Edge)	Edge action=product
`/s/:storeSlug/carrinho` pages/storefront/StorefrontCartPage.vue · StorefrontLayout	`storefront-cart`	Pública	—	cart store (localStorage) + Edge catalog/product p/ revalidar
`/s/:storeSlug/checkout` pages/storefront/StorefrontCheckoutPage.vue · StorefrontLayout	`storefront-checkout`	Pública	— (Edge)	Edge action=checkout
`/s/:storeSlug/pedido/:orderId` pages/storefront/StorefrontOrderPage.vue · StorefrontLayout	`storefront-order`	Pública (token no #fragmento)	— (Edge)	Edge action=order-status
`/s/:storeSlug/:pathMatch(.*)*` pages/storefront/StorefrontNotFound.vue · StorefrontLayout	`storefront-not-found`	Pública	—	—
`/:pathMatch(.*)*` pages/NotFoundPage.vue · —	`not-found`	Pública	—	—

3.1 Meta de rota e definição

TS

```
// src/router/index.ts (PROPOSTA)
import { createRouter, createWebHistory, type RouteRecordRaw } from 'vue-router'
import type { PermissionCode } from '@/types/backend'

declare module 'vue-router' {
  interface RouteMeta {
    public?: boolean          // não exige sessão
    guestOnly?: boolean       // redireciona logado para /app
    requiresStore?: boolean   // exige loja ativa (senão → onboarding)
    noStoreOnly?: boolean     // onboarding: se já tem loja → /app
    sudo?: boolean            // exige get_my_access().can_manage_security
    perm?: PermissionCode     // permissão de loja mínima (get_store_access)
    title?: string
  }
}

const routes: RouteRecordRaw[] = [
  { path: '/', redirect: { name: 'app-dashboard' } },
  { path: '/auth', component: () => import('@/layouts/AuthLayout.vue'), children: [
    { path: 'login', name: 'auth-login', component: () => import('@/pages/auth/LoginPage.vue'), meta: { public: true,
guestOnly: true } },
    { path: 'signup', name: 'auth-signup', component: () => import('@/pages/auth/SignupPage.vue'), meta: { public: true,
guestOnly: true } },
    { path: 'forgot-password', name: 'auth-forgot-password', component: () => import('@/pages/auth/ForgotPasswordPage.vue'),
```

```
meta: { public: true } },
    { path: 'reset-password', name: 'auth-reset-password', component: () => import('@/pages/auth/ResetPasswordPage.vue'),
meta: { public: true } },
    { path: 'callback', name: 'auth-callback', component: () => import('@/pages/auth/AuthCallbackPage.vue'), meta: { public:
true } },
  ]},
  { path: '/app/onboarding', component: () => import('@/layouts/OnboardingLayout.vue'), meta: { noStoreOnly: true },
children: [
    { path: '', redirect: { name: 'app-onboarding-plan' } },
    { path: 'plan', name: 'app-onboarding-plan', component: () => import('@/pages/app/onboarding/PlanStepPage.vue') },
    { path: 'store', name: 'app-onboarding-store', component: () => import('@/pages/app/onboarding/StoreStepPage.vue') },
  ]},
  { path: '/app', component: () => import('@/layouts/AppLayout.vue'), children: [
    { path: '', name: 'app-dashboard', component: () => import('@/pages/app/DashboardPage.vue'), meta: { requiresStore:
true, perm: 'store.read' } },
    { path: 'profile', name: 'app-profile', component: () => import('@/pages/app/ProfilePage.vue') },
    { path: 'store/settings', name: 'app-store-settings', component: () =>
import('@/pages/app/store/StoreSettingsPage.vue'), meta: { requiresStore: true, perm: 'store.read' } },
    { path: 'catalog/categories', name: 'app-categories', component: () => import('@/pages/app/catalog/CategoriesPage.vue'),
meta: { requiresStore: true, perm: 'catalog.read' } },
    { path: 'catalog/products', name: 'app-products', component: () => import('@/pages/app/catalog/ProductsPage.vue'), meta:
{ requiresStore: true, perm: 'catalog.read' } },
    { path: 'catalog/products/new', name: 'app-product-new', component: () =>
import('@/pages/app/catalog/ProductFormPage.vue'), meta: { requiresStore: true, perm: 'catalog.products.manage' } },
    { path: 'catalog/products/:productId', name: 'app-product-edit', component: () =>
import('@/pages/app/catalog/ProductFormPage.vue'), props: true, meta: { requiresStore: true, perm: 'catalog.read' } },
    { path: 'inventory', name: 'app-inventory', component: () => import('@/pages/app/inventory/InventoryPage.vue'), meta: {
requiresStore: true, perm: 'inventory.read' } },
    { path: 'inventory/movements', name: 'app-inventory-movements', component: () =>
import('@/pages/app/inventory/MovementsPage.vue'), meta: { requiresStore: true, perm: 'inventory.read' } },
    { path: 'orders', name: 'app-orders', component: () => import('@/pages/app/orders/OrdersPage.vue'), meta: {
requiresStore: true, perm: 'orders.read' } },
    { path: 'orders/:orderId', name: 'app-order-detail', component: () => import('@/pages/app/orders/OrderDetailPage.vue'),
props: true, meta: { requiresStore: true, perm: 'orders.read' } },
    { path: 'customers', name: 'app-customers', component: () => import('@/pages/app/customers/CustomersPage.vue'), meta: {
requiresStore: true, perm: 'customers.read' } },
    { path: 'expenses', name: 'app-expenses', component: () => import('@/pages/app/expenses/ExpensesPage.vue'), meta: {
requiresStore: true, perm: 'expenses.read' } },
    { path: 'expenses/categories', name: 'app-expense-categories', component: () =>
import('@/pages/app/expenses/ExpenseCategoriesPage.vue'), meta: { requiresStore: true, perm: 'expenses.read' } },
    { path: 'reports', name: 'app-reports', component: () => import('@/pages/app/reports/ReportsPage.vue'), meta: {
requiresStore: true, perm: 'reports.read' } },
  ]},
  { path: '/sudo', component: () => import('@/layouts/SudoLayout.vue'), meta: { sudo: true }, children: [
    { path: '', name: 'sudo-home', component: () => import('@/pages/sudo/SudoHomePage.vue') },
    { path: 'users', name: 'sudo-users', component: () => import('@/pages/sudo/SudoUsersPage.vue') },
    { path: 'roles', name: 'sudo-roles', component: () => import('@/pages/sudo/SudoRolesPage.vue') },
    { path: 'permissions', name: 'sudo-permissions', component: () => import('@/pages/sudo/SudoPermissionsPage.vue') },
    { path: 'role-permissions', name: 'sudo-role-permissions', component: () =>
import('@/pages/sudo/SudoRolePermissionsPage.vue') },
  ]},
  { path: '/s/:storeSlug', component: () => import('@/layouts/StorefrontLayout.vue'), props: true, meta: { public: true },
children: [
    { path: '', name: 'storefront-home', component: () => import('@/pages/storefront/StorefrontHomePage.vue'), meta: {
public: true } },
    { path: 'produto/:productSlug', name: 'storefront-product', component: () =>
import('@/pages/storefront/StorefrontProductPage.vue'), props: true, meta: { public: true } },
    { path: 'carrinho', name: 'storefront-cart', component: () => import('@/pages/storefront/StorefrontCartPage.vue'), meta:
{ public: true } },
    { path: 'checkout', name: 'storefront-checkout', component: () =>
import('@/pages/storefront/StorefrontCheckoutPage.vue'), meta: { public: true } },
    { path: 'pedido/:orderId', name: 'storefront-order', component: () =>
import('@/pages/storefront/StorefrontOrderPage.vue'), props: true, meta: { public: true } },
    { path: ':pathMatch(.*)*', name: 'storefront-not-found', component: () =>
import('@/pages/storefront/StorefrontNotFound.vue'), meta: { public: true } },
  ]},
  { path: '/:pathMatch(.*)*', name: 'not-found', component: () => import('@/pages/NotFoundPage.vue'), meta: { public: true }
},
]

export const router = createRouter({
  history: createWebHistory(),
  routes,
```

```
  scrollBehavior: (to, _f, saved) => saved ?? (to.hash && !to.hash.startsWith('#token=') ? { el: to.hash } : { top: 0 }),
})
```

Fragmento #token

Na rota `/s/:storeSlug/pedido/:orderId` o token chega em `#token=…`. O `scrollBehavior` acima ignora esse hash (não é âncora). A página lê o token, o move para memória/sessionStorage e chama `router.replace({ hash: '' })`. Ver seção 19.6.

3.2 Guards

TS

```ts
// src/router/guards.ts (PROPOSTA)
import type { Router } from 'vue-router'
import { useAuthStore } from '@/stores/auth'
import { useAccessStore } from '@/stores/access'

export function installGuards(router: Router) {
  router.beforeEach(async (to) => {
    const auth = useAuthStore()
    await auth.ready                                  // getSession() inicial resolvido (seção 6.2)
    const isPublic = to.matched.every(r => r.meta.public)

    if (to.meta.guestOnly && auth.session) return { name: 'app-dashboard' }
    if (isPublic) return true
    if (!auth.session) return { name: 'auth-login', query: { redirect: to.fullPath } }

    const access = useAccessStore()
    await access.ensureLoaded()                       // get_my_access + loja ativa + get_store_access

    if (!access.my?.account_active) { await auth.signOut(); return { name: 'auth-login', query: { reason: 'inactive' } } }
    if (to.matched.some(r => r.meta.sudo)) return access.my.can_manage_security ? true : { name: 'app-dashboard' }

    const needsStore = to.matched.some(r => r.meta.requiresStore)
    const noStoreOnly = to.matched.some(r => r.meta.noStoreOnly)
    if (noStoreOnly && access.storeId) return { name: 'app-dashboard' }
    if (needsStore && !access.storeId) {
      return access.can('store.create') ? { name: 'app-onboarding-plan' } : { name: 'app-profile' }
    }
    const perm = [...to.matched].reverse().find(r => r.meta.perm)?.meta.perm
    if (perm && !access.hasStore(perm)) return { name: 'app-dashboard', query: { denied: perm } }
    return true
  })
}
```

4. Matriz de permissões

Fonte: seeds de `role_permissions` dos módulos 001–007 (estado efetivo no manifesto). **Permissões globais** vêm de `user_platform_roles` (USER/SUDO) e são retornadas por `get_my_access()`. **Permissões de loja** vêm do papel contextual OWNER (dono da loja) ou SUDO + `store.admin`, e são retornadas por `get_store_access(store_id)`.

Permissão	Escopo	USER	OWNER (contextual)	SUDO	Descrição (banco)
`auth.access`	G	✅	(seed)	✅	Acessar area autenticada da aplicacao.
`profile.update`	G	✅	(seed)	✅	Editar o proprio perfil.
`security.manage`	G	—	—	✅	Indicador de acesso ao painel interno de seguranca.
`plans.read`	G	✅	(seed)	✅	Consultar plano gratuito disponivel.
`plans.manage`	G	—	—	✅	SUDO: editar nome, descricao e disponibilidade do plano.
`store.create`	G	✅	—	✅	Criar a primeira loja ativa pelo onboarding.
`store.read`	L	—	✅	✅	Consultar a loja autorizada.
`store.update`	L	—	✅	✅	Editar dados e personalizacao da loja.
`store.publish`	L	—	✅	✅	Publicar ou despublicar a loja.
`store.archive`	L	—	✅	✅	Arquivar a loja.
`store.branding.manage`	L	—	✅	✅	Enviar, vincular e remover logo/banner.
`store.admin`	G	—	—	✅	SUDO: administrar lojas de outros proprietarios.
`catalog.read`	L	—	✅	✅	Consultar o catalogo privado da loja.
`catalog.categories.manage`	L	—	✅	✅	Criar, editar e arquivar categorias/subcategorias.
`catalog.products.manage`	L	—	✅	✅	Criar, editar, arquivar produtos e substituir atributos.
`catalog.products.media`	L	—	✅	✅	Enviar, vincular, ordenar e desvincular imagens de produtos.
`inventory.read`	L	—	✅	✅	Consultar saldo, valor de estoque e historico de movimentos.
`inventory.entry`	L	—	✅	✅	Registrar entradas manuais de estoque.
`inventory.exit`	L	—	✅	✅	Registrar saidas manuais de estoque.
`inventory.adjust`	L	—	✅	✅	Ajustar contagem fisica individual ou em lote.
`inventory.reverse`	L	—	✅	✅	Reverter o ultimo movimento ainda nao revertido.
`storefront.read`	Pública (OWNER)	—	✅	✅	Disponibilizar a vitrine publica da loja.
`orders.create`	Pública (OWNER)	—	✅	✅	Habilitar criacao publica de pedidos pelo WhatsApp.
`orders.read`	L	—	✅	✅	Consultar pedidos privados da loja.
`customers.read`	L	—	✅	✅	Consultar contatos privados da loja.
`orders.terms`	L	—	✅	✅	Confirmar frete manual antes do pagamento.
`orders.pay`	L	—	✅	✅	Confirmar recebimento manual e baixar estoque.
`orders.ship`	L	—	✅	✅	Registrar envio de pedido pago.
`orders.deliver`	L	—	✅	✅	Registrar entrega de pedido enviado.
`orders.cancel`	L	—	✅	✅	Cancelar pendente ou pago ainda nao enviado.
`orders.return`	L	—	✅	✅	Receber devolucao total e registrar reembolso manual.
`expenses.read`	L	—	✅	✅	Consultar categorias e lancamentos de despesas.
`expenses.categories.manage`	L	—	✅	✅	Criar, renomear e ativar/desativar categorias.
`expenses.record`	L	—	✅	✅	Declarar despesas pagas e compras de estoque.
`expenses.reverse`	L	—	✅	✅	Corrigir lancamento ou declarar reembolso recebido.
`reports.read`	L	—	✅	✅	Consultar ERP gerencial com valores financeiros e custos.

Como ler

- **G** = verificada por `has_global_permission` (vem de `get_my_access().permissions`). **L** = verificada por `has_store_permission(store_id, …)` (vem de `get_store_access().permissions`).
- Toda permissão de loja também exige conta ativa e `auth.access` global.
- **(seed)**: o papel OWNER recebe a permissão em `role_permissions`, mas ela é avaliada como GLOBAL (via papel USER do usuário); o OWNER não a acrescenta. `storefront.read` e `orders.create` são avaliadas pela vitrine pública sobre o papel OWNER do dono (loja publicada + dono ativo). O lojista não as "usa" na UI.
- SUDO só acessa uma loja de terceiro se tiver `store.admin`; nesse caso `get_store_access` retorna `role: 'SUDO'`. O SUDO também pode criar sua própria loja (é OWNER dela).
- Desativar uma permissão em `role_permissions` (painel SUDO) altera imediatamente o comportamento de todas as RPCs/policies — o frontend deve sempre refletir `get_store_access`, nunca hardcode por papel.

4.1 Matriz rota × ação × permissão

Tela	Ação na UI	Permissão exigida	Backend
Configurações da loja	Editar dados/cores/redes	store.update	UPDATE stores
	Enviar/remover logo e banner	store.branding.manage (+ store.update para gravar o path)	Storage + UPDATE stores
	Publicar/despublicar	store.publish	set_store_publication
	Arquivar loja	store.archive	archive_store
Categorias	Criar/editar/ordenar/arquivar	catalog.categories.manage	create/update/archive_category
Produtos	Criar/editar/atributos/arquivar	catalog.products.manage	create/update_product, replace_product_attributes, archive_product
	Imagens	catalog.products.media	Storage product-images + attach/detach/reorder
	Coluna de estoque na lista	inventory.read	product_stock_balances
Estoque	Entrada	inventory.entry	register_inventory_movement(ENTRY)
	Saída manual	inventory.exit	register_inventory_movement(EXIT)
	Contagem individual / lote	inventory.adjust	adjust_inventory_count / adjust_inventory_batch
Movimentações	Reverter último	inventory.reverse	reverse_inventory_movement
Pedido	Confirmar frete	orders.terms	update_order_terms
	Confirmar pagamento	orders.pay + inventory.exit	confirm_order_payment
	Enviar	orders.ship	ship_order
	Entregue	orders.deliver	deliver_order
	Cancelar	orders.cancel (+ inventory.entry se PAID)	cancel_order
	Devolução	orders.return (+ inventory.entry se houver RESTOCKED)	return_order
Despesas	Lançar	expenses.record	record_expense
	Corrigir/Reembolso	expenses.reverse	reverse_expense
Categorias de despesa	Criar/renomear/ativar	expenses.categories.manage	create/update_expense_category
Relatórios	Ver ERP	reports.read	get_erp_*
SUDO	Tudo	can_manage_security	CRUD roles/permissions/role_permissions, sudo_set_user_active

5. Camada de services e stores

5.1 Services propostos (1 por domínio)

Arquivo	Responsabilidade	Fontes backend
services/auth.service.ts	Wrapper do SDK Auth (login, cadastro, recuperação, troca de senha/e-mail, logout)	supabase.auth.*
services/access.service.ts	Acesso global e por loja; loja ativa do usuário	get_my_access, get_store_access, stores SELECT
services/profile.service.ts	Ler/atualizar o próprio perfil	profiles SELECT/UPDATE
services/plans.service.ts	Listar plano FREE; SUDO edita nome/descrição/ativo	plans SELECT/UPDATE
services/stores.service.ts	Onboarding, configurações, publicação, arquivamento, branding	check_store_slug, create_store, stores UPDATE, set_store_publication, archive_store, Storage
services/catalog.service.ts	Categorias, produtos, atributos, imagens	10 RPCs do módulo 03 + SELECTs
services/inventory.service.ts	Saldos, movimentos, ajustes	4 RPCs do módulo 04 + product_stock_balances, inventory_movements
services/orders.service.ts	Listagem, detalhe e 6 transições	6 RPCs do módulo 06 + SELECTs
services/customers.service.ts	Lista de contatos + pedidos do contato	customers, orders SELECT
services/expenses.service.ts	Categorias de despesa, lançamentos, estornos	4 RPCs de despesa do módulo 07
services/reports.service.ts	ERP gerencial	4 RPCs get_erp_*
services/sudo.service.ts	Usuários, papéis, permissões, matriz, auditoria	profiles, user_platform_roles, roles, permissions, role_permissions, security_audit_logs, sudo_set_user_active
services/storefront.service.ts	Cliente HTTP da Edge (sem sessão)	POST functions/v1/storefront-api

5.2 Stores Pinia propostas

Store	Estado	Ações principais	Persistência
useAuthStore	session, user, ready (Promise)	init(), signIn, signUp, signOut, refreshUser	SDK (localStorage do supabase-js)
useAccessStore	my: MyAccess, store: StoresRow|null, storeAccess: StoreAccess, storeId	ensureLoaded(), reload(), can(perm global), hasStore(perm)	memória; invalidar em SIGNED_IN/SIGNED_OUT/USER_UPDATED
useStoreSettingsStore	store, saving	load, update(patch), publish, archive, uploadLogo/Banner	memória
useCartStore (vitrine)	items: {product_id, slug, name, price, image_path, quantity}[] por loja	add, setQty, remove, clear, revalidate()	`localStorage[vibestore:cart:{slug}]` (preço é apenas exibição)
useStorefrontStore	catalog: CatalogData, filters	loadCatalog, loadProduct	memória (cache curto por slug)
useCheckoutStore	draft (form), pending {request_id, client_secret, input}	submit(), retry(), clearSensitive()	`sessionStorage[vibestore:checkout:{slug}]` até sucesso

TS

```ts
// src/stores/access.ts (PROPOSTA)
import { defineStore } from 'pinia'
import { rpc } from '@/lib/rpc'
import { supabase } from '@/lib/supabase'
import type { MyAccess, StoreAccess, PermissionCode } from '@/types/backend'
import type { StoresRow } from '@/types/backend'

export const useAccessStore = defineStore('access', {
  state: () => ({
    my: null as MyAccess | null,
```

```
    store: null as StoresRow | null,
    storeAccess: null as StoreAccess,
    loading: null as Promise<void> | null,
  }),
  getters: {
    storeId: (s) => s.store?.id ?? null,
    can: (s) => (p: PermissionCode) => !!s.my?.permissions.includes(p),            // GLOBAL
    hasStore: (s) => (p: PermissionCode) => !!s.storeAccess?.permissions.includes(p), // LOJA
    isSudoPanel: (s) => !!s.my?.can_manage_security,
  },
  actions: {
    ensureLoaded() { return this.loading ??= this.reload() },
    async reload() {
      this.my = await rpc<object, MyAccess>('get_my_access')
      const uid = this.my.user_id
      // Loja própria: no máximo UMA não arquivada (índice stores_one_live_per_owner).
      // Filtrar por owner_id evita listar lojas de terceiros quando o usuário é SUDO (store.admin).
      const { data, error } = await supabase.from('stores')
        .select('*').eq('owner_id', uid!).is('archived_at', null).maybeSingle()
      if (error) throw error
      this.store = data
      this.storeAccess = data ? await rpc<{ p_store_id: string }, StoreAccess>('get_store_access', { p_store_id: data.id })
: null
    },
    reset() { this.$reset() },
  },
})
```

6. Autenticação (Supabase Auth SDK)

Autenticação usa **exclusivamente o SDK Supabase Auth** (nenhuma RPC). O banco cria automaticamente `profiles` (com `full_name` de `raw_user_meta_data.full_name`, truncado a 120) e o papel global `USER` via trigger `vibestore_auth_user_created`. `profiles` não guarda e-mail nem senha; o e-mail exibido vem de `session.user.email`.

6.1 Service de autenticação

TS

```ts
// src/services/auth.service.ts (PROPOSTA)
import { supabase } from '@/lib/supabase'

const APP_URL = import.meta.env.VITE_APP_URL as string
const callbackUrl = (next: string) =>
  `${APP_URL}/auth/callback?next=${encodeURIComponent(next)}`

export const authService = {
  signIn: (email: string, password: string) =>
    supabase.auth.signInWithPassword({ email, password }),

  signUp: (email: string, password: string, fullName: string) =>
    supabase.auth.signUp({
      email,
      password,
      options: {
        data: { full_name: fullName.trim().slice(0, 120) },      // lido pelo trigger handle_new_auth_user
        emailRedirectTo: callbackUrl('/app/onboarding/plan'),
      },
    }),

  requestPasswordReset: (email: string) =>
    supabase.auth.resetPasswordForEmail(email, { redirectTo: callbackUrl('/auth/reset-password') }),

  updatePassword: (password: string) => supabase.auth.updateUser({ password }),

  /** Troca de e-mail: Supabase envia confirmação; o e-mail muda só após o clique. */
  updateEmail: (email: string) =>
    supabase.auth.updateUser({ email }, { emailRedirectTo: callbackUrl('/app/profile') }),

  signOut: () => supabase.auth.signOut(),                       // escopo 'global' por padrão
  getSession: () => supabase.auth.getSession(),                 // leitura local (pode estar expirada)
  getUser: () => supabase.auth.getUser(),                       // valida o JWT no servidor Auth
}
```

6.2 Store de autenticação e onAuthStateChange (sem await dentro do callback)

> **Regra obrigatória**
>
> Não chamar `await supabase.…` (nem RPC, nem `getUser`) **dentro** do callback de `onAuthStateChange`: o SDK mantém um lock interno durante o callback e chamadas aguardadas ali podem causar deadlock. Atualize estado síncrono e agende o trabalho assíncrono com `setTimeout(…, 0)` (ou `queueMicrotask`).

TS

```ts
// src/stores/auth.ts (PROPOSTA)
import { defineStore } from 'pinia'
import type { Session, User } from '@supabase/supabase-js'
import { supabase } from '@/lib/supabase'
import { useAccessStore } from './access'
import { router } from '@/router'

export const useAuthStore = defineStore('auth', {
  state: () => ({ session: null as Session | null, user: null as User | null, ready: null as Promise<void> | null }),
  actions: {
    init() {
      if (this.ready) return this.ready
      this.ready = (async () => {
```

```
        const { data } = await supabase.auth.getSession()      // FORA do callback: permitido
        this.session = data.session
        this.user = data.session?.user ?? null
      })()

      supabase.auth.onAuthStateChange((event, session) => {
        // 1) SOMENTE atualização síncrona de estado aqui dentro
        this.session = session
        this.user = session?.user ?? null

        // 2) Trabalho assíncrono agendado para FORA do callback
        setTimeout(() => {
          const access = useAccessStore()
          if (event === 'SIGNED_OUT') {
            access.reset()
            localStorage.removeItem('vibestore:active-store')
            router.replace({ name: 'auth-login' })
          } else if (event === 'SIGNED_IN' || event === 'USER_UPDATED') {
            access.loading = null                              // força novo get_my_access na próxima rota
          } else if (event === 'PASSWORD_RECOVERY') {
            router.replace({ name: 'auth-reset-password' })
          }
        }, 0)
      })
      return this.ready
    },
    async signOut() { await supabase.auth.signOut() },
  },
})

// main.ts: useAuthStore(pinia).init() ANTES de app.use(router) / router.isReady()
```

6.3 Redirect interno seguro (anti open-redirect)

TS

```
// src/lib/safeRedirect.ts
const ALLOWED_PREFIXES = ['/app', '/sudo', '/auth/reset-password', '/s/']

/** Aceita SOMENTE caminhos relativos internos. Qualquer outro valor → fallback. */
export function safeRedirect(raw: unknown, fallback = '/app'): string {
  if (typeof raw !== 'string' || raw.length === 0 || raw.length > 512) return fallback
  let value: string
  try { value = decodeURIComponent(raw) } catch { return fallback }
  if (!value.startsWith('/') || value.startsWith('//') || value.startsWith('/\\')) return fallback
  if (/[\u0000-\u001f\\]/.test(value)) return fallback            // controle e barra invertida
  let url: URL
  try { url = new URL(value, window.location.origin) } catch { return fallback }
  if (url.origin !== window.location.origin) return fallback     // ex.: '/\t//evil.com'
  if (url.pathname.startsWith('/auth/callback')) return fallback  // evita loop
  if (!ALLOWED_PREFIXES.some(p => url.pathname === p.replace(/\/$/, '') || url.pathname.startsWith(p))) return fallback
  return url.pathname + url.search                                // descarta hash (nunca propagar #token)
}
```

6.4 /auth/callback (confirmação de cadastro, recuperação e troca de e-mail)

Todos os links de e-mail apontam para `{APP_URL}/auth/callback?next=…`. Com `flowType: 'pkce'` o Supabase devolve `?code=…`; a página troca o code explicitamente (o cliente usa `detectSessionInUrl: false` para não haver troca dupla). Em caso de erro o Supabase devolve `?error=…&error_description=…` (às vezes no hash).

VUE

```
<!-- src/pages/auth/AuthCallbackPage.vue (PROPOSTA) -->
<script setup lang="ts">
import { onMounted, ref } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import { supabase } from '@/lib/supabase'
import { safeRedirect } from '@/lib/safeRedirect'

const route = useRoute(); const router = useRouter()
```

```
const state = ref<'loading' | 'error'>('loading'); const message = ref('')

onMounted(async () => {
  const q = new URLSearchParams(window.location.search)
  const h = new URLSearchParams(window.location.hash.slice(1))
  const err = q.get('error_description') ?? h.get('error_description')
  if (err) { state.value = 'error'; message.value = 'Link inválido ou expirado. Solicite um novo.'; return }

  const code = q.get('code')
  if (code) {
    const { error } = await supabase.auth.exchangeCodeForSession(code)
    if (error) { state.value = 'error'; message.value = 'Não foi possível validar o link. Ele pode ter expirado ou sido
aberto em outro navegador.'; return }
  }
  const { data } = await supabase.auth.getSession()
  if (!data.session) { router.replace({ name: 'auth-login' }); return }
  router.replace(safeRedirect(route.query.next, '/app'))     // limpa ?code da URL
})
</script>
```

> **PKCE e outro navegador**
>
> No PKCE o `code_verifier` fica no armazenamento do navegador que iniciou o fluxo. Se o usuário abrir o link de e-mail em outro dispositivo/navegador a troca falha: mostre “abra o link no mesmo navegador ou faça login”. Para confirmação de cadastro, o usuário pode simplesmente fazer login após confirmar.

6.5 Telas de autenticação

Tela	Campos / validação cliente	Chamada	Sucesso	Erros a tratar
/auth/login	email (type=email, obrigatório), senha (obrigatória)	signInWithPassword	`router.replace(safeRedirect(route.query.redirect))`	`invalid_credentials` → “E-mail ou senha inválidos”; `email_not_confirmed` → oferecer reenviar (`auth.resend({type:'signup', email})`); `over_request_rate_limit`; conta inativa (get_my_access.account_active=false) → signOut + aviso
/auth/signup	nome (2..120), email, senha (mín. definida no projeto Auth; sugerido ≥ 8), confirmação	signUp	Se `data.session` existir (confirmação desligada) → onboarding; senão tela “Verifique seu e-mail”	`user_already_exists` / `email_exists`; `weak_password`; quando a confirmação está ativa o Supabase pode responder sucesso para e-mail existente (anti-enumeração) — usar mensagem neutra
/auth/forgot-password	email	resetPasswordForEmail	Sempre mensagem neutra “Se existir uma conta, enviaremos um link”	rate limit
/auth/reset-password	nova senha + confirmação	updateUser({ password })	Toast + `router.replace('/app')`	Sem sessão → “link expirado” + link para forgot-password; `same_password`; `weak_password`
/auth/callback	—	exchangeCodeForSession	safeRedirect(next)	link expirado/inválido

TS

```
// LoginPage.vue — trecho do submit
const { execute, loading, error } = useAsync(async () => {
  const { error } = await authService.signIn(form.email.trim(), form.password)
  if (error) throw error
  await useAccessStore().reload()
  if (!useAccessStore().my?.account_active) { await authService.signOut(); throw new Error('ACCOUNT_INACTIVE') }
  await router.replace(safeRedirect(route.query.redirect, '/app'))
})
```

6.6 Configuração necessária no Supabase Auth

- **Site URL** = `VITE_APP_URL`.
- **Redirect URLs**: `{APP_URL}/auth/callback` (com e sem query; usar `{APP_URL}/auth/callback**`) e as URLs equivalentes de desenvolvimento (ex.: `http://localhost:5173/auth/callback**`).
- Templates de e-mail (Confirm signup, Reset password, Change email) usando `{{ .ConfirmationURL }}` (padrão), que respeita o `redirectTo` enviado.
- Política de senha mínima e “Confirm email” definidos no painel; o frontend lê o comportamento a partir da resposta de `signUp` (presença de `session`).

7. Acesso, sessão e perfil

7.1 Fluxo de inicialização após login

1. `getSession()` (store auth) → se não há sessão, rota protegida redireciona para login.
2. `get_my_access()` → `account_active`, `can_manage_security`, papéis/permissões globais.
3. `stores` SELECT `owner_id = user_id AND archived_at IS NULL` → `maybeSingle()`: loja ativa ou null.
4. Se há loja: `get_store_access(store.id)` → permissões efetivas na loja (OWNER).
5. Sem loja e com `store.create` → onboarding. SUDO com `can_manage_security` vê o item de menu “Administração”.

> **OWNER é contextual**
>
> `get_my_access().roles` nunca contém OWNER (a tabela `user_platform_roles` só aceita USER/SUDO). Para saber se o usuário é dono, use `get_store_access(store_id).role === 'OWNER'`. Não existe “OWNER” atribuível pelo painel.

7.2 /app/profile

Campo	Origem	Regra	Gravação
E-mail	session.user.email	Somente leitura no perfil; troca via “Alterar e-mail” (Auth)	authService.updateEmail → confirmação por e-mail
Nome completo	profiles.full_name	≤ 120 caracteres (CHECK). Pode ser vazio (default '')	UPDATE profiles
Telefone	profiles.phone	null ou E.164 `^\+[1-9][0-9]{7,14}$` ; não é credencial	UPDATE profiles
Avatar	profiles.avatar_path	**Bucket de avatar não existe.** Mostrar iniciais; não exibir upload; não gravar caminho	—
Senha	Auth	nova + confirmação	authService.updatePassword

TS

```ts
// src/services/profile.service.ts (PROPOSTA)
import { supabase } from '@/lib/supabase'
import type { ProfilesRow, ProfileUpdatePatch } from '@/types/backend'

export const profileService = {
  async get(userId: string): Promise<ProfilesRow> {
    const { data, error } = await supabase.from('profiles')
      .select('id, full_name, phone, avatar_path, is_active, created_at, updated_at')
      .eq('id', userId).single()
    if (error) throw error
    return data
  },
  async update(userId: string, patch: Pick<ProfileUpdatePatch, 'full_name' | 'phone'>) {
    const { data, error } = await supabase.from('profiles')
      .update({ full_name: patch.full_name?.trim() ?? undefined, phone: patch.phone || null })
      .eq('id', userId).select('id, full_name, phone, updated_at')
    if (error) throw error                       // 23514 profiles_phone_check / profiles_full_name_check
    if (!data?.length) throw new Error('PROFILE_UPDATE_FORBIDDEN')   // RLS: conta inativa ou sem profile.update
    return data[0]
  },
}
```

Aceitação: salvar nome/telefone válidos persiste e reflete no menu; telefone fora do E.164 mostra erro no campo antes de enviar; nenhuma opção de upload de avatar; troca de senha com sessão ativa funciona; e-mail não é editável por UPDATE em profiles.

8. Onboarding (plano + loja)

Pré-condição: usuário autenticado, conta ativa, permissão global `store.create` (papel USER) e **nenhuma loja não arquivada**. Uma loja ativa por usuário.

8.1 Passo 1 — /app/onboarding/plan

TS

```
const { data: plans, error } = await supabase
  .from('plans')
  .select('id, code, name, description, price, currency, is_active')
  .eq('is_active', true)          // usuário comum só enxerga ativos (policy plans_select)
  .eq('code', 'FREE')
// Esperado: exatamente 1 linha. 0 linhas → estado "Cadastro de lojas indisponível" (create_store retornaria
PLAN_UNAVAILABLE).
```

UI: card único “Gratuito — R$ 0,00” com nome/descrição vindos do banco (SUDO pode editá-los). Botão “Continuar” guarda `plan_id` no estado do wizard (memória + sessionStorage) e navega para `/app/onboarding/store`. Não exibir comparativo de planos pagos.

8.2 Passo 2 — /app/onboarding/store

Campo	Obrigatório	Validação no cliente (espelha o banco)	Envio
Nome da loja	Sim	trim 2..120	name
Título da vitrine	Não	trim 2..160; vazio → usa o nome	title (null se vazio)
Subtítulo	Não	≤ 300	subtitle
Endereço (slug)	Sim	lower, 3..63, `^[a-z0-9]+(-[a-z0-9]+)*$`, não reservado; sugerir a partir do nome; checar com debounce 400 ms via `check_store_slug`	slug
WhatsApp	Sim	E.164 `^\+[1-9][0-9]{7,14}$` (UI pode mascarar +55 DDD número e converter)	whatsapp_number
CNPJ	Não	1..32, texto livre (sem validação fiscal)	cnpj
E-mail de contato	Não	≤ 254, formato `x@y.z`	email
Cor primária / secundária	Não	`#RRGGBB` (v-color-picker modo hex)	primary_color / secondary_color
Redes sociais	Não	chaves instagram/facebook/tiktok/youtube/website; URL `https://` sem espaços, ≤ 2048	social_links (omitir chaves vazias)

TS

```
// src/services/stores.service.ts — onboarding (PROPOSTA)
import { rpc } from '@/lib/rpc'
import { stablePending, clearPending } from '@/lib/pending'
import type { CheckStoreSlugResult, CreateStoreInput, CreateStoreResult } from '@/types/backend'

export const RESERVED_SLUGS = ['admin','api','app','auth','login','signup','dashboard','settings','sudo','supabase']

export const checkSlug = (slug: string) =>
  rpc<{ p_slug: string }, CheckStoreSlugResult>('check_store_slug', { p_slug: slug })

export async function createStore(planId: string, input: CreateStoreInput) {
  const clean: CreateStoreInput = {
    ...input,
    slug: input.slug.trim().toLowerCase(),
    title: input.title?.trim() || null,
    social_links: Object.fromEntries(Object.entries(input.social_links ?? {}).filter(([, v]) => !!v)),
  }
  const op = stablePending('onboarding:create-store', { planId, clean })
  const result = await rpc<object, CreateStoreResult>('create_store', {
    p_request_id: op.requestId, p_plan_id: planId, p_input: clean,
  })
  clearPending('onboarding:create-store')
```

```
  return result   // created=false é replay bem-sucedido
}
```

Código	Tratamento na UI
SLUG_ALREADY_EXISTS	Campo slug: “Endereço já em uso” + nova sugestão; gerar NOVO request_id (payload mudou)
INVALID_STORE_SLUG	Campo slug: regra de formato/reservado
INVALID_WHATSAPP_NUMBER	Campo WhatsApp
INVALID_SOCIAL_LINKS	Seção redes: “use links https:// completos”
INVALID_STORE_NAME_OR_TITLE	Campos nome/título
ACTIVE_STORE_ALREADY_EXISTS	Recarregar acesso e ir para /app (outra aba já criou)
PLAN_UNAVAILABLE	Voltar ao passo 1 com aviso
STORE_CREATE_FORBIDDEN / ACCOUNT_UNAVAILABLE	Tela de acesso negado
ONBOARDING_ALREADY_ARCHIVED	Limpar pending e gerar novo request_id
23514 stores_email_check / stores_primary_color_check …	Campo correspondente

Sucesso: `await access.reload()` e `router.replace({ name: 'app-store-settings', query: { welcome: '1' } })`. A loja nasce **não publicada**; o checklist pós-onboarding deve guiar: logo/banner → categoria → produto ACTIVE → entrada de estoque → publicar.

Aceitação: slug indisponível é sinalizado antes do envio; duplo clique/refresh durante o envio não cria duas lojas (mesmo request_id → `created:false`); usuário que já tem loja é redirecionado do onboarding para /app.

9. Configurações da loja, branding e publicação

Leitura: `stores` (policy `stores_select`: não arquivada + `store.read`). Escrita de dados: **UPDATE direto** apenas nas 13 colunas com GRANT: `name, title, subtitle, slug, cnpj, email, whatsapp_number, logo_path, banner_path, primary_color, secondary_color, social_links, is_published`. Colunas como `owner_id, plan_id, onboarding_*` e `archived_at` não podem ser alteradas.

TS

```
// src/services/stores.service.ts — configurações
export async function updateStore(storeId: string, patch: StoreUpdatePatch) {
  const { is_published, ...rest } = patch          // publicação vai pela RPC (mensagem/regra explícitas)
  const { data, error } = await supabase.from('stores').update(rest)
    .eq('id', storeId).select('*')
  if (error) throw toBackendError(error)           // 23505 stores_slug_key, 23514 *_check, STORE_BRANDING_FORBIDDEN,
LOGO_OBJECT_NOT_FOUND…
  if (!data?.length) throw new BackendError('STORE_UNAVAILABLE', '42501')   // sem store.update ou arquivada
  return data[0]
}
export const setPublication = (storeId: string, published: boolean) =>
  rpc<SetStorePublicationArgs, SetStorePublicationResult>('set_store_publication', { p_store_id: storeId, p_published:
published })
export const archiveStore = (storeId: string) =>
  rpc<ArchiveStoreArgs, ArchiveStoreResult>('archive_store', { p_store_id: storeId })
```

Alterar o slug

O slug é único inclusive entre lojas arquivadas e muda a URL pública `/s/:slug`. Links de acompanhamento de pedidos já enviados (`/s/{slug-antigo}/pedido/…`) deixam de funcionar porque `get_public_order_status` filtra por slug atual. Exigir confirmação explícita com esse aviso. O banco não tem regra adicional.

9.1 Logo e banner (Storage)

Bucket	Limite	MIME	Caminho obrigatório	Coluna
store-logos (público)	2 MB (2097152)	image/png, image/jpeg, image/webp	`{store_id}/{uuid}.{png|jpg|jpeg|webp}`	stores.logo_path
store-banners (público)	5 MB (5242880)	idem	`{store_id}/{uuid}.{ext}`	stores.banner_path

1. Validar no cliente: tipo MIME e tamanho; extensão a partir do MIME (`image/jpeg→jpg`).
2. Upload imutável: `storage.from(bucket).upload(path, file, { upsert: false, contentType, cacheControl: '31536000' })`. Não existe UPDATE de objetos (sem policy) — nunca sobrescrever.
3. Gravar o caminho RELATIVO na loja: `update({ logo_path: path })`. O trigger valida que o objeto existe e pertence à loja (`LOGO_OBJECT_NOT_FOUND`, `INVALID_LOGO_PATH`).
4. Somente após o UPDATE bem-sucedido, apagar o arquivo anterior: `storage.from(bucket).remove([oldPath])`. A policy de DELETE só permite remover objetos **não vinculados** à loja.
5. Remover logo: `update({ logo_path: null })` → depois `remove([oldPath])`.
6. Falha no UPDATE após upload → apagar o objeto recém-enviado (órfão).

TS

```
// src/lib/media.ts
const EXT: Record<string, string> = { 'image/png': 'png', 'image/jpeg': 'jpg', 'image/webp': 'webp' }
export const LIMITS = { 'store-logos': 2_097_152, 'store-banners': 5_242_880, 'product-images': 5_242_880 } as const
export type Bucket = keyof typeof LIMITS

export function assertImage(file: File, bucket: Bucket) {
  if (!EXT[file.type]) throw new Error('INVALID_IMAGE_TYPE')
  if (file.size > LIMITS[bucket]) throw new Error('IMAGE_TOO_LARGE')
}
export const storeMediaPath = (storeId: string, file: File) => `${storeId}/${crypto.randomUUID()}.${EXT[file.type]}`
export const productMediaPath = (storeId: string, productId: string, file: File) =>
  `${storeId}/${productId}/${crypto.randomUUID()}.${EXT[file.type]}`
```

```
/** Banco guarda SÓ o caminho; a URL é montada no cliente (bucket público). */
export const publicUrl = (bucket: Bucket, path: string | null) =>
  path ? supabase.storage.from(bucket).getPublicUrl(path).data.publicUrl : null

// Troca de logo (ordem segura)
export async function replaceStoreMedia(storeId: string, kind: 'logo' | 'banner', file: File, oldPath: string | null) {
  const bucket: Bucket = kind === 'logo' ? 'store-logos' : 'store-banners'
  assertImage(file, bucket)
  const path = storeMediaPath(storeId, file)
  const up = await supabase.storage.from(bucket).upload(path, file, { upsert: false, contentType: file.type })
  if (up.error) throw up.error
  try {
    await updateStore(storeId, kind === 'logo' ? { logo_path: path } : { banner_path: path })
  } catch (e) {
    await supabase.storage.from(bucket).remove([path])       // limpa órfão
    throw e
  }
  if (oldPath) await supabase.storage.from(bucket).remove([oldPath])   // best-effort
  return path
}
```

9.2 Publicação e arquivamento

Ação	Pré-condições/efeitos	UX
Publicar	store.publish; loja não arquivada. A vitrine só responde se o dono estiver ativo e o papel OWNER tiver storefront.read	Switch “Loja publicada”. Antes de publicar, mostrar checklist (≥1 produto ACTIVE com estoque). Mostrar link público `{APP_URL}/s/{slug}` com copiar.
Despublicar	Vitrine e checkout passam a 404/STORE_UNAVAILABLE; acompanhamento de pedidos continua funcionando	Confirmação simples
Arquivar	store.archive; sem saldo de estoque (STORE_HAS_STOCK) e sem pedidos PENDING/PAID/SHIPPED (STORE_HAS_OPEN_ORDERS). Irreversível pelo app	Zona de perigo: digitar o slug para confirmar. Após sucesso: `access.reload()` → onboarding (pode criar outra loja; slug antigo segue reservado)

Aceitação: usuário sem `store.update` vê formulário somente leitura; troca de logo nunca deixa a loja apontando para objeto inexistente; arquivar com estoque mostra mensagem clara e link para Estoque.

10. Catálogo: categorias

Aspecto	Contrato
Leitura	`categories` — policy: não arquivada + `catalog.read`. Arquivadas nunca aparecem.
Escrita	Somente RPC: `create_category`, `update_category`, `archive_category`.
Hierarquia	Máximo 2 níveis (raiz → subcategoria). Pai deve ser raiz e viva.
Código	`CAT-000001` gerado no servidor, imutável, exibido como referência.
Ordenação	`sort_order` inteiro 0..1000000 (empates por id).

TS

```
// Lista hierárquica da loja (sem paginação: volume pequeno; até ~centenas)
const { data, error } = await supabase.from('categories')
  .select('id, parent_id, code, name, slug, is_active, sort_order, created_at, updated_at')
  .eq('store_id', storeId)
  .order('sort_order', { ascending: true }).order('name', { ascending: true })

// montar árvore no cliente
const roots = data.filter(c => !c.parent_id)
const children = (id: string) => data.filter(c => c.parent_id === id)

// Criar
const op = stablePending(`category:new`, form)
await rpc<CreateCategoryArgs, CreateCategoryResult>('create_category', {
  p_store_id: storeId, p_request_id: op.requestId,
  p_input: { name: form.name.trim(), slug: form.slug || null, parent_id: form.parentId ?? null,
             is_active: form.isActive, sort_order: form.sortOrder ?? 0 },
})

// Editar (patch parcial) — enviar SOMENTE campos alterados
await rpc<UpdateCategoryArgs, UpdateCategoryResult>('update_category', {
  p_category_id: id, p_patch: { name: 'Camisetas', is_active: false },
})

// Reordenar (drag-and-drop): N chamadas update_category com sort_order = índice*10
for (const [i, cat] of ordered.entries())
  await rpc('update_category', { p_category_id: cat.id, p_patch: { sort_order: i * 10 } })
```

Interação	Regra de UI
Excluir	Não existe DELETE: botão “Arquivar”. Desabilitar se a categoria tem filhos vivos ou produtos (contar no cliente: `products.select('id', {count:'exact', head:true}).eq('category_id', id)`). Mensagens: CATEGORY_HAS_LIVE_CHILDREN / CATEGORY_HAS_LIVE_PRODUCTS.
Mover para subcategoria	Seletor de pai lista apenas raízes ≠ própria; desabilitar se a categoria tem filhos (CATEGORY_WITH_CHILDREN_CANNOT_BECOME_CHILD).
Slug	Opcional na criação (gerado pelo servidor); na edição, se alterado, validar `^[a-z0-9]+(-[a-z0-9]+)*$` 2..100. Conflito: `CATEGORY_SLUG_EXISTS` (create) / `23505:categories_store_slug` (update).
Inativa	Badge “Oculta da vitrine”; produtos dela e das filhas somem da vitrine.
Empty	“Nenhuma categoria. Produtos podem ficar sem categoria.” + CTA criar (se permissão).

Aceitação: árvore com 2 níveis; arquivar categoria com produto mostra erro amigável; reordenar persiste após reload; usuário sem `catalog.categories.manage` vê lista sem ações.

11. Catálogo: produtos, SKU, atributos e imagens

11.1 Lista /app/catalog/products

TS

```
// Paginação por range + count exato. Filtros: busca, categoria, status, destaque.
const PAGE = 20
let q = supabase.from('products')
  .select(`id, code, name, slug, status, price, cost_price, is_featured, category_id, updated_at,
           category:categories(id, name, parent_id),
           images:product_images(path, sort_order)`, { count: 'exact' })
  .eq('store_id', storeId)
if (f.search) {
  const term = f.search.replace(/[%_,()]/g, ' ').trim()       // não injetar sintaxe PostgREST no .or()
  q = q.or(`name.ilike.%${term}%,code.ilike.%${term}%`)
}
if (f.categoryId) q = q.eq('category_id', f.categoryId)
if (f.status) q = q.eq('status', f.status)                    // 'ACTIVE' | 'INACTIVE'
if (f.featured) q = q.eq('is_featured', true)
const { data, count, error } = await q
  .order(f.sort ?? 'updated_at', { ascending: f.asc ?? false }).order('id')
  .range((page - 1) * PAGE, page * PAGE - 1)

// Estoque (somente se hasStore('inventory.read')) — mesma página de ids
const ids = data.map(p => p.id)
const { data: stock } = await supabase.from('product_stock_balances')
  .select('product_id, quantity, reserved_quantity, available_quantity, average_cost, revision')
  .in('product_id', ids)
```

Colunas: capa (menor `sort_order` → `publicUrl('product-images', path)`), nome, SKU (`code`), categoria, preço, status, destaque, estoque disponível/reservado (se inventory.read). **Não há coluna de quantidade em products**: estoque vem sempre da view.

11.2 Formulário (novo / edição)

Campo	Regra	Mensagem / código
Nome	trim 2..160	23514 products_name_check
Slug	opcional; 2..160, `^[a-z0-9]+(-[a-z0-9]+)*$`; gerar sugestão do nome	PRODUCT_SLUG_EXISTS (create) / 23505:products_store_slug (update)
SKU	**Somente leitura** (code gerado `PRD-000001`)	—
Descrição	≤ 10000, texto simples (renderizar escapado; não usar v-html)	23514
Categoria	v-autocomplete com categorias vivas; opção “Sem categoria” (null)	PRODUCT_CATEGORY_UNAVAILABLE
Status	ACTIVE / INACTIVE (default INACTIVE)	—
Preço	obrigatório, ≥ 0, ≤ 9999999999.99, 2 casas	INVALID_MONEY_VALUE
Custo de referência	≥ 0, 2 casas; rótulo “Custo de referência (não altera o custo do estoque)”	INVALID_MONEY_VALUE
Destaque	boolean	—
Atributos	até 30 linhas {nome 1..80, valor 1..500}; nome único ignorando maiúsculas	DUPLICATE_ATTRIBUTE_NAME, ATTRIBUTES_LIMIT_30, INVALID_ATTRIBUTE_NAME_OR_VALUE
Estoque inicial	**Não faz parte do produto.** Após criar, oferecer “Registrar entrada de estoque” (ENTRY com custo unitário)	—

TS

```
// Criar → navegar para edição (mídia só existe depois que há product_id)
const op = stablePending('product:new', form)
const r = await rpc<CreateProductArgs, CreateProductResult>('create_product', {
```

```
  p_store_id: storeId, p_request_id: op.requestId,
  p_input: {
    name: form.name.trim(), slug: form.slug || null, description: form.description ?? '',
    category_id: form.categoryId ?? null, status: form.status, price: form.price,
    cost_price: form.costPrice ?? 0, is_featured: form.featured,
    attributes: form.attributes.filter(a => a.name.trim() && a.value.trim()),
  },
})
clearPending('product:new')
router.replace({ name: 'app-product-edit', params: { productId: r.product_id }, query: { created: '1' } })

// Editar: enviar só o diff. 'attributes' presente => substitui todos.
await rpc<UpdateProductArgs, UpdateProductResult>('update_product', {
  p_product_id: id, p_patch: diff(original, form),
})

// Carregar para edição (produto arquivado → null → tela 404)
const { data: product } = await supabase.from('products')
  .select(`*, attributes:product_attributes(name, value, sort_order),
           images:product_images(id, path, alt_text, sort_order)`)
  .eq('id', productId).maybeSingle()
```

11.3 Imagens do produto (até 4)

1. Pré-requisito: produto existente e não arquivado; permissão `catalog.products.media`.
2. Validar MIME (png/jpeg/webp) e ≤ 5 MB; bloquear seleção quando já há 4 vinculadas.
3. Upload: `storage.from('product-images').upload(`${storeId}/${productId}/${uuid}.${ext}`, file, { upsert: false })`.
4. Vincular: `attach_product_image(p_product_id, p_path, p_alt_text)` → recebe `image_id` e `sort_order`. Falhou (ex.: PRODUCT_IMAGES_LIMIT_4) → `remove([path])`.
5. Reordenar (arrastar): `reorder_product_images(p_product_id, todosOsIdsNaNovaOrdem)`. A posição 0 é a capa da vitrine.
6. Remover: `detach_product_image` → se `detached` e `path`, `storage.remove([path])`. A policy só deixa apagar objeto **já desvinculado**.
7. Alt text: sem RPC de edição. Para alterar, desvincular e vincular de novo (novo upload não é necessário: re-attach do mesmo path é permitido enquanto o objeto existir — faça `detach` sem apagar o arquivo e depois `attach` com o novo alt).

TS

```
export async function addProductImage(storeId: string, productId: string, file: File, alt = '') {
  assertImage(file, 'product-images')
  const path = productMediaPath(storeId, productId, file)
  const up = await supabase.storage.from('product-images').upload(path, file, { upsert: false, contentType: file.type })
  if (up.error) throw up.error
  try {
    return await rpc<AttachProductImageArgs, AttachProductImageResult>('attach_product_image',
      { p_product_id: productId, p_path: path, p_alt_text: alt.slice(0, 200) })
  } catch (e) { await supabase.storage.from('product-images').remove([path]); throw e }
}
export async function removeProductImage(productId: string, imageId: string) {
  const r = await rpc<DetachProductImageArgs, DetachProductImageResult>('detach_product_image',
    { p_product_id: productId, p_image_id: imageId })
  if (r.detached && r.path) await supabase.storage.from('product-images').remove([r.path])
  return r
}
```

> **URLs completas**
>
> O banco guarda apenas o `path` relativo (sem domínio, sem bucket). Nunca gravar URL pública. Na vitrine, as imagens vêm como `images[].path` e o frontend monta a URL com `getPublicUrl` do bucket público `product-images`.

11.4 Arquivar produto

Botão “Arquivar” (não há exclusão). Desabilitar quando `product_stock_balances.quantity > 0` com tooltip “Zere o estoque antes (contagem = 0)”. Erros: `PRODUCT_HAS_STOCK`, `PRODUCT_HAS_OPEN_ORDERS`. Sugestão: antes de arquivar, desvincular e apagar imagens (após arquivar, a policy não permite mais apagar).

Aceitação: criar produto com 3 atributos e 4 imagens; 5ª imagem bloqueada; reordenar muda a capa na vitrine; edição envia apenas campos alterados; SKU nunca editável; duplicar envio não cria dois produtos.

12. Estoque: saldos, movimentos, contagem e lote

12.1 Modelo de estoque

Conceito	Onde	Significado para a UI
quantity	product_stock_balances / inventory_balances	Estoque físico (integer ≥ 0)
reserved_quantity	product_stock_balances	Soma das reservas HELD não expiradas (pedidos PENDING criados nas últimas 24 h)
available_quantity	product_stock_balances	`max(0, quantity − reserved)` — limite de saída manual e o que a vitrine considera
average_cost	inventory_balances (GENERATED)	Custo médio móvel (6 casas). Base do CMV na baixa por pagamento
stock_value	—	Valor contábil do estoque (6 casas)
revision	—	Contador por produto (0 sem movimentos). Usado como `expected_revision` em contagens
Ledger	inventory_movements (imutável)	Cada mudança = 1 linha com antes/depois; nada é editado ou apagado

Reserva ≠ baixa

- O checkout público cria **reservas** de 24 h; o estoque físico **não** é baixado.
- A reserva **expira sem cancelar o pedido**: o pedido continua PENDING, mas a unidade volta a ficar disponível para outros clientes.
- A baixa física acontece somente em `confirm_order_payment` (movimento EXIT “Baixa por pagamento manual do pedido PED-…”).
- Saídas manuais/ajustes não podem deixar `quantity < reservas ativas` (`STOCK_RESERVED_BY_PENDING_ORDERS`).

12.2 /app/inventory — saldos e ações

TS

```
const { data, count } = await supabase.from('product_stock_balances')
  .select('product_id, code, name, status, category_id, quantity, reserved_quantity, available_quantity, stock_value,
average_cost, revision, updated_at', { count: 'exact' })
  .eq('store_id', storeId)
  .ilike('name', `%${term}%`)                  // opcional
  .order('name').range(from, to)
// Filtros úteis: .eq('quantity', 0) (zerados); .gt('reserved_quantity', 0) (com reservas); .eq('status','ACTIVE')
```

Ação (diálogo)	Campos	RPC	Regras
Entrada	quantidade inteira ≥1, custo unitário (obrigatório, até 6 casas), motivo 3..500, referência ≤120 (ex.: NF)	register_inventory_movement(ENTRY)	Atualiza custo médio. Para vincular a uma despesa de compra, ver 15.3
Saída manual	quantidade ≤ available_quantity, motivo, referência	register_inventory_movement(EXIT)	Sem custo (automático). Avisar: “não é venda; aparecerá como saída não classificada nos relatórios”
Contagem	quantidade contada (absoluta), motivo; custo opcional só se contagem > saldo	adjust_inventory_count	Enviar `p_expected_revision` = revision exibida
Contagem em lote	tabela editável (até 50 linhas) com contado/custo	adjust_inventory_batch	Atômico; erro em uma linha desfaz todas

TS

```
// Entrada
const op = stablePending(`inv:${productId}:entry`, form)
const r = await rpc<RegisterInventoryMovementArgs, InventoryMovementResult>('register_inventory_movement', {
  p_product_id: productId, p_request_id: op.requestId, p_type: 'ENTRY',
```

```
  p_quantity: form.qty, p_unit_cost: form.unitCost, p_reason: form.reason.trim(), p_reference: form.ref || null,
})

// Contagem individual com concorrência otimista
try {
  await rpc<AdjustInventoryCountArgs, AdjustInventoryCountResult>('adjust_inventory_count', {
    p_product_id: row.product_id, p_request_id: op.requestId, p_counted_quantity: form.counted,
    p_expected_revision: row.revision, p_reason: form.reason, p_unit_cost: form.counted > row.quantity ? form.unitCost ??
null : null,
  })
} catch (e) {
  if (e instanceof BackendError && e.code === 'STALE_INVENTORY_REVISION') {
    clearPending(`inv:${row.product_id}:count`)   // nova intenção após recarregar
    await reloadRow(row.product_id)                // mostrar novo saldo e pedir confirmação
  } else throw e
}

// Lote
await rpc<AdjustInventoryBatchArgs, AdjustInventoryBatchResult>('adjust_inventory_batch', {
  p_store_id: storeId, p_request_id: op.requestId, p_reason: 'Inventário mensal', p_reference: 'INV-2026-10',
  p_items: rows.filter(r => r.counted !== null && r.counted !== r.quantity).map(r => ({
    product_id: r.product_id, counted_quantity: r.counted!, expected_revision: r.revision,
    ...(r.counted! > r.quantity && r.unitCost != null ? { unit_cost: r.unitCost } : {}),
  })),
})
```

12.3 /app/inventory/movements — ledger

TS

```
let q = supabase.from('inventory_movements')
  .select('id, product_id, product_code, product_name, movement_type, quantity_delta, unit_cost, value_delta,
quantity_before, quantity_after, average_cost_after, revision, reason, reference, reversal_of_id, batch_request_id,
created_by, created_at', { count: 'exact' })
  .eq('store_id', storeId)
if (f.productId) q = q.eq('product_id', f.productId)
if (f.types?.length) q = q.in('movement_type', f.types)
if (f.from) q = q.gte('created_at', startOfDayIso(f.from, tz))
if (f.to) q = q.lt('created_at', startOfDayIso(addDays(f.to, 1), tz))
if (f.batch) q = q.eq('batch_request_id', f.batch)
const { data, count } = await q.order('created_at', { ascending: false }).order('id').range(from, to)
```

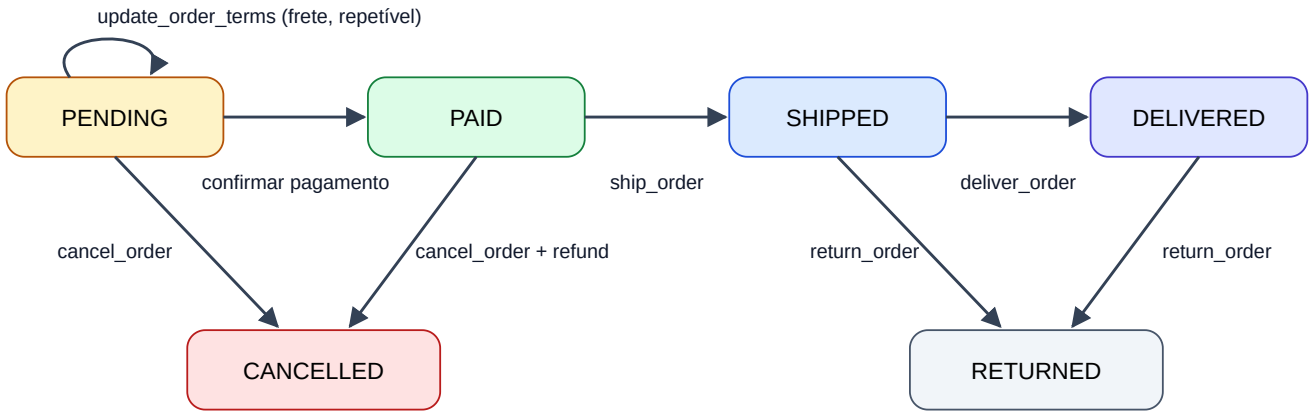
Botão “Reverter” somente quando: `inventory.reverse`, o movimento é o `last_movement_id` do produto (consultar `product_stock_balances.last_movement_id`), não é REVERSAL, não tem reversão (verificar `reversal_of_id` em outra linha) e não está ligado a pedido (motivo começa com “Baixa por pagamento”/“Reposicao por” — confirmar via `order_stock_operations.select('movement_id').eq('movement_id', id)` com `orders.read`). Erros esperados: `ONLY_LAST_MOVEMENT_CAN_BE_REVERSED`, `MOVEMENT_ALREADY_REVERSED`, `ORDER_STOCK_REVERSAL_REQUIRES_ORDER_FLOW`, `STOCK_RESERVED_BY_PENDING_ORDERS`.

`created_by` é UUID de `auth.users`; não há join com nome de usuário para não-SUDO (profiles só lê a própria linha). Exibir “Você” quando igual ao usuário atual, senão UUID abreviado.

Aceitação: entrada sem custo é bloqueada no form; saída maior que o disponível é bloqueada; contagem concorrente em duas abas → a segunda recebe STALE e recarrega; lote com 1 linha inválida não aplica nada; reverter só aparece no último movimento elegível.

13. Pedidos (operação manual)

13.1 Máquina de estados (final, módulo 06)



Transições permitidas pelo trigger `guard_order_update`. CANCELLED e RETURNED são terminais.

Status	Ações disponíveis (se permitido)	Estoque	Dinheiro
PENDING	Confirmar frete (repetível) → Confirmar pagamento (exige frete confirmado) · Cancelar	Reservado (24 h) — sem baixa	—
PAID	Enviar · Cancelar (com reembolso declarado)	Baixado (EXIT por item)	RECEIPT = total
SHIPPED	Marcar entregue · Devolução	Baixado	RECEIPT
DELIVERED	Devolução	Baixado	RECEIPT
CANCELLED	— (terminal)	PENDING: reserva liberada · PAID: reposto	PAID: REFUND = total
RETURNED	— (terminal)	RESTOCKED volta; DAMAGED vira perda	REFUND = total (inclui frete)

13.2 Lista /app/orders

TS

```
let q = supabase.from('orders')
  .select(`id, code, status, revision, subtotal, shipping_amount, shipping_confirmed, total,
           created_at, updated_at, paid_at, shipped_at, delivered_at, cancelled_at, returned_at,
           reservation_expires_at, customer_snapshot, customer_id`, { count: 'exact' })
  .eq('store_id', storeId)
if (f.status?.length) q = q.in('status', f.status)
if (f.code) q = q.ilike('code', `%${f.code.replace(/[%_]/g, '')}%`)
if (f.customerId) q = q.eq('customer_id', f.customerId)
if (f.from) q = q.gte('created_at', startOfDayIso(f.from, tz))
if (f.to) q = q.lt('created_at', startOfDayIso(addDays(f.to, 1), tz))
const { data, count } = await q.order('created_at', { ascending: false }).order('id').range(from, to)
```

Colunas: código, data, cliente (`customer_snapshot.full_name` + telefone mascarado), status (chip colorido), subtotal, frete (“a combinar” se `shipping_confirmed=false`), total. Badge “Reserva expirada” quando `status=PENDING` e `reservation_expires_at < agora`. Busca por telefone/nome do cliente: use `/app/customers` (o filtro em jsonb do snapshot não é indexado; opcional `.ilike('customer_snapshot->>full_name', …)`).

13.3 Detalhe /app/orders/:orderId

TS

```
const [order, items, history, payments, reservations, returns] = await Promise.all([
  supabase.from('orders').select('*').eq('id', orderId).maybeSingle(),
  supabase.from('order_items').select('id, product_id, product_code, product_name, product_slug, image_path,
attributes_snapshot, quantity, unit_price, unit_cost_snapshot, line_total').eq('order_id', orderId).order('product_name'),
  supabase.from('order_status_history').select('id, revision, event_type, status_before, status_after, note, actor_id,
created_at').eq('order_id', orderId).order('revision'),
  supabase.from('order_payments').select('id, kind, method, amount, external_reference, note, confirmed_by,
confirmed_at').eq('order_id', orderId).order('confirmed_at'),
  supabase.from('inventory_reservations').select('id, product_id, quantity, state, expires_at').eq('order_id', orderId),
  supabase.from('order_return_items').select('id, order_item_id, disposition, quantity, allocated_cost, restocked_value,
loss_value, note, created_at').eq('order_id', orderId),
])
// order.data === null → 404 (inexistente ou sem orders.read)
```

Blocos: cabeçalho (código, status, revisão, datas), cliente e endereço (snapshot imutável), itens (snapshot de nome/preço/imagem/atributos), resumo financeiro (subtotal + frete = total), reservas (estado HELD/RELEASED/CONSUMED e expiração), pagamentos (RECEIPT/REFUND), devoluções, timeline. Botão “Abrir WhatsApp do cliente”: `https://wa.me/${phone.replace(/\D/g,'')}` (link externo; sem API). `unit_cost_snapshot` é referência na criação — o CMV real está em `order_stock_operations.allocated_value` (SALE).

13.4 Ações (diálogos) — todas exigem nota 3..1000 e expected_revision

Ação	Visível quando	Campos	Chamada
Confirmar frete	PENDING + orders.terms	Valor do frete (≥ 0, 2 casas; 0 = frete grátis confirmado), nota	update_order_terms
Confirmar pagamento	PENDING + shipping_confirmed + orders.pay + inventory.exit	Valor (pré-preenchido com total, somente leitura), método, referência (≤160), nota; checkbox “Confirmo que o valor foi recebido”	confirm_order_payment
Enviar	PAID + orders.ship	Código de rastreio (opcional ≤120), nota	ship_order
Entregue	SHIPPED + orders.deliver	nota	deliver_order
Cancelar	PENDING ou PAID + orders.cancel (+ inventory.entry se PAID)	PENDING: nota. PAID: nota, método do reembolso, referência, checkbox obrigatório “Já devolvi integralmente R$ X ao cliente”	cancel_order
Devolução	SHIPPED/DELIVERED + orders.return	Para CADA item: RESTOCKED ou DAMAGED; checkbox “Recebi os produtos”; checkbox “Já devolvi R$ total (inclui frete)”; método; referência; nota	return_order

TS

```
// src/services/orders.service.ts (PROPOSTA)
type Op = 'terms' | 'pay' | 'ship' | 'deliver' | 'cancel' | 'return'
const key = (orderId: string, op: Op) => `order:${orderId}:${op}`

async function run<A extends object>(orderId: string, op: Op, fn: string, args: Omit<A, 'p_request_id'>) {
  const pending = stablePending(key(orderId, op), args)
  try {
    const r = await rpc<A, OrderOperationResult>(fn, { ...args, p_request_id: pending.requestId } as A)
    clearPending(key(orderId, op))
    return r                                  // r.replayed=true também é sucesso
  } catch (e) {
    if (e instanceof BackendError && !e.isNetwork) clearPending(key(orderId, op))
    throw e
  }
}

export const ordersService = {
  confirmTerms: (o: OrdersRow, shipping: number, note: string) =>
    run<UpdateOrderTermsArgs>(o.id, 'terms', 'update_order_terms',
      { p_order_id: o.id, p_expected_revision: o.revision, p_shipping_amount: shipping, p_note: note }),

  confirmPayment: (o: OrdersRow, method: ManualMethod, note: string, reference?: string) =>
    run<ConfirmOrderPaymentArgs>(o.id, 'pay', 'confirm_order_payment',
      { p_order_id: o.id, p_expected_revision: o.revision, p_amount: o.total, p_method: method,
        p_note: note, p_reference: reference || null }),

  ship: (o: OrdersRow, note: string, tracking?: string) =>
    run<ShipOrderArgs>(o.id, 'ship', 'ship_order',
      { p_order_id: o.id, p_expected_revision: o.revision, p_note: note, p_tracking_code: tracking?.trim() || null }),
```

```
  deliver: (o: OrdersRow, note: string) =>
    run<DeliverOrderArgs>(o.id, 'deliver', 'deliver_order', { p_order_id: o.id, p_expected_revision: o.revision, p_note:
note }),

  cancel: (o: OrdersRow, note: string, refund?: { method: ManualMethod; reference?: string }) =>
    run<CancelOrderArgs>(o.id, 'cancel', 'cancel_order', o.status === 'PENDING'
      ? { p_order_id: o.id, p_expected_revision: o.revision, p_note: note,
          p_refund_confirmed: false, p_refund_method: null, p_refund_reference: null }
      : { p_order_id: o.id, p_expected_revision: o.revision, p_note: note,
          p_refund_confirmed: true, p_refund_method: refund!.method, p_refund_reference: refund?.reference || null }),

  returnAll: (o: OrdersRow, items: ReturnOrderItem[], method: ManualMethod, note: string, reference?: string) =>
    run<ReturnOrderArgs>(o.id, 'return', 'return_order',
      { p_order_id: o.id, p_expected_revision: o.revision, p_items: items, p_goods_received: true,
        p_refund_confirmed: true, p_refund_method: method, p_note: note, p_refund_reference: reference || null }),
}
```

Erro	UX
STALE_ORDER_REVISION	Recarregar o pedido; fechar diálogo; toast “O pedido foi atualizado por outra ação. Revise e tente novamente.”
SHIPPING_TERMS_NOT_CONFIRMED	Abrir diálogo de frete
PAYMENT_MUST_MATCH_ORDER_TOTAL	Recarregar (o total mudou após novo frete)
INSUFFICIENT_AVAILABLE_STOCK_FOR_PAYMENT	“A reserva expirou e o estoque foi usado por outro pedido. Registre entrada de estoque ou cancele.” + link Estoque
ORDER_NOT_PENDING / ORDER_NOT_PAID / ORDER_NOT_SHIPPED / ORDER_CANNOT_BE_*	Recarregar; o status mudou
MANUAL_REFUND_CONFIRMATION_REQUIRED / PHYSICAL_RETURN_CONFIRMATION_REQUIRED	Destacar checkbox de declaração
FULL_ORDER_RETURN_REQUIRED	Exigir disposição para todos os itens
ORDER_NOTE_REQUIRED	Campo nota (mín. 3 caracteres)
ORDER_FORBIDDEN / INVENTORY_FORBIDDEN	Sem permissão

Declarações manuais

`confirm_order_payment`, `p_refund_confirmed` e `p_goods_received` registram **declarações do lojista**. O sistema não movimenta dinheiro nem verifica banco/PIX. A UI deve deixar isso explícito nos textos dos checkboxes.

Aceitação: pagamento indisponível até frete confirmado; frete 0 exige confirmação explícita; duplo clique não duplica operação (mesmo request_id → `replayed`); pedido alterado em outra aba gera STALE e recarrega; devolução exige todos os itens; botões respeitam `get_store_access().permissions`.

14. Clientes (somente leitura)

`customers` é **somente leitura** (criado/reaproveitado pelo checkout por `(store_id, phone)`). Não há CRUD, edição, mesclagem, exclusão, LGPD-export ou anonimização no backend atual. O nome em `customers.full_name` é o do *primeiro* pedido daquele telefone (checkout não sobrescreve); o nome usado em cada pedido está no `orders.customer_snapshot`.

TS

```
let q = supabase.from('customers')
  .select('id, full_name, phone, created_at, updated_at', { count: 'exact' })
  .eq('store_id', storeId)
if (term) {
  const t = term.replace(/[%_,()]/g, ' ').trim()
  q = q.or(`full_name.ilike.%${t}%,phone.ilike.%${t.replace(/\D/g, '')}%`)
}
const { data, count } = await q.order('created_at', { ascending: false }).range(from, to)

// Pedidos do cliente (painel lateral) — requer orders.read
const { data: orders } = await supabase.from('orders')
  .select('id, code, status, total, created_at').eq('store_id', storeId).eq('customer_id', customerId)
  .order('created_at', { ascending: false }).limit(50)
```

Métricas por cliente (nº de pedidos, total pago) podem ser calculadas no cliente sobre essa lista limitada; não há agregação no servidor. Exportação: opcional gerar CSV no navegador a partir da página carregada (sem endpoint). **Aceitação:** busca por nome ou dígitos do telefone; clique abre pedidos; nenhuma ação de edição.

15. Despesas pagas

15.1 Categorias de despesa — /app/expenses/categories

TS

```
const { data } = await supabase.from('expense_categories')
  .select('id, name, classification, is_active, created_at, updated_at')
  .eq('store_id', storeId).order('classification').order('name')

await rpc<CreateExpenseCategoryArgs, CreateExpenseCategoryResult>('create_expense_category', {
  p_store_id: storeId, p_request_id: op.requestId, p_name: 'Aluguel', p_classification: 'OPERATING' })

await rpc<UpdateExpenseCategoryArgs, UpdateExpenseCategoryResult>('update_expense_category', {
  p_category_id: id, p_request_id: op.requestId, p_name: 'Aluguel loja', p_active: false })
```

Classificação	Significado	Efeito no ERP
OPERATING	Aluguel, internet, embalagens, marketing, taxas…	Entra em `net_operating_expenses` (reduz resultado) e no caixa
INVENTORY_PURCHASE	Pagamento de mercadoria para estoque	Somente caixa (`net_inventory_purchase_cash`); o custo entra no resultado via CMV quando vendido — evita dupla contagem

Classificação é **imutável** após criar (escolher com cuidado; texto de ajuda no form). Nome único por loja ignorando maiúsculas/espaços (`23505:expense_categories_store_name`). Sem exclusão: desativar.

15.2 Lançamentos — /app/expenses

TS

```
let q = supabase.from('expense_entries')
  .select('id, category_id, category_name_snapshot, classification, kind, amount, method, description, external_reference,
occurred_at, recorded_at, recorded_by, inventory_movement_id, reversal_of_id', { count: 'exact' })
  .eq('store_id', storeId)
  .gte('occurred_at', startOfDayIso(f.from, tz)).lt('occurred_at', startOfDayIso(addDays(f.to, 1), tz))
if (f.categoryId) q = q.eq('category_id', f.categoryId)
if (f.kind) q = q.eq('kind', f.kind)
const { data, count } = await q.order('occurred_at', { ascending: false }).order('id').range(from, to)
// "Já revertido?" = existe outra linha com reversal_of_id = id (carregar ids revertidos da página)
```

Form “Nova despesa paga”	Regra
Categoria	somente ativas
Valor	> 0, ≤ 9999999999.99, 2 casas
Método	PIX · CASH · BANK_TRANSFER · CARD · OTHER (rótulos: Pix, Dinheiro, Transferência, Cartão, Outro)
Data do pagamento	datetime ≤ agora (+5 min tolerância) e ≥ 2000-01-01. **Não aceita data futura**: não existe agendamento nem contas a pagar
Descrição	3..500
Referência	≤ 160 (NF, comprovante)
Movimento de estoque (opcional)	Só para categoria INVENTORY_PURCHASE: selecionar uma ENTRY manual ainda não vinculada

TS

```
await rpc<RecordExpenseArgs, RecordExpenseResult>('record_expense', {
  p_store_id: storeId, p_request_id: op.requestId, p_category_id: form.categoryId,
  p_amount: form.amount, p_method: form.method, p_description: form.description.trim(),
  p_occurred_at: new Date(form.paidAtLocal).toISOString(),       // converter do fuso local
  p_reference: form.reference || null, p_inventory_movement_id: form.movementId ?? null,
})
```

15.3 Vincular compra de estoque

TS

```
// ENTRY manuais elegíveis (excluir as já vinculadas e as geradas por pedido)
const { data: entries } = await supabase.from('inventory_movements')
  .select('id, product_name, quantity_delta, unit_cost, value_delta, reference, created_at')
  .eq('store_id', storeId).eq('movement_type', 'ENTRY').order('created_at', { ascending: false }).limit(50)
const { data: linked } = await supabase.from('expense_entries')
  .select('inventory_movement_id').eq('store_id', storeId).not('inventory_movement_id', 'is', null)
const { data: fromOrders } = await supabase.from('order_stock_operations')
  .select('movement_id').eq('store_id', storeId).eq('kind', 'RESTOCK')       // requer orders.read
// candidatos = entries − linked − fromOrders
```

15.4 Correção × Reembolso

Tipo	Quando usar	Data efetiva	Campos enviados
CORRECTION	O lançamento foi um erro (valor/categoria errados, duplicado)	**Data original** do lançamento (anula no mesmo período)	descrição. `p_occurred_at`, `p_method`, `p_reference` = null (senão CORRECTION_USES_ORIGINAL_DATE)
REFUND	O fornecedor devolveu o dinheiro de fato	**Nova data** (≥ original, ≤ agora)	descrição, data, método, referência

Ambos revertem o valor integral, uma vez por lançamento, e somente lançamentos `kind=EXPENSE` (`EXPENSE_ALREADY_REVERSED_OR_NOT_ORIGINAL`). Para corrigir um valor: CORRECTION + novo lançamento correto.

Aceitação: data futura bloqueada no picker; categoria inativa não aparece no form; linha revertida mostra badge e esconde o botão; correção e reembolso exibem campos condicionais corretos.

16. Relatórios ERP

Regra	Implementação na UI
Período	Seletor de datas (from/to, inclusivos). Bloquear > 366 dias (`to − from ≤ 365`). Presets: hoje, 7 dias, mês atual, mês anterior, ano até hoje (cortado em 366 dias).
Fuso	Default `America/Sao_Paulo` . Se oferecer seletor, usar nomes IANA válidos; erro `INVALID_REPORT_PERIOD_OR_TIMEZONE` .
Base	“Gerencial, baseada em caixa manual” (`accounting_basis`). Exibir os `warnings` retornados.
Estimativa	`is_estimate: true` sempre → rótulo “Resultado operacional estimado”. Não é lucro fiscal nem saldo bancário.
Reconhecimento	Receita e CMV na data do **recebimento manual**; estornos/devoluções na data do REFUND (com CMV revertido e perda de itens DAMAGED). O status atual do pedido não altera fatos passados.
Custo	CMV = valor exato alocado na baixa (custo médio vigente na data do pagamento). Reposição usa o mesmo valor (sem duplicar). Perdas = custo dos itens DAMAGED.
Saídas manuais	`needs_stock_review=true` quando houver EXIT manual (sem pedido) no período: alerta “R$ X em saídas de estoque sem classificação (N eventos). Elas não entram no resultado.” + link para Movimentações filtradas por EXIT.
Ajustes	`inventory_adjustment_cost` (contagens) entra no resultado (positivo = perda).
Estoque	`inventory_snapshot_current` é o estoque ATUAL (rótulo “agora”), não o do fim do período.
Despesas	Somente pagas; compras de estoque só no caixa.

TS

```ts
// src/services/reports.service.ts (PROPOSTA)
const TZ = 'America/Sao_Paulo'
export function assertPeriod(from: string, to: string) {
  const days = (Date.parse(to) - Date.parse(from)) / 86_400_000
  if (!(days >= 0 && days <= 365)) throw new Error('INVALID_REPORT_PERIOD_OR_TIMEZONE')
}
export async function loadReports(storeId: string, from: string, to: string, tz = TZ) {
  assertPeriod(from, to)
  const base = { p_store_id: storeId, p_from: from, p_to: to, p_timezone: tz }
  const [summary, daily, products, expenses] = await Promise.all([
    rpc<ErpPeriodArgs, ErpSummary>('get_erp_summary', base),
    rpc<ErpPeriodArgs, ErpDaily>('get_erp_daily', base),
    rpc<ErpProductPerformanceArgs, ErpProductPerformance>('get_erp_product_performance', { ...base, p_limit: 20 }),
    rpc<ErpPeriodArgs, ErpExpenseBreakdown>('get_erp_expense_breakdown', base),
  ])
  return { summary, daily, products, expenses }
}
```

Card / gráfico	Campos
Receita líquida	net_revenue (= net_goods_revenue + net_shipping_revenue)
CMV	net_cogs
Margem bruta	gross_margin
Perdas	returned_damage_loss + inventory_adjustment_cost
Despesas operacionais	net_operating_expenses
Resultado operacional estimado	estimated_operating_result; operating_margin_percent (null se receita 0 → “—”)
Caixa	cash_received, cash_paid_net, net_cash_flow; compras de estoque: net_inventory_purchase_cash
Volume	receipt_count, refund_count, units_sold, units_restocked, average_receipt (null → “—”)
Série diária	daily.days[] (linhas: net_revenue, estimated_operating_result; barras: net_cash_flow)
Top produtos	products.products[] (net_units, net_goods_revenue, net_cogs, damage_loss, contribution_before_shared_expenses) + products.note

Card / gráfico	Campos
Despesas por categoria	expenses.categories[] (name_snapshot, classification, net_amount)

Estados: loading com skeleton dos cards; empty quando todos os totais = 0 (“Sem movimentação no período”); erro `REPORT_FORBIDDEN` → tela sem acesso. **Aceitação:** período de 367 dias é bloqueado antes da chamada; valores formatados em BRL com 2 casas; avisos sempre visíveis; alerta de revisão de estoque quando aplicável.

17. Dashboard (/app)

/app combina dados já cobertos, condicionando cada bloco à permissão:

Bloco	Permissão	Fonte
Checklist de ativação (logo, categoria, produto ACTIVE, estoque, publicada)	store.read	stores, counts de categories/products (head:true), product_stock_balances (quantity>0)
Pedidos aguardando ação	orders.read	`orders.select('id', {count:'exact',head:true}).eq('status','PENDING')`, idem PAID (a enviar)
Reservas expiradas	orders.read	PENDING com `reservation_expires_at < now`
Estoque zerado/baixo	inventory.read	product_stock_balances `.lte('available_quantity', 3)`
Resumo do mês	reports.read	get_erp_summary (mês atual)
Link da vitrine	store.read	`{APP_URL}/s/{slug}` se is_published

18. Painel SUDO

> **Atribuição de SUDO**
>
> O papel SUDO **só** é atribuído pelo SQL Editor do Supabase (INSERT em `user_platform_roles` com service_role/postgres). A tabela não tem GRANT de escrita para `authenticated` : o painel apenas **lê** `user_platform_roles`. Não criar botão “tornar SUDO”.

SQL

```
-- Executar no SQL Editor (fora do app), uma vez por administrador:
insert into public.user_platform_roles (user_id, role_code) values ('<uuid-do-usuario>', 'SUDO');
```

Rota	Leitura	Escrita	Regras
/sudo	get_my_access; security_audit_logs (últimos 100)	—	Layout exige `can_manage_security`
/sudo/users	profiles (todos — policy permite SUDO), user_platform_roles	`sudo_set_user_active(p_user_id, p_active)`	E-mail não está em profiles (Auth admin não é exposto ao navegador): listar por nome/UUID. Não pode desativar a si mesmo (CANNOT_DISABLE_SELF).
/sudo/roles	roles	INSERT/UPDATE/DELETE roles	is_system=true: não pode inserir como sistema, nem deletar, nem mudar code/is_system (SYSTEM_DEFINITION_RESERVED, SYSTEM_DEFINITION_CANNOT_BE_DELETED, SECURITY_CODE_AND_SYSTEM_FLAG_IMMUTABLE). code `^[A-Z][A-Z0-9_]{1,49}$` ; name 1..120. Papéis customizados não podem ser atribuídos a usuários (user_platform_roles aceita só USER/SUDO) — servem apenas como catálogo.
/sudo/permissions	permissions	INSERT/UPDATE/DELETE	code `^[a-z][a-z0-9_.]{2,99}$` ; module 1..80; mesmas proteções de sistema. Permissões novas não têm efeito sem código SQL que as consulte.
/sudo/role-permissions	role_permissions + roles + permissions	INSERT/UPDATE(is_enabled)/DELETE	Matriz papel × permissão com switches. Mudanças são imediatas e auditadas. Avisar: desativar `security.manage` / `auth.access` do SUDO pode bloquear o painel (recuperação só via SQL Editor).
(dentro de /sudo, opcional) Plano	plans (todos)	UPDATE name, description, is_active	Requer SUDO + plans.manage + auth.access. Desativar o FREE bloqueia novos onboardings (PLAN_UNAVAILABLE).

TS

```
// role_permissions: matriz
const [roles, perms, links] = await Promise.all([
  supabase.from('roles').select('id, code, name, is_system, is_active').order('code'),
  supabase.from('permissions').select('id, code, module, description, is_system, is_active').order('module').order('code'),
  supabase.from('role_permissions').select('role_id, permission_id, is_enabled'),
])
// ligar/desligar
await supabase.from('role_permissions').upsert({ role_id, permission_id, is_enabled: true }).select()
await supabase.from('role_permissions').update({ is_enabled: false }).eq('role_id', role_id).eq('permission_id',
permission_id).select()

// usuários
const { data: users } = await supabase.from('profiles').select('id, full_name, phone, is_active,
created_at').order('created_at', { ascending: false }).range(0, 49)
const { data: platform } = await supabase.from('user_platform_roles').select('user_id, role_code').in('user_id', users.map(u
=> u.id))
await rpc<SudoSetUserActiveArgs, null>('sudo_set_user_active', { p_user_id: id, p_active: false })

// auditoria
const { data: logs } = await supabase.from('security_audit_logs')
  .select('id, actor_id, table_name, operation, row_before, row_after, created_at').order('created_at', { ascending: false
}).range(0, 99)
```

SUDO + `store.admin` também pode abrir lojas de terceiros (RLS permite). Este guia **não** propõe tela de administração de lojas de terceiros (não listada no escopo); se for criada, deve usar `get_store_access(store_id)` com `role: 'SUDO'` e nunca alterar o fluxo do OWNER.
Aceitação: usuário comum acessando /sudo é redirecionado; não há UI de atribuição de SUDO; definições de sistema aparecem bloqueadas para exclusão/edição de código.

19. Vitrine pública via Edge storefront-api

> **Regra absoluta**
>
> A vitrine pública fala **exclusivamente** com a Edge Function `storefront-api` via `POST`. O navegador **nunca** chama as 5 RPCs service-only (`get_public_catalog` , `get_public_product` , `create_public_order` , `get_public_order_status` , `consume_public_rate_limit`) — elas não têm EXECUTE para anon/authenticated (testes confirmam `insufficient_privilege`) — e **nunca** usa service_role. Também não lê tabelas: anon não tem SELECT em nenhuma tabela de negócio.

19.1 Contrato HTTP comum

Item	Valor
URL	`{SUPABASE_URL}/functions/v1/storefront-api`
Método	`POST` (`OPTIONS` para preflight → 204). Outros → 405 METHOD_NOT_ALLOWED
Headers	`content-type: application/json` (obrigatório, senão 415 JSON_REQUIRED). `apikey` / `authorization` com a anon key são aceitos pelo CORS; a função é implantada com `verify_jwt=false` , então não há JWT obrigatório. **Não enviar o JWT do lojista** na vitrine.
Corpo	JSON objeto ≤ 32 KiB (413 BODY_TOO_LARGE). Chaves desconhecidas → 400 UNKNOWN_FIELD
Campos comuns	`action` ∈ catalog | product | checkout | order-status; `store_slug` (2..63, slug)
Sucesso	`200 { "data": … }`
Não encontrado	`404 { "error": "NOT_FOUND" }` (RPC retornou null) · `404 STORE_UNAVAILABLE` (checkout em loja indisponível)
Erros	`{ "error": "CODIGO" }` com status 400/403/405/409/413/415/429/503
Rate limit	Teto global por ação/60 s: catalog 300, product 300, checkout 30, order-status 120; checkout também 3/min por (loja, telefone); opcional por IP (5 checkout / 60 demais) se `TRUSTED_CLIENT_IP_HEADER` configurado. 429 + header `Retry-After`
Cache	Respostas `cache-control: no-store` . Cache curto em memória no cliente é permitido para catalog/product.
CORS	Somente origens em `ALLOWED_ORIGINS` (default: origem de APP_URL). Outra origem → 403 ORIGIN_NOT_ALLOWED

HTTP	error	Causa	UX / retry
400	INVALID_BODY, INVALID_JSON, UNKNOWN_FIELD, INVALID_FIELD, INVALID_ACTION, INVALID_SLUG, INVALID_STORE_SLUG, INVALID_UUID, INVALID_TOKEN	Contrato quebrado pelo cliente	Bug: não repetir; logar código (sem payload)
400	INVALID_PRICE_FILTER, INVALID_PRICE_RANGE, INVALID_STOCK_FILTER, INVALID_SORT, INVALID_PAGINATION	Filtros inválidos (ex.: query string manipulada)	Resetar filtros
400	INVALID_PHONE, INVALID_ADDRESS, INVALID_ITEMS, DUPLICATE_PRODUCT, INVALID_QUANTITY	Checkout inválido	Mostrar no campo; corrigir gera NOVO request_id
400	INVALID_REQUEST	Validação SQL 22xxx/23xxx genérica	Revisar dados
403	ORIGIN_NOT_ALLOWED	Origem fora de ALLOWED_ORIGINS	Configuração de deploy
404	NOT_FOUND / STORE_UNAVAILABLE	Loja inexistente, despublicada, arquivada, dono inativo; produto oculto; pedido/token não conferem	StorefrontNotFound
409	PRODUCT_UNAVAILABLE	Produto ficou inativo/arquivado	Remover do carrinho, revalidar
409	INSUFFICIENT_AVAILABLE_STOCK	Disponível < quantidade	Revalidar carrinho e reduzir
409	IDEMPOTENCY_KEY_REUSED	Mesmo request_id com payload diferente	Gerar novo request_id + client_secret
409	ORDER_TOTAL_LIMIT	Total > 9.999.999.999,99	Reduzir itens

HTTP	error	Causa	UX / retry
413/415/405	BODY_TOO_LARGE / JSON_REQUIRED / METHOD_NOT_ALLOWED	Cliente incorreto	Bug
429	RATE_LIMITED	Quota	Aguardar Retry-After; checkout: reenviar com o MESMO request_id/client_secret
503	SERVICE_UNAVAILABLE	Falha interna/timeout (10 s) do banco	Retry com backoff; checkout com a MESMA chave

TS

```
// src/services/storefront.service.ts (PROPOSTA)
const URL_ = (import.meta.env.VITE_STOREFRONT_API_URL as string)
  || `${import.meta.env.VITE_SUPABASE_URL}/functions/v1/storefront-api`
const ANON = import.meta.env.VITE_SUPABASE_ANON_KEY as string

export class StorefrontError extends Error {
  constructor(public status: number, public code: StorefrontErrorCode | 'NETWORK_ERROR', public retryAfter?: number) {
super(code) }
  get retryable() { return this.status === 0 || this.status === 429 || this.status === 503 }
}

async function call<T>(body: object, signal?: AbortSignal): Promise<T> {
  let res: Response
  try {
    res = await fetch(URL_, {
      method: 'POST',
      headers: { 'content-type': 'application/json', apikey: ANON },   // sem Authorization do usuário
      body: JSON.stringify(body),
      signal,
    })
  } catch { throw new StorefrontError(0, 'NETWORK_ERROR') }
  const json = await res.json().catch(() => ({ error: 'SERVICE_UNAVAILABLE' }))
  if (!res.ok || 'error' in json) {
    throw new StorefrontError(res.status, json.error ?? 'SERVICE_UNAVAILABLE', Number(res.headers.get('retry-after')) ||
undefined)
  }
  return json.data as T
}

export const storefrontApi = {
  catalog: (store_slug: string, filters: CatalogFilters = {}, signal?: AbortSignal) =>
    call<CatalogData>({ action: 'catalog', store_slug, filters }, signal),
  product: (store_slug: string, product_slug: string) =>
    call<ProductData>({ action: 'product', store_slug, product_slug }),
  checkout: (req: Omit<CheckoutRequest, 'action'>) =>
    call<CheckoutData>({ action: 'checkout', ...req }),
  orderStatus: (store_slug: string, order_id: string, token: string) =>
    call<OrderStatusData>({ action: 'order-status', store_slug, order_id, token }),
}
```

19.2 Ação `catalog` — /s/:storeSlug

JSON

```
// Request
{ "action": "catalog", "store_slug": "doceria-da-ana",
  "filters": { "search": "brigadeiro", "category_id": null, "min_price": 0, "max_price": 50,
               "in_stock": true, "sort": "price_asc", "page": 1, "page_size": 24 } }
// 200
{ "data": {
  "store": { "name": "Doceria da Ana", "title": "...", "subtitle": "...", "slug": "doceria-da-ana",
             "logo_path": "<store_id>/<uuid>.png", "banner_path": null, "primary_color": "#6750A4",
             "secondary_color": "#03A9F4", "social_links": { "instagram": "https://..." }, "whatsapp_number":
"+5511999999999" },
  "categories": [ { "id": "...", "name": "Doces", "slug": "doces", "parent_id": null } ],
  "products": [ { "id": "...", "code": "PRD-000001", "name": "Brigadeiro", "slug": "brigadeiro", "description": "...",
                  "price": 4.5, "category_id": "...", "is_featured": true, "in_stock": true,
```

```
                "images": [ { "path": "<store>/<product>/<uuid>.webp", "alt_text": "", "sort_order": 0 } ],
                "attributes": [ { "name": "Peso", "value": "20 g" } ] } ],
  "total": 37, "page": 1, "page_size": 24 } }
```

Item de UI	Regra
Tema	Aplicar `primary_color` / `secondary_color` no tema Vuetify do StorefrontLayout (`theme.themes.value.storefront.colors.primary = …`)
Logo/banner	`publicUrl('store-logos', logo_path)`, `publicUrl('store-banners', banner_path)`
Filtros na URL	`?q=&categoria=<uuid>&min=&max=&estoque=1&ordem=price_asc&pagina=2`; converter para `filters` validando tipos (min/max number, página inteira 1..1000). Valores inválidos → descartar
Categorias	Menu de 2 níveis a partir de `parent_id`. Selecionar a mãe inclui as subcategorias (servidor). Não há rota pública de categoria: usar o filtro
Preço	Exibir `price`. Nunca exibir custo (não é retornado)
Estoque	Somente booleano `in_stock` ("Esgotado"); quantidade não é exposta
Paginação	`total / page_size`; máximo 24 por página
Debounce	Busca 300–400 ms; cancelar request anterior com AbortController
404	`NOT_FOUND` → StorefrontNotFound ("Loja indisponível")

19.3 Ação `product` — /s/:storeSlug/produto/:productSlug

JSON

```
{ "action": "product", "store_slug": "doceria-da-ana", "product_slug": "brigadeiro" }
// 200 { "data": PublicProduct }   — 404 NOT_FOUND se inativo/arquivado/categoria oculta
```

Galeria com `images` ordenadas (≤4), atributos em tabela, descrição como texto (escape). Seletor de quantidade: inteiro 1..999. "Adicionar ao carrinho" desabilitado se `!in_stock`. Como a quantidade disponível não é exposta, a validação final de estoque ocorre no checkout (409 INSUFFICIENT_AVAILABLE_STOCK). O header da loja (logo/cores) vem do catálogo: carregar `catalog` com `page_size: 1` no layout (cache por slug) quando o usuário entra direto na página do produto.

19.4 Carrinho — /s/:storeSlug/carrinho

- Estado local por loja: `localStorage["vibestore:cart:{slug}"] = [{ product_id, slug, name, price, image_path, quantity }]`. O preço é só exibição; o servidor recalcula.
- Limites espelhados: 1..20 itens distintos, quantidade inteira 1..999, sem produto repetido (somar quantidades).
- Revalidar ao abrir o carrinho: para cada item, `storefrontApi.product(slug, item.slug)` (ou catálogo) → atualizar nome/preço/in_stock; remover 404; avisar mudanças de preço.
- Subtotal estimado = Σ price × qty. Texto fixo: "Frete e pagamento combinados pelo WhatsApp após o pedido."

19.5 Ação `checkout` — /s/:storeSlug/checkout

Campo	Regra (Edge + SQL)
customer.full_name	2..120 (trim)
customer.phone	E.164 `^\+[1-9][0-9]{7,14}$` — UI com máscara BR e conversão para `+55DDDNUMERO`
address.postal_code	8 dígitos, sem hífen (máscara 00000-000 → remover hífen)
address.street / number / district / city	1..160 cada (obrigatórios)
address.state	UF maiúscula (`v-select` com as 27 UFs)
address.complement	opcional, ≤ 160 (omitir se vazio)
note	opcional, ≤ 1000
items	1..20, `{ product_id, quantity }` apenas — **sem preço** (campo extra → 400 UNKNOWN_FIELD)

Campo	Regra (Edge + SQL)
request_id	UUID v4 gerado ao iniciar a tentativa; reutilizado em retries
client_secret	32 bytes aleatórios base64url sem `=` (43 chars) gerado junto; reutilizado em retries

Sem login e sem lookup

- O checkout não exige conta. Não existe busca de cliente por telefone nem preenchimento automático de nome/endereço (não há endpoint, por privacidade).
- Se o telefone já existe na loja, o cadastro do cliente **não é sobrescrito**; o nome/endereço deste pedido ficam no snapshot do pedido.
- Telefone não autentica ninguém: não usar para “ver meus pedidos”.

TS

```
// src/stores/checkout.ts — submissão idempotente (PROPOSTA)
interface PendingCheckout { request_id: string; client_secret: string; input: CheckoutInput; fingerprint: string }
const KEY = (slug: string) => `vibestore:checkout:${slug}`

export async function submitCheckout(slug: string, input: CheckoutInput): Promise<CheckoutData> {
  const fingerprint = JSON.stringify(input)
  let p: PendingCheckout | null = JSON.parse(sessionStorage.getItem(KEY(slug)) ?? 'null')
  if (!p || p.fingerprint !== fingerprint) {
    // NOVA intenção (primeira tentativa ou dados alterados) => nova chave e novo segredo
    p = { request_id: crypto.randomUUID(), client_secret: newClientSecret(), input, fingerprint }
    sessionStorage.setItem(KEY(slug), JSON.stringify(p))
  }
  try {
    const data = await storefrontApi.checkout({ store_slug: slug, request_id: p.request_id, client_secret: p.client_secret,
input: p.input })
    // SUCESSO (inclusive replayed=true): limpar payload sensível
    sessionStorage.removeItem(KEY(slug))
    useCartStore().clear(slug)
    return data
  } catch (e) {
    if (e instanceof StorefrontError && e.retryable) throw e      // manter p: retry usa a MESMA chave
    if (e instanceof StorefrontError && e.code === 'IDEMPOTENCY_KEY_REUSED') sessionStorage.removeItem(KEY(slug))
    throw e                                                        // 400/409: usuário corrige → fingerprint muda
  }
}
```

JSON

```
// Response 200
{ "data": {
  "order_id": "6f0c…", "code": "PED-000123", "subtotal": 90, "status": "PENDING",
  "reservation_expires_at": "2026-10-07T15:04:05.123Z", "replayed": false,
  "whatsapp_number": "+5511999999999",
  "token": "q9…(43 chars)",
  "tracking_url": "https://app.vibestore.com.br/s/doceria-da-ana/pedido/6f0c…#token=q9…",
  "whatsapp_url": "https://wa.me/5511999999999?text=Ol%C3%A1!%20Fiz%20o%20pedido…",
  "shipping_included": false } }
```

Após sucesso	Regra
Tela de confirmação	Código do pedido, subtotal, aviso “Frete e pagamento a combinar”, prazo da reserva (24 h, `reservation_expires_at`), botão principal “Enviar pedido no WhatsApp” (`whatsapp_url`, `target="_blank" rel="noopener noreferrer"`)
Link de acompanhamento	Botão “Copiar link de acompanhamento” (`tracking_url`) e aviso: “Guarde este link; ele é a única forma de acompanhar o pedido.” O link também vai na mensagem do WhatsApp
Navegação	`router.replace({ name: 'storefront-order', params: { storeSlug, orderId: data.order_id }, hash: '#token=' + data.token })` — ou exibir confirmação e oferecer o link
Limpeza	Remover request_id/client_secret/input do sessionStorage e esvaziar carrinho. Não manter o token em localStorage

Após sucesso	Regra
replayed=true	Mesmo tratamento de sucesso (a reserva NÃO é renovada; mostrar a expiração retornada)

19.6 Ação `order-status` — /s/:storeSlug/pedido/:orderId#token=…

TS

```
// StorefrontOrderPage.vue — leitura do fragmento (PROPOSTA)
const TOKEN_RE = /^[A-Za-z0-9_-]{43}$/
const tokenKey = (orderId: string) => `vibestore:order-token:${orderId}`

onMounted(async () => {
  const fromHash = new URLSearchParams(window.location.hash.slice(1)).get('token')
  if (fromHash && TOKEN_RE.test(fromHash)) {
    sessionStorage.setItem(tokenKey(orderId), fromHash)
    // remove o token da barra de endereço/histórico (não vai em Referer, analytics, prints)
    history.replaceState(history.state, '', window.location.pathname + window.location.search)
  }
  const token = sessionStorage.getItem(tokenKey(orderId))
  if (!token) { state.value = 'missing-token'; return }
  try { order.value = await storefrontApi.orderStatus(storeSlug, orderId, token) }
  catch (e) { state.value = e instanceof StorefrontError && e.status === 404 ? 'not-found' : 'error' }
})
```

- O fragmento `#token` nunca é enviado ao servidor HTTP pelo navegador; o token vai no **body** do POST. Nunca colocar o token em query string, analytics, logs, Sentry breadcrumbs ou `Referer` (meta `<meta name="referrer" content="no-referrer">` na página).
- Sem token → tela “Abra o link completo recebido no WhatsApp”. 404 → “Pedido não encontrado ou link inválido”.
- Exibir: status + timeline (`timeline[]`), itens, subtotal, frete (“a combinar” se `!shipping_confirmed`), total, pagamento recebido/estornado, rastreio, endereço/cliente do snapshot, reserva ativa/expirada.
- Atualização: botão “Atualizar” e polling opcional a cada 60 s com a aba visível (limite 120/min global por ação). Não há realtime público.
- Funciona mesmo com a loja despublicada (exige apenas loja não arquivada e dono ativo); quebra se o slug da loja mudar.

19.7 StorefrontNotFound

Rota curinga dentro de `/s/:storeSlug` e destino de qualquer 404 da Edge. Mensagem genérica (“Loja ou página indisponível”) sem distinguir “não existe” de “não publicada” (o backend também não distingue). Sem links para o painel.

Aceitação: nenhum request do navegador para `/rest/v1/rpc/get_public_*` ; catálogo com filtros reflete a URL; checkout reenviado após falha de rede retorna o mesmo pedido (`replayed:true`); alterar um campo após falha gera nova chave; token removido da barra de endereço após a primeira leitura; preço nunca é enviado.

20. Catálogo completo de RPCs (40 = 35 client + 5 service-only)

O manifesto contém **40 RPCs** em `public`: **35 executáveis por `authenticated`** (frontend autenticado via `supabase.rpc`) e **5 service-only** (EXECUTE apenas para `service_role`; chamadas exclusivamente pela Edge `storefront-api`). Nenhuma é executável por `anon`. As assinaturas abaixo são copiadas do manifesto (`arguments` / `result`); interfaces TypeScript completas no Apêndice A.

RPC	Módulo	EXECUTE	nº defaults	Retorno (TS)
`get_my_access`	001	authenticated	0	MyAccess
`sudo_set_user_active`	001	authenticated	0	void (data === null)
`check_store_slug`	002	authenticated	0	CheckStoreSlugResult
`create_store`	002	authenticated	0	CreateStoreResult
`get_store_access`	002	authenticated	0	StoreAccess
`set_store_publication`	002	authenticated	0	SetStorePublicationResult
`archive_store`	002	authenticated	0	ArchiveStoreResult
`create_category`	003	authenticated	0	CreateCategoryResult
`update_category`	003	authenticated	0	UpdateCategoryResult
`archive_category`	003	authenticated	0	ArchiveCategoryResult
`create_product`	003	authenticated	0	CreateProductResult
`update_product`	003	authenticated	0	UpdateProductResult
`archive_product`	003	authenticated	0	ArchiveProductResult
`replace_product_attributes`	003	authenticated	0	ReplaceProductAttributesResult
`attach_product_image`	003	authenticated	1	AttachProductImageResult
`detach_product_image`	003	authenticated	0	DetachProductImageResult
`reorder_product_images`	003	authenticated	0	ReorderProductImagesResult
`register_inventory_movement`	004	authenticated	2	InventoryMovementResult
`adjust_inventory_count`	004	authenticated	2	AdjustInventoryCountResult
`adjust_inventory_batch`	004	authenticated	1	AdjustInventoryBatchResult
`reverse_inventory_movement`	004	authenticated	0	ReverseInventoryMovementResult
`consume_public_rate_limit`	005	service_role	1	{ allowed: boolean; retry_after: number }
`get_public_catalog`	005	service_role	1	CatalogData | null
`get_public_product`	005	service_role	0	PublicProduct | null
`create_public_order`	005	service_role	0	CheckoutData (parte do banco)
`get_public_order_status`	005→006 (versão FINAL do 006)	service_role	0	OrderStatusData | null
`update_order_terms`	006	authenticated	0	OrderOperationResult
`confirm_order_payment`	006	authenticated	1	OrderOperationResult
`ship_order`	006	authenticated	1	OrderOperationResult
`deliver_order`	006	authenticated	0	OrderOperationResult
`cancel_order`	006	authenticated	3	OrderOperationResult
`return_order`	006	authenticated	1	OrderOperationResult
`create_expense_category`	007	authenticated	0	CreateExpenseCategoryResult
`update_expense_category`	007	authenticated	0	UpdateExpenseCategoryResult
`record_expense`	007	authenticated	2	RecordExpenseResult
`reverse_expense`	007	authenticated	3	ReverseExpenseResult
`get_erp_summary`	007	authenticated	1	ErpSummary

RPC	Módulo	EXECUTE	nº defaults	Retorno (TS)
`get_erp_daily`	007	authenticated	1	ErpDaily
`get_erp_product_performance`	007	authenticated	2	ErpProductPerformance
`get_erp_expense_breakdown`	007	authenticated	1	ErpExpenseBreakdown

20.1 RPCs client (35) — chamadas pelo frontend autenticado

`get_my_access` client

módulo 001

`public.get_my_access() RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Qualquer usuário autenticado (sem checagem de permissão).
Entrada (TS)	`— (sem argumentos)`
Saída (TS)	`MyAccess`
Regras	• Retorna papéis e permissões GLOBAIS de `user_platform_roles` (USER/SUDO). OWNER nunca aparece: é contextual à loja (use `get_store_access`). • Se a conta estiver inativa (`profiles.is_active=false`), `account_active=false` e as listas vêm vazias. • `can_manage_security=true` somente para SUDO ativo com `security.manage` + `auth.access`: é a condição do layout /sudo.
Erros	—
Efeitos	Somente leitura (STABLE).

`sudo_set_user_active` client

módulo 001

`public.sudo_set_user_active(p_user_id uuid, p_active boolean) RETURNS void`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	`can_manage_security()` (SUDO + security.manage + auth.access).
Entrada (TS)	`SudoSetUserActiveArgs`
Saída (TS)	`void (data === null)`
Regras	• Não permite desativar a si mesmo. • Grava `security_audit_logs` (UPDATE profiles) com antes/depois. • Desativar um proprietário torna a vitrine dele indisponível (as funções públicas exigem dono ativo) e bloqueia toda RPC dele (has_store_permission exige conta ativa).
Erros	`FORBIDDEN` `ACTIVE_REQUIRED` `CANNOT_DISABLE_SELF` `USER_NOT_FOUND`
Efeitos	UPDATE profiles.is_active + auditoria.

`check_store_slug` client

módulo 002

`public.check_store_slug(p_slug text) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Global `auth.access` + `store.create` (papel USER).
Entrada (TS)	`CheckStoreSlugArgs`
Saída (TS)	`CheckStoreSlugResult`
Regras	• Normaliza com lower(trim()). Inválido se fora de 3..63, fora do regex `^[a-z0-9]+(-[a-z0-9]+)*$` ou reservado (admin, api, app, auth, login, signup, dashboard, settings, sudo, supabase) → `{valid:false, available:false}`. • `available` considera TODAS as lojas, inclusive arquivadas (slug nunca é reaproveitado). • É consultivo: corrida entre check e create é resolvida por `SLUG_ALREADY_EXISTS`.
Erros	`STORE_CREATE_FORBIDDEN`
Efeitos	Somente leitura.

`create_store` client

módulo 002

`public.create_store(p_request_id uuid, p_plan_id uuid, p_input jsonb) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Global `auth.access` + `store.create`; conta ativa.
Entrada (TS)	`CreateStoreArgs (p_input: CreateStoreInput)`
Saída (TS)	`CreateStoreResult`
Regras	• Idempotente por `(owner_id, p_request_id)`: mesma chave + mesmo payload normalizado → `created:false` (replay). Payload diferente → IDEMPOTENCY_KEY_REUSED. • Uma loja NÃO arquivada por usuário (índice `stores_one_live_per_owner`). • `p_plan_id` deve ser o plano `FREE`, preço 0, ativo. • Chaves aceitas em p_input: name, title, subtitle, slug, whatsapp_number, cnpj, email, primary_color, secondary_color, social_links. Todas string/null, exceto social_links (objeto). • A loja nasce `is_published=false` (rascunho), sem logo/banner.
Erros	`STORE_CREATE_FORBIDDEN` `REQUEST_AND_PLAN_REQUIRED` `INVALID_STORE_INPUT` `UNKNOWN_STORE_FIELD` `INVALID_STORE_FIELD_TYPE` `INVALID_STORE_NAME_OR_TITLE` `INVALID_STORE_SLUG` `INVALID_WHATSAPP_NUMBER` `INVALID_SOCIAL_LINKS` `ACCOUNT_UNAVAILABLE` `IDEMPOTENCY_KEY_REUSED` `ONBOARDING_ALREADY_ARCHIVED` `ACTIVE_STORE_ALREADY_EXISTS` `PLAN_UNAVAILABLE` `OWNER_ACCESS_UNAVAILABLE` `SLUG_ALREADY_EXISTS` `23514 (CHECK: e-mail/cor/cnpj inválidos)`
Efeitos	INSERT stores. O usuário passa a ser OWNER contextual da loja.

`get_store_access` client

módulo 002

`public.get_store_access(p_store_id uuid) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Qualquer autenticado; retorna `null` sem `store.read`.
Entrada (TS)	`{ p_store_id: Uuid }`
Saída (TS)	`StoreAccess`
Regras	• `null` se a loja não existe, está arquivada ou o usuário não tem `store.read` nela. • `role` = OWNER se `owner_id = auth.uid()` ; caso contrário SUDO (acesso via `store.admin`). • `permissions` é a lista efetiva usada pelas RPCs — use-a para habilitar/ocultar ações (a autorização real continua no banco).
Erros	—
Efeitos	Somente leitura.

`set_store_publication` client

módulo 002

`public.set_store_publication(p_store_id uuid, p_published boolean) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `store.publish`.
Entrada (TS)	`SetStorePublicationArgs`
Saída (TS)	`SetStorePublicationResult`
Regras	• Loja arquivada não pode ser publicada. • Publicar habilita a vitrine via Edge (exige também dono ativo e OWNER com storefront.read).
Erros	`PUBLICATION_REQUIRED` `STORE_UNAVAILABLE`
Efeitos	UPDATE stores.is_published.

`archive_store` client

módulo 002

`public.archive_store(p_store_id uuid) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `store.archive`.
Entrada (TS)	`ArchiveStoreArgs`
Saída (TS)	`ArchiveStoreResult`
Regras	• Idempotente: já arquivada → `already_archived:true`. • Despublica e define `archived_at`. Irreversível pelo cliente (não há RPC de desarquivar). • Bloqueada se houver saldo de estoque > 0 (STORE_HAS_STOCK) ou pedidos PENDING/PAID/SHIPPED (STORE_HAS_OPEN_ORDERS). • Após arquivar, a loja some do SELECT (policy exige archived_at IS NULL) e o usuário pode criar outra loja; o slug antigo continua reservado.
Erros	`STORE_UNAVAILABLE` `STORE_HAS_STOCK` `STORE_HAS_OPEN_ORDERS`
Efeitos	UPDATE stores (is_published=false, archived_at=now()).

`create_category` client

módulo 003

`public.create_category(p_store_id uuid, p_request_id uuid, p_input jsonb) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `catalog.categories.manage`.
Entrada (TS)	`CreateCategoryArgs`
Saída (TS)	`CreateCategoryResult`
Regras	• Idempotente por `(store_id, request_id)`; replay → `created:false`. Replay de categoria arquivada → CATEGORY_ARCHIVED. • Código sequencial gerado no servidor: `CAT-000001` (imutável). • Slug ausente/vazio é gerado a partir do nome (minúsculas, sem acento, hífens). Único por loja, inclusive arquivadas. • Hierarquia de no máximo 2 níveis: o pai precisa ser raiz e não arquivado; categoria com filhos não pode virar filha.
Erros	`CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `REQUEST_REQUIRED` `INVALID_CATALOG_INPUT` `UNKNOWN_CATALOG_FIELD` `INVALID_CATEGORY_FIELD_TYPE` `IDEMPOTENCY_KEY_REUSED` `CATEGORY_ARCHIVED` `CATEGORY_SLUG_EXISTS` `CATEGORY_PARENT_UNAVAILABLE` `CATEGORY_MAX_TWO_LEVELS` `CATEGORY_WITH_CHILDREN_CANNOT_BECOME_CHILD` `23514 (nome/slug/sort_order fora do CHECK)`
Efeitos	INSERT categories + auditoria.

`update_category` client

módulo 003

`public.update_category(p_category_id uuid, p_patch jsonb) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `catalog.categories.manage`.
Entrada (TS)	`UpdateCategoryArgs`
Saída (TS)	`UpdateCategoryResult`
Regras	• PATCH: somente chaves presentes são alteradas. `parent_id: null` ou `""` torna raiz. • Slug enviado NÃO é regenerado: deve ser válido (2..100, regex). Não é idempotente por request_id (operação de sobrescrita). • Desativar (`is_active:false`) oculta da vitrine a categoria e os produtos vinculados (inclusive de subcategorias de uma mãe inativa).
Erros	`CATEGORY_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `INVALID_CATALOG_INPUT` `UNKNOWN_CATALOG_FIELD` `INVALID_CATEGORY_FIELD_TYPE` `CATEGORY_PARENT_UNAVAILABLE` `CATEGORY_MAX_TWO_LEVELS` `CATEGORY_WITH_CHILDREN_CANNOT_BECOME_CHILD` `23505 categories_store_slug` `23514`
Efeitos	UPDATE categories.

`archive_category` client

módulo 003

`public.archive_category(p_category_id uuid) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `catalog.categories.manage`.
Entrada (TS)	`ArchiveCategoryArgs`
Saída (TS)	`ArchiveCategoryResult`
Regras	• Idempotente (`already_archived`). • Bloqueada se houver subcategorias vivas ou produtos vivos vinculados: mova/arquive antes. • Arquivada some do SELECT (policy) e não pode ser editada.
Erros	`CATEGORY_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `CATEGORY_HAS_LIVE_CHILDREN` `CATEGORY_HAS_LIVE_PRODUCTS`
Efeitos	UPDATE categories (is_active=false, archived_at).

`create_product` client

módulo 003

`public.create_product(p_store_id uuid, p_request_id uuid, p_input jsonb) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `catalog.products.manage`.
Entrada (TS)	`CreateProductArgs`
Saída (TS)	`CreateProductResult`
Regras	• Idempotente por `(store_id, request_id)`. Payload comparado APÓS normalização (trim, slug gerado, defaults). • SKU = `code` gerado no servidor (`PRD-000001`), imutável. O frontend NÃO envia código. • `price` obrigatório (number, até 2 casas). `cost_price` é custo de referência — o custo real de estoque é o custo médio do ledger. • Status default INACTIVE: produto não aparece na vitrine até ser ACTIVE. • `attributes` (≤30, nomes únicos ignorando caixa) gravados na mesma transação.
Erros	`CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `REQUEST_REQUIRED` `INVALID_CATALOG_INPUT` `UNKNOWN_CATALOG_FIELD` `INVALID_PRODUCT_FIELD_TYPE` `INVALID_MONEY_VALUE` `ATTRIBUTES_ARRAY_REQUIRED` `ATTRIBUTES_LIMIT_30` `INVALID_ATTRIBUTE` `UNKNOWN_ATTRIBUTE_FIELD` `INVALID_ATTRIBUTE_NAME_OR_VALUE` `DUPLICATE_ATTRIBUTE_NAME` `IDEMPOTENCY_KEY_REUSED` `PRODUCT_ARCHIVED` `PRODUCT_SLUG_EXISTS` `PRODUCT_CATEGORY_UNAVAILABLE` `22P02 (status inválido / uuid inválido)` `23514`
Efeitos	INSERT products + product_attributes + auditoria. Saldo inicial NÃO é criado (use entrada de estoque).

`update_product` client

módulo 003

`public.update_product(p_product_id uuid, p_patch jsonb) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `catalog.products.manage`.
Entrada (TS)	`UpdateProductArgs`
Saída (TS)	`UpdateProductResult`
Regras	• PATCH por chave presente. `category_id: null` remove categoria. `attributes` presente substitui a lista inteira. • Produto arquivado não pode ser alterado. Não é idempotente por request_id.
Erros	`PRODUCT_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `INVALID_CATALOG_INPUT` `UNKNOWN_CATALOG_FIELD` `INVALID_PRODUCT_FIELD_TYPE` `INVALID_MONEY_VALUE` `PRODUCT_CATEGORY_UNAVAILABLE` `ATTRIBUTES_LIMIT_30` `DUPLICATE_ATTRIBUTE_NAME` `INVALID_ATTRIBUTE_NAME_OR_VALUE` `23505 products_store_slug` `23514` `22P02`
Efeitos	UPDATE products (+ replace attributes).

`archive_product` client

módulo 003

`public.archive_product(p_product_id uuid) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `catalog.products.manage`.
Entrada (TS)	`ArchiveProductArgs`
Saída (TS)	`ArchiveProductResult`
Regras	• Idempotente. Define status INACTIVE + archived_at. • Bloqueado com saldo > 0 (PRODUCT_HAS_STOCK) ou pedidos PENDING/PAID/SHIPPED contendo o produto (PRODUCT_HAS_OPEN_ORDERS). • Depois de arquivado, imagens NÃO podem mais ser desvinculadas nem apagadas do Storage: limpe mídia antes, se desejar.
Erros	`PRODUCT_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `PRODUCT_HAS_STOCK` `PRODUCT_HAS_OPEN_ORDERS`
Efeitos	UPDATE products.

`replace_product_attributes` client

módulo 003

`public.replace_product_attributes(p_product_id uuid, p_attributes jsonb) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `catalog.products.manage`.
Entrada (TS)	`ReplaceProductAttributesArgs`
Saída (TS)	`ReplaceProductAttributesResult`
Regras	• Apaga e reinsere todos os atributos na ordem do array (sort_order 0..29). Array vazio limpa.
Erros	`PRODUCT_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `ATTRIBUTES_ARRAY_REQUIRED` `ATTRIBUTES_LIMIT_30` `INVALID_ATTRIBUTE` `UNKNOWN_ATTRIBUTE_FIELD` `INVALID_ATTRIBUTE_NAME_OR_VALUE` `DUPLICATE_ATTRIBUTE_NAME`
Efeitos	DELETE + INSERT product_attributes.

`attach_product_image` client

módulo 003

`public.attach_product_image(p_product_id uuid, p_path text, p_alt_text text DEFAULT ''::text) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `catalog.products.media`.
Entrada (TS)	`AttachProductImageArgs`
Saída (TS)	`AttachProductImageResult`
Regras	• O objeto precisa JÁ existir no bucket `product-images` no caminho `{store_id}/{product_id}/{uuid}.{png|jpg|jpeg|webp}` (hex minúsculo). • Idempotente por caminho: mesmo path no mesmo produto → `created:false`. • Ocupa a menor posição livre 0..3 (0 = capa). 5ª imagem → PRODUCT_IMAGES_LIMIT_4. • alt_text ≤ 200.
Erros	`PRODUCT_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `PRODUCT_IMAGES_LIMIT_4` `INVALID_PRODUCT_IMAGE_PATH` `PRODUCT_IMAGE_OBJECT_NOT_FOUND` `23505 product_images_path_key` `23514`
Efeitos	INSERT product_images.

`detach_product_image` client

módulo 003

`public.detach_product_image(p_product_id uuid, p_image_id uuid) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `catalog.products.media`.
Entrada (TS)	`DetachProductImageArgs`
Saída (TS)	`DetachProductImageResult`
Regras	• Remove o vínculo e devolve `path` para o frontend apagar o objeto com `storage.remove([path])`. • `detached:false, path:null` se o vínculo não existia (noop). • Não reordena as demais: posições podem ficar com lacunas (a próxima imagem preenche a menor livre).
Erros	`PRODUCT_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE`
Efeitos	DELETE product_images.

`reorder_product_images` client

módulo 003

`public.reorder_product_images(p_product_id uuid, p_image_ids uuid[]) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `catalog.products.media`.
Entrada (TS)	`ReorderProductImagesArgs`
Saída (TS)	`ReorderProductImagesResult`
Regras	• Envie TODOS os ids vinculados, sem repetição, na nova ordem; posição = índice (0 = capa).
Erros	`PRODUCT_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `INVALID_IMAGE_ORDER`
Efeitos	UPDATE product_images.sort_order.

`register_inventory_movement` client

módulo 004

```
public.register_inventory_movement(p_product_id uuid, p_request_id uuid, p_type text, p_quantity integer, p_reason text, p_unit_cost numeric DEFAULT NULL::numeric, p_reference text DEFAULT NULL::text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `inventory.entry` (ENTRY) ou `inventory.exit` (EXIT).
Entrada (TS)	`RegisterInventoryMovementArgs`
Saída (TS)	`InventoryMovementResult`
Regras	• Somente ENTRY ou EXIT manuais. Quantidade inteira positiva; o sinal é aplicado pelo servidor. • ENTRY exige `p_unit_cost` (até 6 casas) e recalcula custo médio móvel. EXIT não aceita custo: valor sai pelo custo médio atual. • EXIT não pode reduzir o saldo abaixo das reservas ativas (STOCK_RESERVED_BY_PENDING_ORDERS) nem abaixo de zero (INSUFFICIENT_STOCK). Limite prático na UI: `available_quantity`. • Idempotente por `(store_id, request_id)`: replay → resultado original + `replayed:true`. • Saída manual NÃO é venda: aparece no ERP como "saída não classificada" (needs_stock_review).
Erros	`MANUAL_TYPE_MUST_BE_ENTRY_OR_EXIT` `INVENTORY_FORBIDDEN` `PRODUCT_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `INVALID_MOVEMENT_REASON_OR_REFERENCE` `QUANTITY_MUST_BE_POSITIVE_INTEGER` `INVALID_UNIT_COST` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `INVALID_MOVEMENT_QUANTITY_OR_REQUEST` `INSUFFICIENT_STOCK` `STOCK_QUANTITY_LIMIT` `INVALID_MOVEMENT_SIGN` `OUTBOUND_COST_IS_AUTOMATIC` `INVALID_UNIT_COST` `STOCK_RESERVED_BY_PENDING_ORDERS` `INVALID_LEDGER_CHAIN`
Efeitos	INSERT inventory_movements (ledger imutável) → trigger atualiza inventory_balances (revision+1).

`adjust_inventory_count` client

módulo 004

```
public.adjust_inventory_count(p_product_id uuid, p_request_id uuid, p_counted_quantity integer, p_expected_revision bigint, p_reason text, p_unit_cost numeric DEFAULT NULL::numeric, p_reference text DEFAULT NULL::text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `inventory.adjust`.
Entrada (TS)	`AdjustInventoryCountArgs`
Saída (TS)	`AdjustInventoryCountResult`
Regras	• Informe a quantidade CONTADA (absoluta). O servidor calcula delta = contado − saldo e gera ADJUSTMENT_IN/OUT. • Controle otimista: `p_expected_revision` deve ser igual a `inventory_balances.revision` atual (produto sem movimento = 0). • Sem mudança (delta 0) → `changed:false`, nenhum movimento. • `p_unit_cost` só é aceito quando o ajuste aumenta o saldo; sem custo, um aumento usa o custo médio atual (ou exige custo se saldo era 0 → INVALID_UNIT_COST). • A idempotência é verificada ANTES da revisão: retry com mesma chave e payload devolve o resultado gravado, mesmo que a revisão já tenha avançado.
Erros	`INVENTORY_FORBIDDEN` `PRODUCT_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `INVALID_MOVEMENT_REASON_OR_REFERENCE` `INVALID_COUNT_OR_EXPECTED_REVISION` `INVALID_UNIT_COST` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `STALE_INVENTORY_REVISION` `UNIT_COST_NOT_ALLOWED_FOR_OUTBOUND_OR_NO_CHANGE` `INVALID_MOVEMENT_QUANTITY_OR_REQUEST` `INSUFFICIENT_STOCK` `STOCK_QUANTITY_LIMIT` `INVALID_MOVEMENT_SIGN` `OUTBOUND_COST_IS_AUTOMATIC` `INVALID_UNIT_COST` `STOCK_RESERVED_BY_PENDING_ORDERS` `INVALID_LEDGER_CHAIN`
Efeitos	0 ou 1 movimento ADJUSTMENT_* + projeção.

`adjust_inventory_batch` client

módulo 004

```
public.adjust_inventory_batch(p_store_id uuid, p_request_id uuid, p_items jsonb, p_reason text, p_reference text DEFAULT NULL::text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `inventory.adjust` (e o produto acessível).
Entrada (TS)	`AdjustInventoryBatchArgs`
Saída (TS)	`AdjustInventoryBatchResult`
Regras	• 1..50 itens, payload ≤ 32 KiB, sem produto repetido; campos permitidos: product_id, counted_quantity, expected_revision, unit_cost. • Atômico: qualquer erro (revisão desatualizada, reserva, produto indisponível) desfaz o lote inteiro. • Itens processados em ordem de product_id; `items` do resultado segue essa ordem (não a ordem enviada). • Cada movimento recebe `batch_request_id = p_request_id` (filtrável em inventory_movements).
Erros	`CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `INVALID_MOVEMENT_REASON_OR_REFERENCE` `BATCH_ITEMS_ARRAY_REQUIRED` `BATCH_LIMIT_50` `INVALID_BATCH_ITEM` `UNKNOWN_BATCH_FIELD` `INVALID_BATCH_FIELD_TYPE` `BATCH_DUPLICATE_PRODUCT` `INVALID_COUNT_OR_EXPECTED_REVISION` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `BATCH_PRODUCT_UNAVAILABLE` `STALE_INVENTORY_REVISION` `INVALID_UNIT_COST` `UNIT_COST_NOT_ALLOWED_FOR_OUTBOUND_OR_NO_CHANGE` `INVALID_MOVEMENT_QUANTITY_OR_REQUEST` `INSUFFICIENT_STOCK` `STOCK_QUANTITY_LIMIT` `INVALID_MOVEMENT_SIGN` `OUTBOUND_COST_IS_AUTOMATIC` `INVALID_UNIT_COST` `STOCK_RESERVED_BY_PENDING_ORDERS` `INVALID_LEDGER_CHAIN`
Efeitos	N movimentos ADJUSTMENT_* numa transação.

`reverse_inventory_movement` client

módulo 004

```
public.reverse_inventory_movement(p_movement_id uuid, p_request_id uuid, p_reason text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `inventory.reverse`.
Entrada (TS)	`ReverseInventoryMovementArgs`
Saída (TS)	`ReverseInventoryMovementResult`
Regras	• Só o ÚLTIMO movimento do produto (`inventory_balances.last_movement_id`) pode ser revertido, uma única vez, e não pode ser REVERSAL. • Movimentos gerados por pedidos (vinculados em order_stock_operations) não podem ser revertidos: use o fluxo do pedido. • Restaura exatamente quantidade/valor anteriores ao movimento original.
Erros	`INVENTORY_FORBIDDEN` `INVENTORY_FORBIDDEN` `PRODUCT_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `INVALID_MOVEMENT_REASON_OR_REFERENCE` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `INVALID_REVERSAL_SOURCE` `MOVEMENT_ALREADY_REVERSED` `ONLY_LAST_MOVEMENT_CAN_BE_REVERSED` `ORDER_STOCK_REVERSAL_REQUIRES_ORDER_FLOW` `STOCK_RESERVED_BY_PENDING_ORDERS` `INVALID_LEDGER_CHAIN`
Efeitos	INSERT movimento REVERSAL (reversal_of_id) + projeção.

`update_order_terms` client

módulo 006

```
public.update_order_terms(p_order_id uuid, p_request_id uuid, p_expected_revision bigint, p_shipping_amount numeric, p_note text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `orders.terms`.
Entrada (TS)	`UpdateOrderTermsArgs`
Saída (TS)	`OrderOperationResult`
Regras	• Somente PENDING. Define o frete negociado (≥0, 2 casas), marca `shipping_confirmed=true` e total = subtotal + frete. • Pode ser repetida enquanto PENDING (cada uma avança revision e gera evento TERMS_UPDATED). Frete 0 é válido e precisa ser confirmado explicitamente.
Erros	`ORDER_FORBIDDEN` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `ORDER_NOTE_REQUIRED` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `STALE_ORDER_REVISION` `INVALID_SHIPPING_AMOUNT` `ORDER_NOT_PENDING` `ORDER_TOTAL_LIMIT`
Efeitos	UPDATE orders + history TERMS_UPDATED.

`confirm_order_payment` client

módulo 006

```
public.confirm_order_payment(p_order_id uuid, p_request_id uuid, p_expected_revision bigint, p_amount numeric, p_method text, p_note text, p_reference text DEFAULT NULL::text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `orders.pay` + `inventory.exit`.
Entrada (TS)	`ConfirmOrderPaymentArgs`
Saída (TS)	`OrderOperationResult`
Regras	• Somente PENDING, com frete confirmado e `p_amount === orders.total`. • Declaração MANUAL de recebimento (PIX/CASH/BANK_TRANSFER/CARD/OTHER) — não consulta banco nem gateway. • Consome as reservas do próprio pedido e baixa o estoque físico (movimento EXIT por item, custo médio vigente → CMV). Se a reserva já expirou e o estoque foi usado por outro pedido, falha com INSUFFICIENT_AVAILABLE_STOCK_FOR_PAYMENT.
Erros	`ORDER_FORBIDDEN` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `ORDER_NOTE_REQUIRED` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `STALE_ORDER_REVISION` `INVALID_MANUAL_PAYMENT_METHOD` `ORDER_NOT_PENDING` `SHIPPING_TERMS_NOT_CONFIRMED` `PAYMENT_MUST_MATCH_ORDER_TOTAL` `INSUFFICIENT_AVAILABLE_STOCK_FOR_PAYMENT` `INVENTORY_FORBIDDEN` `PRODUCT_UNAVAILABLE` `INSUFFICIENT_STOCK`
Efeitos	reservations CONSUMED; EXIT por item; order_stock_operations SALE; order_payments RECEIPT; status PAID; history PAYMENT_CONFIRMED.

`ship_order` client

módulo 006

```
public.ship_order(p_order_id uuid, p_request_id uuid, p_expected_revision bigint, p_note text, p_tracking_code text DEFAULT NULL::text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `orders.ship`.
Entrada (TS)	`ShipOrderArgs`
Saída (TS)	`OrderOperationResult`
Regras	• Somente PAID. tracking_code opcional (≤120). Nota obrigatória (3..1000).
Erros	`ORDER_FORBIDDEN` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `ORDER_NOTE_REQUIRED` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `STALE_ORDER_REVISION` `INVALID_TRACKING_CODE` `ORDER_NOT_PAID`
Efeitos	status SHIPPED, shipped_at, tracking_code; history SHIPPED.

`deliver_order` client

módulo 006

```
public.deliver_order(p_order_id uuid, p_request_id uuid, p_expected_revision bigint, p_note text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `orders.deliver`.
Entrada (TS)	`DeliverOrderArgs`
Saída (TS)	`OrderOperationResult`
Regras	• Somente SHIPPED.
Erros	`ORDER_FORBIDDEN` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `ORDER_NOTE_REQUIRED` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `STALE_ORDER_REVISION` `ORDER_NOT_SHIPPED`
Efeitos	status DELIVERED, delivered_at; history DELIVERED.

`cancel_order` client

módulo 006

```
public.cancel_order(p_order_id uuid, p_request_id uuid, p_expected_revision bigint, p_note text, p_refund_confirmed boolean DEFAULT false, p_refund_method text DEFAULT NULL::text, p_refund_reference text DEFAULT NULL::text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `orders.cancel` (+ `inventory.entry` quando PAID).
Entrada (TS)	`CancelOrderArgs`
Saída (TS)	`OrderOperationResult`
Regras	• PENDING: sem campos de reembolso (enviar false/null) → libera reservas (RELEASED). • PAID: `p_refund_confirmed=true` + método obrigatórios → registra REFUND do total integral (inclui frete) e repõe estoque pelo valor exato da baixa. • SHIPPED/DELIVERED não cancelam: use devolução. • `p_refund_confirmed` é DECLARAÇÃO de que o lojista já devolveu o dinheiro; o sistema não transfere nada.
Erros	`ORDER_FORBIDDEN` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `ORDER_NOTE_REQUIRED` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `STALE_ORDER_REVISION` `ORDER_CANNOT_BE_CANCELLED` `UNEXPECTED_REFUND_FOR_UNPAID_ORDER` `MANUAL_REFUND_CONFIRMATION_REQUIRED` `INVALID_MANUAL_PAYMENT_METHOD` `ORDER_RECEIPT_MISSING` `ORDER_SALE_ALLOCATION_MISSING` `STOCK_QUANTITY_LIMIT`
Efeitos	status CANCELLED, cancelled_at; reservas RELEASED ou REFUND + ENTRY RESTOCK; history CANCELLED.

`return_order` client

módulo 006

```
public.return_order(p_order_id uuid, p_request_id uuid, p_expected_revision bigint, p_items jsonb, p_goods_received boolean, p_refund_confirmed boolean, p_refund_method text, p_note text, p_refund_reference text DEFAULT NULL::text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `orders.return` (+ `inventory.entry` para RESTOCKED).
Entrada (TS)	`ReturnOrderArgs`
Saída (TS)	`OrderOperationResult`
Regras	• Somente SHIPPED ou DELIVERED. Devolução INTEGRAL: `p_items` deve conter exatamente todos os order_item_id, cada um com RESTOCKED ou DAMAGED. • `p_goods_received` e `p_refund_confirmed` devem ser true (declarações). Reembolso = total integral (produtos + frete). • RESTOCKED: volta ao estoque pelo custo alocado original. DAMAGED: não volta; o custo vira perda (`loss_value`) no ERP.
Erros	`ORDER_FORBIDDEN` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `ORDER_NOTE_REQUIRED` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `STALE_ORDER_REVISION` `RETURN_ITEMS_REQUIRED` `INVALID_RETURN_ITEMS` `INVALID_RETURN_ITEM` `UNKNOWN_RETURN_ITEM_FIELD` `INVALID_RETURN_DISPOSITION` `FULL_ORDER_RETURN_REQUIRED` `ORDER_CANNOT_BE_RETURNED` `PHYSICAL_RETURN_CONFIRMATION_REQUIRED` `MANUAL_REFUND_CONFIRMATION_REQUIRED` `INVALID_MANUAL_PAYMENT_METHOD` `ORDER_RECEIPT_MISSING` `ORDER_SALE_ALLOCATION_MISSING`
Efeitos	REFUND; ENTRY RESTOCK (RESTOCKED); order_return_items; status RETURNED; history RETURNED.

`create_expense_category` client

módulo 007

```
public.create_expense_category(p_store_id uuid, p_request_id uuid, p_name text, p_classification text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `expenses.categories.manage`.
Entrada (TS)	`CreateExpenseCategoryArgs`
Saída (TS)	`CreateExpenseCategoryResult`
Regras	• Nome 2..100, único por loja ignorando caixa/espaços. Classificação imutável: OPERATING (despesa operacional) ou INVENTORY_PURCHASE (compra de estoque, afeta caixa, não resultado). • Idempotente por (store_id, request_id) com cache de resultado (`replayed`).
Erros	`CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `INVALID_EXPENSE_CATEGORY` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `23505 expense_categories_store_name`
Efeitos	INSERT expense_categories.

`update_expense_category` client

módulo 007

`public.update_expense_category(p_category_id uuid, p_request_id uuid, p_name text, p_active boolean) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `expenses.categories.manage`.
Entrada (TS)	`UpdateExpenseCategoryArgs`
Saída (TS)	`UpdateExpenseCategoryResult`
Regras	• Renomeia e ativa/desativa (ambos obrigatórios). Não altera classificação. Lançamentos antigos mantêm `category_name_snapshot`. • Não existe exclusão.
Erros	`EXPENSE_CATEGORY_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `INVALID_EXPENSE_CATEGORY` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `23505 expense_categories_store_name`
Efeitos	UPDATE expense_categories.

`record_expense` client

módulo 007

`public.record_expense(p_store_id uuid, p_request_id uuid, p_category_id uuid, p_amount numeric, p_method text, p_description text, p_occurred_at timestamp with time zone, p_reference text DEFAULT NULL::text, p_inventory_movement_id uuid DEFAULT NULL::uuid) RETURNS jsonb`

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `expenses.record`.
Entrada (TS)	`RecordExpenseArgs`
Saída (TS)	`RecordExpenseResult`
Regras	• Somente despesa JÁ PAGA: `p_occurred_at` entre 2000-01-01 e agora + 5 min (sem agendamento/contas a pagar). • Categoria ativa da mesma loja. Valor > 0 com 2 casas. • `p_inventory_movement_id` (opcional) vincula a compra a um movimento ENTRY manual (não de pedido) da loja; só para categoria INVENTORY_PURCHASE; cada movimento só pode ser vinculado uma vez.
Erros	`CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `INVALID_EXPENSE_DESCRIPTION` `INVALID_EXPENSE_TIMESTAMP` `INVALID_MANUAL_PAYMENT_METHOD` `INVALID_EXPENSE_AMOUNT` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `EXPENSE_CATEGORY_UNAVAILABLE` `INVALID_PURCHASE_STOCK_REFERENCE` `23505 expense_entries_inventory_movement_id_key`
Efeitos	INSERT expense_entries (kind EXPENSE, imutável).

`reverse_expense` client

módulo 007

```
public.reverse_expense(p_expense_id uuid, p_request_id uuid, p_kind text, p_description text, p_occurred_at timestamp with time zone DEFAULT NULL::timestamp with time zone, p_method text DEFAULT NULL::text, p_reference text DEFAULT NULL::text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `expenses.reverse`.
Entrada (TS)	`ReverseExpenseArgs`
Saída (TS)	`ReverseExpenseResult`
Regras	• CORRECTION: anula um lançamento errado NA DATA ORIGINAL; não envie occurred_at/method/reference. • REFUND: dinheiro efetivamente devolvido ao lojista, em NOVA data (≥ data original), com método. • Uma reversão por lançamento; somente lançamentos kind=EXPENSE. Sempre pelo valor integral.
Erros	`EXPENSE_UNAVAILABLE` `CATALOG_FORBIDDEN` `STORE_UNAVAILABLE` `INVALID_EXPENSE_DESCRIPTION` `INVALID_EXPENSE_REVERSAL_KIND` `CORRECTION_USES_ORIGINAL_DATE` `INVALID_EXPENSE_TIMESTAMP` `INVALID_MANUAL_PAYMENT_METHOD` `REFUND_BEFORE_EXPENSE` `REQUEST_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `EXPENSE_ALREADY_REVERSED_OR_NOT_ORIGINAL`
Efeitos	INSERT expense_entries (CORRECTION/REFUND, reversal_of_id).

`get_erp_summary` client

módulo 007

```
public.get_erp_summary(p_store_id uuid, p_from date, p_to date, p_timezone text DEFAULT 'America/Sao_Paulo'::text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `reports.read`.
Entrada (TS)	`ErpPeriodArgs`
Saída (TS)	`ErpSummary`
Regras	• Período [from, to] inclusivo em dias locais do timezone; to − from ≤ 365 (até 366 dias). • Receita/CMV reconhecidos na data do RECEBIMENTO manual; estornos na data do REFUND (não pelo status atual do pedido). • estimated_operating_result = receita líquida − CMV − perdas de devolução − ajustes de estoque − despesas operacionais. Compras de estoque afetam só o caixa. • `inventory_snapshot_current` é o estoque ATUAL, não o do fim do período. `is_estimate: true` sempre.
Erros	`REPORT_FORBIDDEN` `INVALID_REPORT_PERIOD_OR_TIMEZONE`
Efeitos	Somente leitura.

`get_erp_daily` client

módulo 007

```
public.get_erp_daily(p_store_id uuid, p_from date, p_to date, p_timezone text DEFAULT 'America/Sao_Paulo'::text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `reports.read`.
Entrada (TS)	`ErpPeriodArgs`
Saída (TS)	`ErpDaily`
Regras	• Um item por dia do período (dias sem fatos vêm zerados), ordenado por data.
Erros	`REPORT_FORBIDDEN` `INVALID_REPORT_PERIOD_OR_TIMEZONE`
Efeitos	Somente leitura.

`get_erp_product_performance` client

módulo 007

```
public.get_erp_product_performance(p_store_id uuid, p_from date, p_to date, p_timezone text DEFAULT 'America/Sao_Paulo'::text, p_limit integer DEFAULT 20) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `reports.read`.
Entrada (TS)	`ErpProductPerformanceArgs`
Saída (TS)	`ErpProductPerformance`
Regras	• Top N (1..100, default 20) por receita líquida de produtos. Frete e despesas gerais não são rateados (campo `note`).
Erros	`REPORT_FORBIDDEN` `INVALID_REPORT_PERIOD_OR_TIMEZONE` `INVALID_REPORT_LIMIT`
Efeitos	Somente leitura.

`get_erp_expense_breakdown` client

módulo 007

```
public.get_erp_expense_breakdown(p_store_id uuid, p_from date, p_to date, p_timezone text DEFAULT 'America/Sao_Paulo'::text) RETURNS jsonb
```

ACL (manifesto)	`authenticated=True · anon=False · service_role=False`
Permissão	Loja: `reports.read`.
Entrada (TS)	`ErpPeriodArgs`
Saída (TS)	`ErpExpenseBreakdown`
Regras	• Valor líquido por categoria (EXPENSE − CORRECTION − REFUND) em occurred_at; agrupado também por name_snapshot (renomeações geram linhas distintas). • Não retorna `timezone` no JSON, apesar de aceitá-lo.
Erros	`REPORT_FORBIDDEN` `INVALID_REPORT_PERIOD_OR_TIMEZONE`
Efeitos	Somente leitura.

20.2 RPCs service-only (5) — NUNCA chamar do navegador

`consume_public_rate_limit` service-only

módulo 005

```
public.consume_public_rate_limit(p_scope text, p_subject_hash text, p_limit integer, p_window_seconds integer DEFAULT 60) RETURNS jsonb
```

ACL (manifesto)	`authenticated=False · anon=False · service_role=True`
Permissão	SOMENTE service_role (Edge).
Entrada (TS)	`(interno da Edge)`
Saída (TS)	`{ allowed: boolean; retry_after: number }`
Regras	• Escopos: catalog, product, checkout, order-status, checkout-phone, client. subject_hash = HMAC-SHA256 hex. • Janela fixa; expurga até 100 janelas antigas por chamada.
Erros	`INVALID_RATE_LIMIT`
Efeitos	UPSERT private.public_rate_buckets.

`get_public_catalog` service-only

módulo 005

`public.get_public_catalog(p_store_slug text, p_filters jsonb DEFAULT '{}'::jsonb) RETURNS jsonb`

ACL (manifesto)	`authenticated=False · anon=False · service_role=True`
Permissão	SOMENTE service_role (Edge, ação catalog).
Entrada (TS)	`CatalogFilters (via Edge)`
Saída (TS)	`CatalogData | null`
Regras	• null (→ HTTP 404) se a loja não existe, não está publicada, está arquivada, dono inativo ou OWNER sem storefront.read. • Busca literal (strpos) em nome+descrição+código; category_id inclui subcategorias. • Somente produtos ACTIVE, não arquivados, com categoria (e mãe) ativa.
Erros	`INVALID_CATALOG_FILTERS` `INVALID_CATALOG_INPUT` `UNKNOWN_CATALOG_FIELD`
Efeitos	Somente leitura.

`get_public_product` service-only

módulo 005

`public.get_public_product(p_store_slug text, p_product_slug text) RETURNS jsonb`

ACL (manifesto)	`authenticated=False · anon=False · service_role=True`
Permissão	SOMENTE service_role (Edge, ação product).
Entrada (TS)	`product_slug (via Edge)`
Saída (TS)	`PublicProduct | null`
Regras	• Mesmo filtro de visibilidade do catálogo; null → 404.
Erros	—
Efeitos	Somente leitura.

`create_public_order` service-only

módulo 005

`public.create_public_order(p_store_slug text, p_request_id uuid, p_input jsonb, p_token_hash text) RETURNS jsonb`

ACL (manifesto)	`authenticated=False · anon=False · service_role=True`
Permissão	SOMENTE service_role (Edge, ação checkout). Loja publicada + orders.create.
Entrada (TS)	`CheckoutInput (via Edge)`
Saída (TS)	`CheckoutData (parte do banco)`
Regras	• Preço vem do banco (nunca do navegador). Valida disponível = saldo − reservas ativas para cada item. • Cliente: INSERT por (store_id, phone) ON CONFLICT DO NOTHING — cliente existente NÃO é sobrescrito; o snapshot do pedido guarda nome/telefone enviados. • Cria pedido PENDING com código `PED-000001`, total = subtotal (frete 0, não confirmado), reservas HELD com expiração +24h. • Idempotente por (store_id, request_id) e payload {input normalizado, token_hash}; replay → `replayed:true`, reserva NÃO renovada.
Erros	`INVALID_REQUEST_OR_TOKEN` `STORE_UNAVAILABLE` `INVALID_CUSTOMER_OR_ADDRESS` `INVALID_CUSTOMER` `INVALID_DELIVERY_ADDRESS` `INVALID_CUSTOMER_NOTE` `INVALID_ORDER_ITEMS` `ORDER_ITEMS_LIMIT_20` `INVALID_ORDER_ITEM` `INVALID_ORDER_ITEM_FIELDS` `INVALID_ORDER_ITEM_QUANTITY_OR_DUPLICATE` `IDEMPOTENCY_KEY_REUSED` `PRODUCT_UNAVAILABLE` `INSUFFICIENT_AVAILABLE_STOCK` `ORDER_TOTAL_LIMIT`
Efeitos	INSERT customers (se novo), orders, order_items, inventory_reservations, order_access(token_hash), history CREATED.

`get_public_order_status` service-only

módulo 005→006 (versão FINAL do 006)

`public.get_public_order_status(p_store_slug text, p_order_id uuid, p_token_hash text) RETURNS jsonb`

ACL (manifesto)	`authenticated=False · anon=False · service_role=True`
Permissão	SOMENTE service_role (Edge, ação order-status).
Entrada (TS)	`order_id + token (via Edge)`
Saída (TS)	`OrderStatusData | null`
Regras	• Versão final (módulo 06) inclui revision, frete, timeline, pagamento, rastreio e datas de cada etapa. • Exige token_hash correto, loja não arquivada e dono ativo. NÃO exige loja publicada (acompanhamento continua após despublicar).
Erros	—
Efeitos	Somente leitura.

Variantes de resposta (noop / replay)

Família	Primeira execução	Repetição com mesma chave+payload	Noop
create_store / create_category / create_product	`created: true`	`created: false` (mesmo id)	—
attach_product_image	`created: true`	`created: false` (mesmo path)	—
archive_store / archive_category / archive_product	`already_archived: false`	`already_archived: true`	—
detach_product_image	`detached: true, path`	`detached: false, path: null`	idem
Estoque (4 RPCs)	`replayed: false`	resultado original + `replayed: true`	contagem sem diferença: `changed: false`
Pedidos (6 RPCs)	`replayed: false`	resultado original + `replayed: true` (sem checar revisão de novo)	—
Despesas (4 RPCs)	`replayed: false`	resultado original + `replayed: true`	—
update_category / update_product / replace_attributes / reorder / set_store_publication	sem chave de idempotência	repetir reaplica o mesmo estado (idempotente por natureza)	—
Checkout (Edge)	`replayed: false`	`replayed: true`, mesma reserva (não renovada) e mesmo token (derivado de request_id+client_secret)	—

21. Tabelas: leitura, filtros e operações permitidas

Operações permitidas ao frontend por tabela/view, conforme policies (33) e grants de coluna (341) do manifesto. Todas as tabelas têm RLS. `anon` não tem acesso a nenhuma. Colunas completas no Apêndice B.

Tabela/view	Cols	SELECT (policy)	Escrita pelo cliente	Telas	select/filter/paginação típicos
`categories`	13	Não arquivada + catalog.read	Somente RPC	Categorias, Produtos	`.select('id, parent_id, code, name, slug, is_active, sort_order').eq('store_id', sid).order('sort_order')`
`customers`	6	customers.read	Nenhuma (criado pela Edge)	Clientes	`.select('id, full_name, phone, created_at', {count:'exact'}).eq('store_id', sid)`
`expense_categories`	9	expenses.read	Somente RPC (sem DELETE)	Despesas	`.select('*').eq('store_id', sid).order('name')`
`expense_entries`	16	expenses.read	Somente RPC (imutável)	Despesas	`.select('*', {count:'exact'}).eq('store_id', sid).gte('occurred_at', a).lt('occurred_at', b)`
`inventory_balances`	9	inventory.read	Somente RPC (projeção do ledger)	— (preferir a view)	`.select('product_id, quantity, stock_value, average_cost, revision, last_movement_id')`
`inventory_movements`	23	inventory.read	Somente RPC (ledger imutável)	Movimentações, Despesas	`.select('*', {count:'exact'}).eq('store_id', sid).order('created_at', {ascending:false}).range(a,b)`
`inventory_reservations`	8	inventory.read OU orders.read	Nenhuma	Pedido, Estoque	`.select('product_id, quantity, state, expires_at').eq('order_id', oid)`
`order_items`	13	orders.read	Nenhuma	Pedido	`.select('*').eq('order_id', oid)`
`order_payments`	11	orders.read	Nenhuma (imutável)	Pedido	`.select('*').eq('order_id', oid)`
`order_return_items`	11	orders.read	Nenhuma (imutável)	Pedido	`.select('*').eq('order_id', oid)`
`order_status_history`	11	orders.read	Nenhuma (imutável)	Pedido	`.select('*').eq('order_id', oid).order('revision')`
`order_stock_operations`	9	orders.read	Nenhuma (imutável)	Pedido, Movimentações	`.select('order_item_id, kind, movement_id, quantity, allocated_value').eq('order_id', oid)`
`orders`	26	orders.read	Somente RPC (módulo 06)	Pedidos	`.select('id, code, status, revision, total, created_at, customer_snapshot', {count:'exact'}).eq('store_id', sid)`
`permissions`	8	SUDO	SUDO: INSERT/UPDATE/DELETE	SUDO	`.select('*').order('module').order('code')`
`plans`	9	Ativos com plans.read; SUDO+plans.manage: todos	SUDO: UPDATE name, description, is_active	Onboarding, SUDO	`.select('id, code, name, description, price, currency').eq('code','FREE').eq('is_active',true)`

Tabela/view	Cols	SELECT (policy)	Escrita pelo cliente	Telas	select/filter/paginação típicos
`product_attributes`	9	Produto vivo + catalog.read	Somente RPC	Produto	`.select('name, value, sort_order').eq('product_id', pid).order('sort_order')`
`product_images`	8	Produto vivo + catalog.read	Somente RPC (+ Storage)	Produto	`.select('id, path, alt_text, sort_order').eq('product_id', pid).order('sort_order')`
`product_stock_balances`	14	VIEW security_invoker: produto vivo + inventory.read (e catalog.read via products)	Nenhuma	Estoque, Produtos, Dashboard	`.select('product_id, code, name, quantity, reserved_quantity, available_quantity, revision').eq('store_id', sid)`
`products`	16	Não arquivado + catalog.read	Somente RPC	Produtos	`.select('…, category:categories(id,name), images:product_images(path,sort_order)', {count:'exact'}).eq('store_id', sid).range(a,b)`
`profiles`	7	Própria linha (SUDO: todas)	UPDATE full_name, phone, avatar_path (só a própria; requer profile.update)	Perfil, SUDO usuários	`.select('id, full_name, phone, avatar_path, is_active').eq('id', uid).single()`
`role_permissions`	5	SUDO	SUDO: INSERT/UPDATE/DELETE	SUDO	`.select('role_id, permission_id, is_enabled')`
`roles`	8	SUDO (can_manage_security)	SUDO: INSERT/UPDATE/DELETE (proteções is_system)	SUDO	`.select('*').order('code')`
`security_audit_logs`	7	SUDO	Nenhuma (triggers)	SUDO	`.select('*').order('created_at', {ascending:false}).range(0,99)`
`stores`	21	Não arquivada + store.read	UPDATE das 13 colunas com store.update (logo/banner: + branding; is_published: + publish)	Acesso, Configurações	`.select('*').eq('owner_id', uid).is('archived_at', null).maybeSingle()`
`user_platform_roles`	3	Própria linha (SUDO: todas)	Nenhuma (somente SQL Editor)	SUDO usuários	`.select('user_id, role_code').in('user_id', ids)`

21.1 Joins (embedding PostgREST) seguros

Embedding	FK usada	Observação
`products → categories`	products_category_store (category_id, store_id)	`category:categories(id,name,parent_id)`; categoria arquivada vem null (policy)
`products → product_images / product_attributes`	FK composta (product_id, store_id)	`images:product_images(path,sort_order)`
`orders → customers`	orders_customer_store	`customer:customers(id,full_name,phone)` requer customers.read; senão use customer_snapshot
`orders → order_items / order_payments / order_status_history`	FKs (order_id, store_id)	Preferir consultas separadas no detalhe (paralelas) para controlar ordenação
`product_stock_balances`	VIEW (sem FKs declaradas)	Não embutir via products; consultar separadamente com `.in('product_id', ids)`
`expense_entries → expense_categories`	expense_entries_category_store	Normalmente desnecessário: usar category_name_snapshot

21.2 Paginação e contagem

- Listas: `.range(from, to)` + `{ count: 'exact' }` com ordenação estável (coluna + `id`). Tamanho padrão 20–50.

- Contadores de dashboard: `{ count: 'exact', head: true }` (sem payload).
- Datas por dia local: converter início do dia no fuso da loja para ISO (`gte`) e início do dia seguinte (`lt`).
- Busca textual: `ilike` com escape de `%`, `_`, `,`, `(`, `)` antes de compor `.or()`.

22. Catálogo de erros e mensagens

Códigos extraídos dos `RAISE EXCEPTION` das migrations/funções finais (169 códigos). O PostgREST devolve o código em `error.message` e o SQLSTATE em `error.code` (42501 acesso; 22023/22004 validação; 23505 unicidade; 23514 CHECK; 23503 referência; P0002 não encontrado). Violação de CHECK/UNIQUE sem tradução chega como mensagem do PostgreSQL com o nome da constraint (ex.: `products_store_slug`, `stores_email_check`) — o wrapper da seção 2.4 a converte em `SQLSTATE:constraint`.

Grupo	Códigos	Mensagem padrão
Acesso	`FORBIDDEN` `CATALOG_FORBIDDEN` `INVENTORY_FORBIDDEN` `ORDER_FORBIDDEN` `REPORT_FORBIDDEN` `STORE_CREATE_FORBIDDEN` `STORE_PUBLISH_FORBIDDEN` `STORE_BRANDING_FORBIDDEN` `STORE_UNAVAILABLE` `ACCOUNT_UNAVAILABLE` `OWNER_ACCESS_UNAVAILABLE`	Você não tem permissão para esta ação ou o recurso não está disponível.
Idempotência/concorrência	`REQUEST_REQUIRED` `REQUEST_AND_PLAN_REQUIRED` `IDEMPOTENCY_KEY_REUSED` `STALE_INVENTORY_REVISION` `STALE_ORDER_REVISION`	Os dados mudaram ou a operação foi reenviada com alterações. Recarregue e tente novamente.
Loja	`INVALID_STORE_INPUT` `UNKNOWN_STORE_FIELD` `INVALID_STORE_FIELD_TYPE` `INVALID_STORE_NAME_OR_TITLE` `INVALID_STORE_SLUG` `SLUG_ALREADY_EXISTS` `INVALID_WHATSAPP_NUMBER` `INVALID_SOCIAL_LINKS` `ACTIVE_STORE_ALREADY_EXISTS` `ONBOARDING_ALREADY_ARCHIVED` `PLAN_UNAVAILABLE` `PUBLICATION_REQUIRED` `STORE_HAS_STOCK` `STORE_HAS_OPEN_ORDERS` `INVALID_LOGO_PATH` `LOGO_OBJECT_NOT_FOUND` `INVALID_BANNER_PATH` `BANNER_OBJECT_NOT_FOUND`	Mensagem específica no campo/ação (ver seções 8–9).
Catálogo	`INVALID_CATALOG_INPUT` `UNKNOWN_CATALOG_FIELD` `INVALID_CATEGORY_FIELD_TYPE` `INVALID_PRODUCT_FIELD_TYPE` `INVALID_MONEY_VALUE` `CATEGORY_UNAVAILABLE` `CATEGORY_ARCHIVED` `CATEGORY_SLUG_EXISTS` `CATEGORY_PARENT_UNAVAILABLE` `CATEGORY_MAX_TWO_LEVELS` `CATEGORY_WITH_CHILDREN_CANNOT_BECOME_CHILD` `CATEGORY_HAS_LIVE_CHILDREN` `CATEGORY_HAS_LIVE_PRODUCTS` `PRODUCT_UNAVAILABLE` `PRODUCT_ARCHIVED` `PRODUCT_SLUG_EXISTS` `PRODUCT_CATEGORY_UNAVAILABLE` `PRODUCT_HAS_STOCK` `PRODUCT_HAS_OPEN_ORDERS` `ATTRIBUTES_ARRAY_REQUIRED` `ATTRIBUTES_LIMIT_30` `INVALID_ATTRIBUTE` `UNKNOWN_ATTRIBUTE_FIELD` `INVALID_ATTRIBUTE_NAME_OR_VALUE` `DUPLICATE_ATTRIBUTE_NAME` `PRODUCT_IMAGES_LIMIT_4` `INVALID_PRODUCT_IMAGE_PATH` `PRODUCT_IMAGE_OBJECT_NOT_FOUND` `INVALID_IMAGE_ORDER`	Ver seções 10–11.
Estoque	`MANUAL_TYPE_MUST_BE_ENTRY_OR_EXIT` `QUANTITY_MUST_BE_POSITIVE_INTEGER` `INVALID_MOVEMENT_REASON_OR_REFERENCE` `INVALID_UNIT_COST` `OUTBOUND_COST_IS_AUTOMATIC` `INSUFFICIENT_STOCK` `STOCK_QUANTITY_LIMIT` `STOCK_RESERVED_BY_PENDING_ORDERS` `INVALID_COUNT_OR_EXPECTED_REVISION` `UNIT_COST_NOT_ALLOWED_FOR_OUTBOUND_OR_NO_CHANGE` `BATCH_ITEMS_ARRAY_REQUIRED` `BATCH_LIMIT_50` `INVALID_BATCH_ITEM` `UNKNOWN_BATCH_FIELD` `INVALID_BATCH_FIELD_TYPE` `BATCH_DUPLICATE_PRODUCT` `BATCH_PRODUCT_UNAVAILABLE` `INVALID_REVERSAL_SOURCE` `MOVEMENT_ALREADY_REVERSED` `ONLY_LAST_MOVEMENT_CAN_BE_REVERSED` `ORDER_STOCK_REVERSAL_REQUIRES_ORDER_FLOW` `INVALID_MOVEMENT_QUANTITY_OR_REQUEST` `INVALID_MOVEMENT_SIGN` `INVALID_LEDGER_CHAIN`	Ver seção 12.

Grupo	Códigos	Mensagem padrão
Pedidos	`ORDER_NOTE_REQUIRED` `INVALID_SHIPPING_AMOUNT` `ORDER_TOTAL_LIMIT` `ORDER_NOT_PENDING` `SHIPPING_TERMS_NOT_CONFIRMED` `PAYMENT_MUST_MATCH_ORDER_TOTAL` `INVALID_MANUAL_PAYMENT_METHOD` `INSUFFICIENT_AVAILABLE_STOCK_FOR_PAYMENT` `INVALID_TRACKING_CODE` `ORDER_NOT_PAID` `ORDER_NOT_SHIPPED` `ORDER_CANNOT_BE_CANCELLED` `UNEXPECTED_REFUND_FOR_UNPAID_ORDER` `MANUAL_REFUND_CONFIRMATION_REQUIRED` `ORDER_RECEIPT_MISSING` `ORDER_SALE_ALLOCATION_MISSING` `RETURN_ITEMS_REQUIRED` `INVALID_RETURN_ITEMS` `INVALID_RETURN_ITEM` `UNKNOWN_RETURN_ITEM_FIELD` `INVALID_RETURN_DISPOSITION` `FULL_ORDER_RETURN_REQUIRED` `ORDER_CANNOT_BE_RETURNED` `PHYSICAL_RETURN_CONFIRMATION_REQUIRED` `INVALID_ORDER_TRANSITION` `ORDER_TERMINAL` `PAID_ORDER_TERMS_IMMUTABLE`	Ver seção 13.
Despesas/ERP	`INVALID_EXPENSE_CATEGORY` `EXPENSE_CATEGORY_UNAVAILABLE` `INVALID_EXPENSE_AMOUNT` `INVALID_EXPENSE_DESCRIPTION` `INVALID_EXPENSE_TIMESTAMP` `INVALID_PURCHASE_STOCK_REFERENCE` `EXPENSE_UNAVAILABLE` `INVALID_EXPENSE_REVERSAL_KIND` `CORRECTION_USES_ORIGINAL_DATE` `REFUND_BEFORE_EXPENSE` `EXPENSE_ALREADY_REVERSED_OR_NOT_ORIGINAL` `INVALID_REPORT_PERIOD_OR_TIMEZONE` `INVALID_REPORT_LIMIT`	Ver seções 15–16.
SUDO	`ACTIVE_REQUIRED` `CANNOT_DISABLE_SELF` `USER_NOT_FOUND` `SYSTEM_DEFINITION_RESERVED` `SYSTEM_DEFINITION_CANNOT_BE_DELETED` `SECURITY_CODE_AND_SYSTEM_FLAG_IMMUTABLE`	Ver seção 18.
Invariantes internas (não devem ocorrer pela UI)	`CATEGORY_IDENTITY_IMMUTABLE` `PRODUCT_IDENTITY_IMMUTABLE` `PRODUCT_IMAGE_IDENTITY_IMMUTABLE` `INVENTORY_LEDGER_IMMUTABLE` `INVENTORY_PROJECTION_DELETE_FORBIDDEN` `INVENTORY_INITIAL_BALANCE_MUST_BE_ZERO` `INVALID_PROJECTION_UPDATE` `PROJECTION_REQUIRES_LEDGER` `INVENTORY_PROJECTION_MISSING` `UNEXPECTED_REVERSAL_SOURCE` `INVALID_REVERSAL_DELTA` `ORDER_IDENTITY_AND_SNAPSHOTS_IMMUTABLE` `ORDER_REVISION_MUST_ADVANCE` `ORDER_RECORD_IMMUTABLE` `PAID_ORDER_REQUIRES_RECEIPT_AND_STOCK` `CANCEL_REQUIRES_RESERVATION_RELEASE` `PAID_CANCEL_REQUIRES_REFUND_AND_RESTOCK` `RETURN_REQUIRES_REFUND_AND_ITEMS` `EXPENSE_CATEGORY_IDENTITY_IMMUTABLE` `EXPENSE_CATEGORY_DELETE_FORBIDDEN` `EXPENSE_LEDGER_IMMUTABLE` `INVALID_CODE_KIND` `PREEXISTING_ORDER_STATE_REQUIRES_RECONCILIATION`	Erro inesperado: mostrar mensagem genérica e registrar o código.
Somente Edge (SQL de checkout/catálogo)	`INVALID_REQUEST_OR_TOKEN` `INVALID_CUSTOMER_OR_ADDRESS` `INVALID_CUSTOMER` `INVALID_DELIVERY_ADDRESS` `INVALID_CUSTOMER_NOTE` `INVALID_ORDER_ITEMS` `ORDER_ITEMS_LIMIT_20` `INVALID_ORDER_ITEM` `INVALID_ORDER_ITEM_FIELDS` `INVALID_ORDER_ITEM_QUANTITY_OR_DUPLICATE` `INSUFFICIENT_AVAILABLE_STOCK` `INVALID_CATALOG_FILTERS` `INVALID_RATE_LIMIT`	Convertidos pela Edge em 400 INVALID_REQUEST ou 409 (ver 19.1).

TS

```ts
// src/lib/errors.ts (trecho) — mensagens pt-BR por código
export const MESSAGES: Record<string, string> = {
  FORBIDDEN: 'Você não tem permissão para esta ação.',
  STORE_UNAVAILABLE: 'Loja indisponível ou sem permissão.',
  IDEMPOTENCY_KEY_REUSED: 'Esta operação já foi enviada com outros dados. Atualize a página.',
  STALE_ORDER_REVISION: 'O pedido foi alterado por outra ação. Recarregamos os dados.',
  STALE_INVENTORY_REVISION: 'O estoque deste produto mudou. Confira o saldo atual.',
  SLUG_ALREADY_EXISTS: 'Este endereço de loja já está em uso.',
  INVALID_WHATSAPP_NUMBER: 'Informe o WhatsApp no formato internacional, ex.: +5511999999999.',
```

```
  PRODUCT_IMAGES_LIMIT_4: 'Cada produto pode ter no máximo 4 imagens.',
  PRODUCT_HAS_STOCK: 'Zere o estoque antes de arquivar o produto.',
  STOCK_RESERVED_BY_PENDING_ORDERS: 'Há unidades reservadas por pedidos pendentes.',
  INSUFFICIENT_STOCK: 'Estoque insuficiente.',
  SHIPPING_TERMS_NOT_CONFIRMED: 'Confirme o frete antes de registrar o pagamento.',
  PAYMENT_MUST_MATCH_ORDER_TOTAL: 'O valor recebido deve ser igual ao total do pedido.',
  INSUFFICIENT_AVAILABLE_STOCK_FOR_PAYMENT: 'Sem estoque disponível para baixar este pedido (reserva expirada?).',
  MANUAL_REFUND_CONFIRMATION_REQUIRED: 'Confirme que o reembolso integral já foi feito.',
  FULL_ORDER_RETURN_REQUIRED: 'A devolução deve incluir todos os itens do pedido.',
  INVALID_EXPENSE_TIMESTAMP: 'A data deve ser passada (despesas futuras não são aceitas).',
  INVALID_REPORT_PERIOD_OR_TIMEZONE: 'Selecione um período de até 366 dias.',
  // …demais códigos da tabela acima
}
export const messageFor = (code: string) =>
  MESSAGES[code] ?? (code.startsWith('23505:') ? 'Já existe um registro com este valor.'
    : code.startsWith('23514:') ? 'Valor inválido.' : 'Não foi possível concluir. Tente novamente.')
```

23. Fluxos ponta a ponta

23.1 Auth + onboarding

```text
Visitante → /auth/signup ── signUp(email, senha, data.full_name, emailRedirectTo=/auth/callback?next=/app/onboarding/plan)
  ├─ sessão retornada (confirmação desligada) → /app/onboarding/plan
  └─ sem sessão → "verifique seu e-mail" → clique → /auth/callback?code=…&next=…
       → exchangeCodeForSession → safeRedirect(next) → /app/onboarding/plan
Trigger DB: auth.users INSERT → profiles(full_name) + user_platform_roles(USER)
/app/onboarding/plan → plans(FREE) → /app/onboarding/store
  → check_store_slug (debounce) → create_store(request_id estável) → access.reload()
  → /app/store/settings?welcome=1 (loja NÃO publicada)
```

23.2 SKU + estoque inicial + publicação

```text
create_category (opcional) → create_product {status: INACTIVE} → recebe code PRD-000001 (SKU)
→ upload product-images/{store}/{product}/{uuid}.webp → attach_product_image (até 4) → reorder
→ register_inventory_movement ENTRY {quantity, unit_cost, reason} → saldo/custo médio
→ update_product {status: ACTIVE} → set_store_publication(true) → vitrine /s/{slug}
```

23.3 Checkout público → pedido → ERP

```text
Vitrine (Edge catalog/product) → carrinho local
→ Edge checkout {request_id, client_secret, input sem preços}
   = create_public_order: valida disponível, cria customer (se novo), pedido PENDING, reservas HELD 24h
   ← {order_id, code, token, tracking_url#token, whatsapp_url, reservation_expires_at}
→ cliente abre WhatsApp, negocia frete/pagamento
Lojista /app/orders/:id
→ update_order_terms(frete, nota)              [PENDING, revision n → n+1, TERMS_UPDATED]
→ confirm_order_payment(valor = total, método) [reservas CONSUMED, EXIT por item (CMV), RECEIPT, PAID]
→ ship_order(rastreio)                         [SHIPPED]
→ deliver_order                                [DELIVERED]
   (alternativas) cancel_order PENDING → reservas RELEASED
                  cancel_order PAID → REFUND total + reposição exata
                  return_order SHIPPED/DELIVERED → REFUND total (inclui frete) + RESTOCKED/DAMAGED
Cliente /s/{slug}/pedido/{id}#token → Edge order-status (token no body)
ERP: RECEIPT reconhece receita (produtos + frete) e CMV na data do recebimento;
     REFUND reverte receita e CMV na data do estorno e registra perda dos DAMAGED.
```

23.4 Reserva expirada

- Após 24 h a reserva deixa de contar (expires_at); o pedido segue PENDING e visível ao lojista com badge “Reserva expirada”.
- O cliente ainda vê o pedido; `reservation_active=false` na página de acompanhamento (texto: “a disponibilidade será confirmada pela loja”).
- Pagamento ainda é possível se houver estoque disponível; caso contrário `INSUFFICIENT_AVAILABLE_STOCK_FOR_PAYMENT` → registrar entrada ou cancelar.

23.5 Despesa de compra + entrada de estoque

```text
register_inventory_movement ENTRY (qty, unit_cost, reference "NF 123") → movement_id
record_expense(categoria INVENTORY_PURCHASE, amount = valor pago, occurred_at = data do pagamento,
               p_inventory_movement_id = movement_id)
ERP: compra afeta só caixa (net_inventory_purchase_cash); o custo entra no resultado via CMV quando vendido.
```

23.6 SUDO

TEXT

```
SQL Editor: insert user_platform_roles(user, 'SUDO')  ← único caminho
Usuário faz login → get_my_access().can_manage_security = true → menu "Administração" → /sudo
/sudo/role-permissions: liga/desliga permissões (efeito imediato + security_audit_logs)
/sudo/users: sudo_set_user_active(user, false) → conta bloqueada; vitrine do usuário indisponível
```

24. Ordem de implementação e critérios de aceitação

Fase	Entregas	Critérios de aceitação
F0 — Fundação	lib/supabase, rpc, errors, ids, pending, money, media, safeRedirect; tipos (Apêndice A); router + guards; layouts; tema Vuetify	Build TS strict sem erros; guard redireciona corretamente os 5 cenários (anônimo, sem loja, com loja, SUDO, conta inativa)
F1 — Auth	login, signup, forgot, reset, callback; store auth com onAuthStateChange sem await	Cadastro → confirmação → callback → onboarding; reset de senha completo; redirect externo rejeitado (`?redirect=//evil.com` → /app)
F2 — Acesso/perfil/onboarding	access store, profile, plan step, store step	Uma loja por usuário; replay não duplica; slug reservado bloqueado
F3 — Loja	settings, branding (logo/banner), publicação, arquivamento	Troca de logo sem órfãos; arquivar bloqueado com estoque/pedidos
F4 — Catálogo	categorias (árvore), produtos (lista/form), atributos, imagens	4 imagens máx.; reorder muda capa; SKU somente leitura
F5 — Estoque	saldos, entrada/saída, contagem, lote, ledger, reversão	STALE tratado; lote atômico; reversão só do último
F6 — Vitrine	storefront.service, layout com tema, home/filtros, produto, carrinho, checkout, acompanhamento, not-found	Nenhuma chamada a RPC pública pelo navegador; retry de checkout com mesma chave → replayed; token sai da URL
F7 — Pedidos/clientes	lista, detalhe, 6 ações, clientes	Fluxo completo PENDING → DELIVERED e as 3 alternativas; revisão concorrente
F8 — Despesas/ERP	categorias de despesa, lançamentos, correção/reembolso, relatórios	Período 367 dias bloqueado; avisos exibidos; saída manual sinalizada
F9 — Dashboard + SUDO	dashboard condicionado por permissão; painel SUDO	Usuário comum não acessa /sudo; matriz altera permissões em tempo real
F10 — Homologação	checklist de deploy (seção 25), testes E2E contra Supabase de staging	Todos os critérios acima verdes em staging

24.1 Critérios de aceitação transversais

- Nenhum uso de service_role/secret no bundle (`grep` no build).
- Nenhuma escrita direta em tabelas além de stores (13 colunas), profiles (3 colunas), plans (SUDO) e roles/permissions/role_permissions (SUDO).
- Toda RPC com request_id usa chave estável por intenção e trata `replayed` / `created:false` como sucesso.
- Toda operação com revisão envia o valor lido e trata STALE recarregando.
- Todas as telas implementam loading/empty/error/retry (seção 2.7) e respeitam `get_store_access().permissions`.
- Valores monetários enviados com ≤ 2 casas; quantidades inteiras; custos unitários ≤ 6 casas.
- Nenhuma URL completa de Storage gravada no banco.
- PII (telefone/endereço/token) ausente de logs, analytics e query strings.

24.2 Testes sugeridos

Tipo	Casos mínimos
Unit (Vitest)	safeRedirect (≥10 vetores: //, /\, %2F%2F, javascript:, URL absoluta, /auth/callback, controle), newClientSecret (43 chars, regex), parseBrlInput, toBackendError (mensagem código, constraint, 42501), stablePending (mesmo fingerprint → mesma chave)
Componente	Formulários: limites de caracteres/regex espelhando CHECKs; diálogos de pedido mostram campos condicionais
E2E (Playwright, staging)	Fluxos 23.1–23.6; dupla submissão; duas abas (STALE); rede offline no checkout + retry; 429 com Retry-After

25. Ambiguidades, lacunas e checagens de deploy

25.1 Ambiguidades e decisões do guia

#	Ponto	Situação no backend	Decisão/recomendação
1	Rotas, nomes de arquivos, stores	Não existem	Todas são propostas deste guia
2	Avatar	Coluna `avatar_path` sem bucket/policy	Não implementar upload; iniciais
3	Seleção da loja do usuário	RLS de stores retorna também lojas de terceiros para SUDO+store.admin	Sempre filtrar `owner_id = auth.uid()` para “minha loja”
4	Mensagem de erro	Códigos em `error.message`; CHECK/UNIQUE sem tradução trazem o nome da constraint	Wrapper da seção 2.4
5	UPDATE direto negado	Retorna 0 linhas sem erro	Usar `.select()` e tratar vazio
6	update_category/update_product	Sem request_id; slug não é regenerado	Enviar apenas diff; validar slug no cliente
7	Alt text de imagem	Sem RPC de edição	detach + attach do mesmo path
8	Nome do cliente	`customers.full_name` = primeiro pedido; snapshot por pedido	Mostrar ambos quando diferirem
9	Mudança de slug da loja	Quebra links de acompanhamento antigos	Confirmação com aviso
10	Acompanhamento após despublicar	`get_public_order_status` não exige is_published	Página funciona; vitrine 404
11	Quantidade disponível na vitrine	Apenas booleano `in_stock`	Validação final no checkout (409)
12	created_by / confirmed_by / actor_id	UUID sem nome acessível a não-SUDO	“Você” ou UUID abreviado
13	Categoria pública dedicada	Não existe	Filtro `category_id` no catálogo
14	Reserva no replay	Não renovada	Exibir `reservation_expires_at` retornado
15	Período ERP	`p_to − p_from ≤ 365`	UI limita a 366 dias inclusivos
16	get_erp_expense_breakdown	Não devolve `timezone`	Usar o timezone da requisição
17	Papéis customizados	Não atribuíveis (user_platform_roles só USER/SUDO)	Painel trata como catálogo
18	Exclusões	Não há DELETE de negócio	Usar “Arquivar”/“Desativar”
19	IP do visitante	Rate limit por IP só se `TRUSTED_CLIENT_IP_HEADER` for confiável	Manter vazio até verificar o gateway
20	Senha mínima/confirmação de e-mail	Configuração do projeto Auth	UI reage à resposta de signUp

25.2 Checagens necessárias antes do deploy (nada foi implantado remotamente)

Área	Checagem	Como verificar
Migrations	001→007 aplicadas UMA vez, em ordem, em projeto novo (sem o schema antigo); manifesto regenerado bate com o local	Comparar contagens: 25 tabelas/views, 40 funções public, 33 policies, 3 buckets
Schema private	`private` NÃO exposto na API (Settings → API → Exposed schemas = public)	GET `/rest/v1/rpc/…` de função private deve falhar
Edge secrets	`APP_URL`, `ALLOWED_ORIGINS`, `ORDER_TOKEN_SECRET` e `RATE_LIMIT_SECRET` (≥ 32 bytes cada, independentes), `TRUSTED_CLIENT_IP_HEADER` vazio	`supabase secrets list`; a função lança “Invalid server configuration” se faltar

Área	Checagem	Como verificar
Edge deploy	`supabase functions deploy storefront-api --no-verify-jwt` e `verify_jwt = false` no config.toml (mesclar fragmento)	POST sem Authorization retorna 200/404, não 401
Chave service	`SUPABASE_SERVICE_ROLE_KEY` ou `SUPABASE_SECRET_KEYS` provisionada pela plataforma; nunca no frontend	Build do frontend sem a string
CORS	`ALLOWED_ORIGINS` com TODAS as origens do app (produção, staging, localhost em dev) — origem exata, sem barra final	Preflight OPTIONS da origem do app → 204 com access-control-allow-origin
Headers de IP	Só definir `TRUSTED_CLIENT_IP_HEADER` (x-forwarded-for / x-real-ip / cf-connecting-ip) após comprovar que o proxy sobrescreve o header do visitante	Enviar header forjado e verificar que não altera o bucket
ORDER_TOKEN_SECRET	Não rotacionar sem plano: retries de checkout derivam o mesmo token; links existentes continuam (hash já gravado)	Documentar no runbook
Auth redirects	Site URL = APP_URL; Redirect URLs incluem `{APP_URL}/auth/callback**` (e dev)	Testar signup, reset e change-email
Auth templates	E-mails usam `{{ .ConfirmationURL }}`; idioma pt-BR	Receber e-mails de teste
SMTP	SMTP próprio configurado (o padrão do Supabase tem limite baixo)	Enviar 5 e-mails seguidos
Storage	Buckets públicos store-logos/store-banners/product-images com limites e MIME corretos	Upload > limite falha; GET público funciona
SUDO	Pelo menos um SUDO criado via SQL Editor	get_my_access().can_manage_security
Rate limit	Tetos globais (catalog 300/min…) adequados ao tráfego; considerar WAF/CAPTCHA (o rate limit interno não substitui)	Teste de carga leve
Env do frontend	`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_APP_URL` (= APP_URL da Edge), `VITE_STOREFRONT_API_URL`	Página de diagnóstico em dev
SPA fallback	Servidor estático reescreve todas as rotas para index.html (history mode), inclusive `/s/*` e `/auth/callback`	Acessar deep link direto
Referrer	Header/meta `Referrer-Policy: no-referrer` (ou strict-origin) nas páginas de pedido	DevTools → Network
Testes	Reexecutar testes SQL e Deno no ambiente de staging (até agora só locais)	CI

Apêndice A — Tipos TypeScript completos

Arquivo único proposto (pode ser dividido em `backend.ts`, `rpc.ts`, `storefront.ts`). As interfaces de linha são geradas a partir das colunas do manifesto (tipo, nullability, GENERATED).

TS

```ts
// ============================================================
// src/types/backend.ts (PROPOSTA) — tipos base do contrato 001..007
// ============================================================
export type Json = string | number | boolean | null | Json[] | { [key: string]: Json }
export type Uuid = string
export type IsoDateTime = string        // timestamptz serializado (ISO 8601)
export type IsoDate = string            // 'YYYY-MM-DD' (parâmetros date do ERP)

export type ProductStatus = 'ACTIVE' | 'INACTIVE'
export type InventoryMovementType = 'ENTRY' | 'EXIT' | 'ADJUSTMENT_IN' | 'ADJUSTMENT_OUT' | 'REVERSAL'
export type OrderStatus = 'PENDING' | 'PAID' | 'SHIPPED' | 'DELIVERED' | 'RETURNED' | 'CANCELLED'
export type OrderEventType =
  | 'CREATED' | 'TERMS_UPDATED' | 'PAYMENT_CONFIRMED' | 'SHIPPED' | 'DELIVERED' | 'CANCELLED' | 'RETURNED'
export type ManualMethod = 'PIX' | 'CASH' | 'BANK_TRANSFER' | 'CARD' | 'OTHER'
export type ReturnDisposition = 'RESTOCKED' | 'DAMAGED'
export type ExpenseClassification = 'OPERATING' | 'INVENTORY_PURCHASE'
export type ExpenseKind = 'EXPENSE' | 'CORRECTION' | 'REFUND'
export type PlatformRole = 'USER' | 'SUDO'          // user_platform_roles aceita somente estes
export type StoreContextRole = 'OWNER' | 'SUDO'     // retornado por get_store_access

export type PermissionCode =
  | 'auth.access' | 'profile.update' | 'security.manage'
  | 'plans.read' | 'plans.manage'
  | 'store.create' | 'store.read' | 'store.update' | 'store.publish' | 'store.archive'
  | 'store.branding.manage' | 'store.admin'
  | 'catalog.read' | 'catalog.categories.manage' | 'catalog.products.manage' | 'catalog.products.media'
  | 'inventory.read' | 'inventory.entry' | 'inventory.exit' | 'inventory.adjust' | 'inventory.reverse'
  | 'storefront.read' | 'orders.create' | 'orders.read' | 'customers.read'
  | 'orders.terms' | 'orders.pay' | 'orders.ship' | 'orders.deliver' | 'orders.cancel' | 'orders.return'
  | 'expenses.read' | 'expenses.categories.manage' | 'expenses.record' | 'expenses.reverse'
  | 'reports.read'

/** Chaves aceitas por private.valid_social_links: valor https://..., <= 2048 chars */
export interface SocialLinks {
  instagram?: string
  facebook?: string
  tiktok?: string
  youtube?: string
  website?: string
}
export interface ProductAttributeInput { name: string; value: string }   // name 1..80, value 1..500 (após trim)
export interface CustomerSnapshot { full_name: string; phone: string }
export interface DeliveryAddress {
  postal_code: string      // 8 dígitos, sem hífen
  street: string
  number: string
  complement?: string      // opcional, <= 160
  district: string
  city: string
  state: string            // UF em maiúsculas, 2 letras
}
export interface StoreSnapshot { name: string; slug: string; whatsapp_number: string }

// ---------- Linhas das 25 tabelas/views (geradas do manifesto) ----------
/** public.categories */
export interface CategoriesRow {
  id: string
  store_id: string
  parent_id: string | null
  code: string
  name: string
  slug: string
  is_active: boolean
  sort_order: number
  request_id: string
  request_payload: Json
```

```
  created_at: string
  updated_at: string
  archived_at: string | null
}

/** public.customers */
export interface CustomersRow {
  id: string
  store_id: string
  phone: string
  full_name: string
  created_at: string
  updated_at: string
}

/** public.expense_categories */
export interface ExpenseCategoriesRow {
  id: string
  store_id: string
  name: string
  classification: ExpenseClassification
  is_active: boolean
  request_id: string
  request_payload: Json
  created_at: string
  updated_at: string
}

/** public.expense_entries */
export interface ExpenseEntriesRow {
  id: string
  store_id: string
  category_id: string
  category_name_snapshot: string
  classification: ExpenseClassification
  kind: ExpenseKind
  amount: number  // numeric
  method: ManualMethod
  description: string
  external_reference: string | null
  occurred_at: string
  recorded_at: string
  recorded_by: string
  request_id: string
  inventory_movement_id: string | null
  reversal_of_id: string | null
}

/** public.inventory_balances */
export interface InventoryBalancesRow {
  product_id: string
  store_id: string
  quantity: number
  stock_value: number  // numeric
  average_cost: number | null  // numeric, GENERATED
  revision: number  // bigint
  last_movement_id: string | null
  created_at: string
  updated_at: string
}

/** public.inventory_movements */
export interface InventoryMovementsRow {
  id: string
  store_id: string
  product_id: string
  product_code: string
  product_name: string
  request_id: string
  batch_request_id: string | null
  movement_type: InventoryMovementType
  quantity_delta: number
  unit_cost: number  // numeric
  value_delta: number  // numeric
```

```
  quantity_before: number
  quantity_after: number
  value_before: number  // numeric
  value_after: number  // numeric
  average_cost_before: number  // numeric
  average_cost_after: number  // numeric
  revision: number  // bigint
  reason: string
  reference: string | null
  reversal_of_id: string | null
  created_by: string
  created_at: string
}

/** public.inventory_reservations */
export interface InventoryReservationsRow {
  id: string
  store_id: string
  product_id: string
  order_id: string
  quantity: number
  state: 'HELD' | 'RELEASED' | 'CONSUMED'
  expires_at: string
  created_at: string
}

/** public.order_items */
export interface OrderItemsRow {
  id: string
  store_id: string
  order_id: string
  product_id: string
  product_code: string
  product_name: string
  product_slug: string
  image_path: string | null
  attributes_snapshot: ProductAttributeInput[]
  quantity: number
  unit_price: number  // numeric
  unit_cost_snapshot: number  // numeric
  line_total: number  // numeric
}

/** public.order_payments */
export interface OrderPaymentsRow {
  id: string
  store_id: string
  order_id: string
  kind: 'RECEIPT' | 'REFUND'
  method: ManualMethod
  amount: number  // numeric
  external_reference: string | null
  note: string
  confirmed_by: string
  confirmed_at: string
  request_id: string
}

/** public.order_return_items */
export interface OrderReturnItemsRow {
  id: string
  store_id: string
  order_id: string
  order_item_id: string
  disposition: ReturnDisposition
  quantity: number
  allocated_cost: number  // numeric
  restocked_value: number  // numeric
  loss_value: number  // numeric
  note: string
  created_at: string
}

/** public.order_status_history */
```

```ts
export interface OrderStatusHistoryRow {
  id: string
  store_id: string
  order_id: string
  revision: number  // bigint
  event_type: OrderEventType
  status_before: OrderStatus | null
  status_after: OrderStatus
  note: string
  actor_id: string | null
  request_id: string
  created_at: string
}

/** public.order_stock_operations */
export interface OrderStockOperationsRow {
  id: string
  store_id: string
  order_id: string
  order_item_id: string
  kind: 'SALE' | 'RESTOCK'
  movement_id: string
  quantity: number
  allocated_value: number  // numeric
  created_at: string
}

/** public.orders */
export interface OrdersRow {
  id: string
  store_id: string
  customer_id: string
  code: string
  status: OrderStatus
  request_id: string
  request_payload: Json
  customer_snapshot: CustomerSnapshot
  delivery_address: DeliveryAddress
  store_snapshot: StoreSnapshot
  customer_note: string
  payment_method: 'WHATSAPP'
  subtotal: number  // numeric
  total: number  // numeric
  reservation_expires_at: string
  created_at: string
  updated_at: string
  revision: number  // bigint
  shipping_amount: number  // numeric
  shipping_confirmed: boolean
  tracking_code: string | null
  paid_at: string | null
  shipped_at: string | null
  delivered_at: string | null
  cancelled_at: string | null
  returned_at: string | null
}

/** public.permissions */
export interface PermissionsRow {
  id: string
  code: string
  module: string
  description: string
  is_system: boolean
  is_active: boolean
  created_at: string
  updated_at: string
}

/** public.plans */
export interface PlansRow {
  id: string
  code: 'FREE'
  name: string
```

```
  description: string
  price: number  // numeric
  currency: 'BRL'
  is_active: boolean
  created_at: string
  updated_at: string
}

/** public.product_attributes */
export interface ProductAttributesRow {
  id: string
  store_id: string
  product_id: string
  name: string
  name_key: string | null  // GENERATED
  value: string
  sort_order: number
  created_at: string
  updated_at: string
}

/** public.product_images */
export interface ProductImagesRow {
  id: string
  store_id: string
  product_id: string
  path: string
  alt_text: string
  sort_order: number
  created_at: string
  updated_at: string
}

/** public.product_stock_balances */
export interface ProductStockBalancesRow {
  product_id: string | null
  store_id: string | null
  code: string | null
  name: string | null
  status: ProductStatus | null
  category_id: string | null
  quantity: number | null
  stock_value: number | null  // numeric
  average_cost: number | null  // numeric
  revision: number | null  // bigint
  last_movement_id: string | null
  updated_at: string | null
  reserved_quantity: number | null
  available_quantity: number | null
}

/** public.products */
export interface ProductsRow {
  id: string
  store_id: string
  category_id: string | null
  code: string
  name: string
  slug: string
  description: string
  status: ProductStatus
  price: number  // numeric
  cost_price: number  // numeric
  is_featured: boolean
  request_id: string
  request_payload: Json
  created_at: string
  updated_at: string
  archived_at: string | null
}

/** public.profiles */
export interface ProfilesRow {
  id: string
```

```
  full_name: string
  phone: string | null
  avatar_path: string | null
  is_active: boolean
  created_at: string
  updated_at: string
}

/** public.role_permissions */
export interface RolePermissionsRow {
  role_id: string
  permission_id: string
  is_enabled: boolean
  created_at: string
  updated_at: string
}

/** public.roles */
export interface RolesRow {
  id: string
  code: string
  name: string
  description: string
  is_system: boolean
  is_active: boolean
  created_at: string
  updated_at: string
}

/** public.security_audit_logs */
export interface SecurityAuditLogsRow {
  id: number  // bigint
  actor_id: string | null
  table_name: string
  operation: 'INSERT' | 'UPDATE' | 'DELETE'
  row_before: Json | null
  row_after: Json | null
  created_at: string
}

/** public.stores */
export interface StoresRow {
  id: string
  owner_id: string
  plan_id: string
  name: string
  title: string
  subtitle: string
  slug: string
  cnpj: string | null
  email: string | null
  whatsapp_number: string
  logo_path: string | null
  banner_path: string | null
  primary_color: string
  secondary_color: string
  social_links: SocialLinks
  is_published: boolean
  onboarding_request_id: string
  onboarding_payload: Json
  created_at: string
  updated_at: string
  archived_at: string | null
}

/** public.user_platform_roles */
export interface UserPlatformRolesRow {
  user_id: string
  role_code: 'USER' | 'SUDO'
  created_at: string
}

// ============================================================
// src/types/rpc.ts (PROPOSTA) — entradas e saídas das 35 RPCs client
```

```
// ============================================================

// ---------- Identidade / acesso ----------
export interface MyAccess {
  user_id: Uuid | null
  account_active: boolean
  can_manage_security: boolean
  roles: string[]                 // códigos globais ativos: 'USER', 'SUDO' (OWNER NUNCA aparece aqui)
  permissions: PermissionCode[]   // permissões GLOBAIS (via user_platform_roles)
}
export type StoreAccess = null | {
  store_id: Uuid
  role: StoreContextRole          // 'OWNER' se owner_id = auth.uid(); senão 'SUDO' (store.admin)
  permissions: PermissionCode[]   // permissões efetivas NA loja
}
export interface SudoSetUserActiveArgs { p_user_id: Uuid; p_active: boolean }   // retorno: void (data = null)

// ---------- Loja ----------
export interface CheckStoreSlugArgs { p_slug: string }
export interface CheckStoreSlugResult { valid: boolean; available: boolean }

export interface CreateStoreInput {
  name: string                 // 2..120 (trim)
  slug: string                 // 3..63, ^[a-z0-9]+(-[a-z0-9]+)*$, sem palavras reservadas
  whatsapp_number: string      // E.164: ^\+[1-9][0-9]{7,14}$
  title?: string | null        // 2..160; default = name
  subtitle?: string | null     // <= 300; default ''
  cnpj?: string | null         // 1..32, texto livre (sem validação fiscal)
  email?: string | null        // normalizado para minúsculas
  primary_color?: string | null   // #RRGGBB; default #6750A4 (gravado em MAIÚSCULAS)
  secondary_color?: string | null // #RRGGBB; default #03A9F4
  social_links?: SocialLinks   // ÚNICA chave que pode ser objeto
}
export interface CreateStoreArgs { p_request_id: Uuid; p_plan_id: Uuid; p_input: CreateStoreInput }
export interface CreateStoreResult { store_id: Uuid; slug: string; plan_id: Uuid; created: boolean }  // created=false =>
replay

export interface SetStorePublicationArgs { p_store_id: Uuid; p_published: boolean }
export interface SetStorePublicationResult { store_id: Uuid; is_published: boolean }
export interface ArchiveStoreArgs { p_store_id: Uuid }
export interface ArchiveStoreResult { store_id: Uuid; archived: true; already_archived: boolean }

/** UPDATE direto em public.stores (único caso de escrita direta). Somente estas colunas têm GRANT UPDATE. */
export type StoreUpdatePatch = Partial<Pick<StoresRow,
  'name' | 'title' | 'subtitle' | 'slug' | 'cnpj' | 'email' | 'whatsapp_number' |
  'logo_path' | 'banner_path' | 'primary_color' | 'secondary_color' | 'social_links' | 'is_published'>>

/** UPDATE direto em public.profiles (somente a própria linha). */
export type ProfileUpdatePatch = Partial<Pick<ProfilesRow, 'full_name' | 'phone' | 'avatar_path'>>

// ---------- Catálogo ----------
export interface CategoryInput {
  name: string                 // 2..100 (trim)
  slug?: string | null         // 2..100; omitido/'' => gerado de name (sem acentos)
  parent_id?: Uuid | null      // no máximo 2 níveis
  is_active?: boolean          // default true
  sort_order?: number          // inteiro 0..1000000; default 0
}
export type CategoryPatch = Partial<CategoryInput>   // chave ausente = mantém; parent_id null = raiz
export interface CreateCategoryArgs { p_store_id: Uuid; p_request_id: Uuid; p_input: CategoryInput }
export interface CreateCategoryResult { category_id: Uuid; code: string; slug: string; created: boolean }
export interface UpdateCategoryArgs { p_category_id: Uuid; p_patch: CategoryPatch }
export interface UpdateCategoryResult { category_id: Uuid; updated_at: IsoDateTime }
export interface ArchiveCategoryArgs { p_category_id: Uuid }
export interface ArchiveCategoryResult { category_id: Uuid; archived: true; already_archived: boolean }

export interface ProductInput {
  name: string                 // 2..160 (trim)
  price: number                // obrigatório; 0..9999999999.99; no máximo 2 casas
  slug?: string | null         // 2..160; omitido/'' => gerado de name
  description?: string | null  // <= 10000; texto simples
  category_id?: Uuid | null
  status?: ProductStatus | null   // default 'INACTIVE'
  cost_price?: number          // default 0; custo de REFERÊNCIA (não é CMV)
```

```
  is_featured?: boolean        // default false
  attributes?: ProductAttributeInput[]  // <= 30; nomes únicos (case-insensitive)
}
export type ProductPatch = Partial<ProductInput>      // 'attributes' presente => SUBSTITUI todos
export interface CreateProductArgs { p_store_id: Uuid; p_request_id: Uuid; p_input: ProductInput }
export interface CreateProductResult { product_id: Uuid; code: string; slug: string; created: boolean }
export interface UpdateProductArgs { p_product_id: Uuid; p_patch: ProductPatch }
export interface UpdateProductResult { product_id: Uuid; updated_at: IsoDateTime; status: ProductStatus }
export interface ArchiveProductArgs { p_product_id: Uuid }
export interface ArchiveProductResult { product_id: Uuid; archived: true; already_archived: boolean }
export interface ReplaceProductAttributesArgs { p_product_id: Uuid; p_attributes: ProductAttributeInput[] }
export interface ReplaceProductAttributesResult { product_id: Uuid; count: number }

export interface AttachProductImageArgs { p_product_id: Uuid; p_path: string; p_alt_text?: string }
export interface AttachProductImageResult { image_id: Uuid; sort_order: 0 | 1 | 2 | 3; created: boolean }
export interface DetachProductImageArgs { p_product_id: Uuid; p_image_id: Uuid }
export interface DetachProductImageResult { product_id: Uuid; image_id: Uuid; detached: boolean; path: string | null }
export interface ReorderProductImagesArgs { p_product_id: Uuid; p_image_ids: Uuid[] }  // TODOS os ids vinculados
export interface ReorderProductImagesResult { product_id: Uuid; count: number }

// ---------- Estoque ----------
export interface InventoryMovementResult {
  movement_id: Uuid
  product_id: Uuid
  movement_type: InventoryMovementType
  quantity_delta: number
  quantity: number             // saldo após
  stock_value: number          // numeric(26,6)
  average_cost: number         // numeric(18,6)
  revision: number             // bigint
  replayed: boolean
}
export interface RegisterInventoryMovementArgs {
  p_product_id: Uuid; p_request_id: Uuid
  p_type: 'ENTRY' | 'EXIT'
  p_quantity: number           // inteiro 1..1000000000
  p_reason: string             // 3..500
  p_unit_cost?: number | null  // OBRIGATÓRIO em ENTRY (0..9999999999.999999, <=6 casas); PROIBIDO em EXIT
  p_reference?: string | null  // <= 120
}
export interface AdjustInventoryCountArgs {
  p_product_id: Uuid; p_request_id: Uuid
  p_counted_quantity: number   // inteiro 0..1000000000
  p_expected_revision: number  // product_stock_balances.revision lido na tela
  p_reason: string
  p_unit_cost?: number | null  // só quando a contagem AUMENTA o saldo
  p_reference?: string | null
}
export type AdjustInventoryCountResult =
  | { product_id: Uuid; quantity: number; stock_value: number; average_cost: number; revision: number; changed: false;
replayed: boolean }
  | (InventoryMovementResult & { changed: true })

export interface AdjustBatchItem {
  product_id: Uuid
  counted_quantity: number
  expected_revision: number
  unit_cost?: number | null
}
export interface AdjustInventoryBatchArgs {
  p_store_id: Uuid; p_request_id: Uuid
  p_items: AdjustBatchItem[]   // 1..50, sem produto repetido, <= 32 KiB
  p_reason: string
  p_reference?: string | null
}
export interface AdjustInventoryBatchResult {
  store_id: Uuid
  request_id: Uuid
  items: Array<
    | { product_id: Uuid; quantity: number; stock_value: number; average_cost: number; revision: number; changed: false }
    | (InventoryMovementResult & { changed: true })
  >
  replayed: boolean
}
```

```ts
export interface ReverseInventoryMovementArgs { p_movement_id: Uuid; p_request_id: Uuid; p_reason: string }
export type ReverseInventoryMovementResult = InventoryMovementResult   // movement_type = 'REVERSAL'

// ---------- Pedidos (lojista) ----------
export interface OrderOperationResult {
  order_id: Uuid
  status: OrderStatus
  revision: number
  subtotal: number
  shipping_amount: number
  shipping_confirmed: boolean
  total: number
  replayed: boolean
}
interface OrderOpBase { p_order_id: Uuid; p_request_id: Uuid; p_expected_revision: number }
export interface UpdateOrderTermsArgs extends OrderOpBase { p_shipping_amount: number; p_note: string }
export interface ConfirmOrderPaymentArgs extends OrderOpBase {
  p_amount: number             // DEVE ser igual a orders.total
  p_method: ManualMethod
  p_note: string               // 3..1000
  p_reference?: string | null  // <= 160
}
export interface ShipOrderArgs extends OrderOpBase { p_note: string; p_tracking_code?: string | null }  // <= 120
export interface DeliverOrderArgs extends OrderOpBase { p_note: string }
export interface CancelOrderArgs extends OrderOpBase {
  p_note: string
  p_refund_confirmed?: boolean        // PENDING: false/omitido | PAID: true
  p_refund_method?: ManualMethod | null   // PENDING: null | PAID: obrigatório
  p_refund_reference?: string | null      // PENDING: null
}
export interface ReturnOrderItem { order_item_id: Uuid; disposition: ReturnDisposition }
export interface ReturnOrderArgs extends OrderOpBase {
  p_items: ReturnOrderItem[]   // TODOS os itens do pedido (devolução integral), 1..20
  p_goods_received: boolean    // deve ser true
  p_refund_confirmed: boolean  // deve ser true (declaração)
  p_refund_method: ManualMethod
  p_note: string
  p_refund_reference?: string | null
}

// ---------- Despesas ----------
export interface CreateExpenseCategoryArgs {
  p_store_id: Uuid; p_request_id: Uuid; p_name: string; p_classification: ExpenseClassification
}
export interface CreateExpenseCategoryResult {
  category_id: Uuid; name: string; classification: ExpenseClassification; replayed: boolean
}
export interface UpdateExpenseCategoryArgs { p_category_id: Uuid; p_request_id: Uuid; p_name: string; p_active: boolean }
export interface UpdateExpenseCategoryResult { category_id: Uuid; name: string; is_active: boolean; replayed: boolean }
export interface RecordExpenseArgs {
  p_store_id: Uuid; p_request_id: Uuid; p_category_id: Uuid
  p_amount: number                     // > 0, <= 9999999999.99, 2 casas
  p_method: ManualMethod
  p_description: string                // 3..500
  p_occurred_at: IsoDateTime           // >= 2000-01-01 e <= agora + 5 min (somente despesa JÁ PAGA)
  p_reference?: string | null          // <= 160
  p_inventory_movement_id?: Uuid | null  // só categoria INVENTORY_PURCHASE + movimento ENTRY manual
}
export interface RecordExpenseResult {
  expense_id: Uuid; amount: number; classification: ExpenseClassification; occurred_at: IsoDateTime; replayed: boolean
}
export interface ReverseExpenseArgs {
  p_expense_id: Uuid; p_request_id: Uuid
  p_kind: 'CORRECTION' | 'REFUND'
  p_description: string
  p_occurred_at?: IsoDateTime | null   // CORRECTION: null | REFUND: obrigatório (>= data original)
  p_method?: ManualMethod | null       // CORRECTION: null | REFUND: obrigatório
  p_reference?: string | null          // CORRECTION: null
}
export interface ReverseExpenseResult {
  expense_id: Uuid; reversal_of_id: Uuid; kind: 'CORRECTION' | 'REFUND'; amount: number; occurred_at: IsoDateTime; replayed:
boolean
}
```

```
// ---------- ERP ----------
export interface ErpPeriodArgs {
  p_store_id: Uuid
  p_from: IsoDate          // inclusivo
  p_to: IsoDate            // inclusivo; p_to - p_from <= 365 (=> até 366 dias)
  p_timezone?: string      // default 'America/Sao_Paulo'; deve existir em pg_timezone_names
}
export interface ErpSummary {
  store_id: Uuid; from: IsoDate; to: IsoDate; timezone: string
  net_goods_revenue: number; net_shipping_revenue: number; net_revenue: number
  net_cogs: number; returned_damage_loss: number; inventory_adjustment_cost: number
  net_operating_expenses: number; net_inventory_purchase_cash: number
  gross_margin: number; estimated_operating_result: number
  operating_margin_percent: number | null
  cash_received: number; cash_paid_net: number; net_cash_flow: number
  receipt_count: number; refund_count: number; units_sold: number; units_restocked: number
  average_receipt: number | null
  unclassified_stock_outbound_value: number; unclassified_stock_event_count: number
  needs_stock_review: boolean
  is_estimate: true
  accounting_basis: 'manual_cash_based_management'
  inventory_snapshot_current: { quantity: number; stock_value: number; reserved_quantity: number }
  warnings: string[]
}
export interface ErpDay {
  date: IsoDate
  net_revenue: number; net_cogs: number; returned_damage_loss: number
  net_operating_expenses: number; inventory_adjustment_cost: number
  estimated_operating_result: number
  cash_received: number; cash_paid_net: number; net_cash_flow: number
}
export interface ErpDaily { from: IsoDate; to: IsoDate; timezone: string; days: ErpDay[] }
export interface ErpProductPerformanceArgs extends ErpPeriodArgs { p_limit?: number }   // 1..100, default 20
export interface ErpProductRow {
  product_id: Uuid; code: string; name_snapshot: string
  net_units: number; net_goods_revenue: number; net_cogs: number; damage_loss: number
  contribution_before_shared_expenses: number
}
export interface ErpProductPerformance { from: IsoDate; to: IsoDate; products: ErpProductRow[]; note: string }
export interface ErpExpenseBreakdown {
  from: IsoDate; to: IsoDate
  categories: Array<{ category_id: Uuid; name_snapshot: string; classification: ExpenseClassification; net_amount: number }>
}

// ============================================================
// src/types/storefront.ts (PROPOSTA) — contrato HTTP da Edge storefront-api
// ============================================================
export type StorefrontAction = 'catalog' | 'product' | 'checkout' | 'order-status'

export interface CatalogFilters {
  search?: string                 // 0..100
  category_id?: Uuid | null       // inclui subcategorias quando é categoria-mãe
  min_price?: number | null       // 0..9999999999.99
  max_price?: number | null       // >= min_price
  in_stock?: boolean
  sort?: 'newest' | 'price_asc' | 'price_desc' | 'name_asc'   // default 'newest'
  page?: number                   // inteiro 1..1000, default 1
  page_size?: number              // inteiro 1..24, default 20
}
export interface CatalogRequest { action: 'catalog'; store_slug: string; filters?: CatalogFilters }
export interface ProductRequest { action: 'product'; store_slug: string; product_slug: string }
export interface CheckoutInput {
  customer: { full_name: string; phone: string }     // 2..120 / E.164
  address: DeliveryAddress
  items: Array<{ product_id: Uuid; quantity: number }> // 1..20 itens, inteiros 1..999, sem repetição
  note?: string                                        // 0..1000
}
export interface CheckoutRequest {
  action: 'checkout'; store_slug: string
  request_id: Uuid           // UUID v1..v5, gerado UMA vez por tentativa e reutilizado nos retries
  client_secret: string      // 32 bytes aleatórios em base64url sem '=' (43 chars), idem
  input: CheckoutInput
}
export interface OrderStatusRequest { action: 'order-status'; store_slug: string; order_id: Uuid; token: string }
```

```
export interface PublicImage { path: string; alt_text: string; sort_order: number }
export interface PublicProduct {
  id: Uuid; code: string; name: string; slug: string; description: string
  price: number; category_id: Uuid | null; is_featured: boolean
  in_stock: boolean               // disponível - reservado > 0 (quantidade NÃO é exposta)
  images: PublicImage[]           // ordenadas por sort_order; [0] = capa
  attributes: ProductAttributeInput[]
}
export interface PublicStore {
  name: string; title: string; subtitle: string; slug: string
  logo_path: string | null; banner_path: string | null
  primary_color: string; secondary_color: string
  social_links: SocialLinks; whatsapp_number: string
}
export interface PublicCategory { id: Uuid; name: string; slug: string; parent_id: Uuid | null }
export interface CatalogData {
  store: PublicStore; categories: PublicCategory[]; products: PublicProduct[]
  total: number; page: number; page_size: number
}
export type ProductData = PublicProduct
export interface CheckoutData {
  order_id: Uuid; code: string          // ex.: 'PED-000001'
  subtotal: number; status: 'PENDING' | OrderStatus
  reservation_expires_at: IsoDateTime   // +24h; NÃO renovado em replay
  replayed: boolean
  whatsapp_number: string
  token: string                         // 43 chars; segredo de acompanhamento
  tracking_url: string                  // `${APP_URL}/s/{slug}/pedido/{order_id}#token=...`
  whatsapp_url: string                  // https://wa.me/<digits>?text=...
  shipping_included: false
}
export interface OrderStatusData {
  order_id: Uuid; code: string; status: OrderStatus; revision: number; created_at: IsoDateTime
  subtotal: number; shipping_amount: number; shipping_confirmed: boolean
  total: number; shipping_included: boolean; payment_method: 'WHATSAPP'
  tracking_code: string | null
  paid_at: IsoDateTime | null; shipped_at: IsoDateTime | null; delivered_at: IsoDateTime | null
  cancelled_at: IsoDateTime | null; returned_at: IsoDateTime | null
  customer: CustomerSnapshot; address: DeliveryAddress; note: string
  reservation_expires_at: IsoDateTime; reservation_active: boolean
  payment: { received: boolean; refunded: boolean; method: ManualMethod | null }
  timeline: Array<{ event: OrderEventType; status: OrderStatus; created_at: IsoDateTime }>
  items: Array<{ name: string; code: string; quantity: number; unit_price: number; line_total: number;
                 image_path: string | null; attributes: ProductAttributeInput[] }>
}
export type StorefrontErrorCode =
  | 'NOT_FOUND' | 'INVALID_BODY' | 'UNKNOWN_FIELD' | 'INVALID_FIELD' | 'INVALID_SLUG' | 'INVALID_UUID'
  | 'INVALID_TOKEN' | 'INVALID_JSON' | 'INVALID_ACTION' | 'INVALID_STORE_SLUG' | 'INVALID_PRICE_FILTER'
  | 'INVALID_PRICE_RANGE' | 'INVALID_STOCK_FILTER' | 'INVALID_SORT' | 'INVALID_PAGINATION' | 'INVALID_PHONE'
  | 'INVALID_ADDRESS' | 'INVALID_ITEMS' | 'DUPLICATE_PRODUCT' | 'INVALID_QUANTITY' | 'INVALID_REQUEST'
  | 'ORIGIN_NOT_ALLOWED' | 'METHOD_NOT_ALLOWED' | 'BODY_TOO_LARGE' | 'JSON_REQUIRED' | 'RATE_LIMITED'
  | 'STORE_UNAVAILABLE' | 'PRODUCT_UNAVAILABLE' | 'INSUFFICIENT_AVAILABLE_STOCK' | 'IDEMPOTENCY_KEY_REUSED'
  | 'ORDER_TOTAL_LIMIT' | 'SERVICE_UNAVAILABLE'
export type StorefrontResponse<T> = { data: T } | { error: StorefrontErrorCode }
```

Apêndice B — Dicionário das 25 tabelas/views

Gerado do manifesto efetivo. Todas as colunas listadas têm GRANT SELECT para authenticated (filtradas por RLS).

categories

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`store_id`	uuid	não		
`parent_id`	uuid	sim		
`code`	text	não		
`name`	text	não		
`slug`	text	não		
`is_active`	bool	não	true	
`sort_order`	int4	não	0	
`request_id`	uuid	não		
`request_payload`	jsonb	não		
`created_at`	timestamptz	não	now()	
`updated_at`	timestamptz	não	now()	
`archived_at`	timestamptz	sim		

Policies: categories_read SELECT → `((archived_at IS NULL) AND private.catalog_store_allowed(store_id, 'catalog.read'::text))`
GRANT UPDATE (authenticated): nenhum
Constraints: `categories_name_check` : CHECK (((char_length(btrim(name)) >= 2) AND (char_length(btrim(name)) <= 100))); `categories_slug_check` : CHECK ((((char_length(slug) >= 2) AND (char_length(slug) <= 100)) AND (slug ~ '^[a-z0-9]+(-[a-z0-9]+)*$'::text))); `categories_sort_order_check` : CHECK (((sort_order >= 0) AND (sort_order <= 1000000))); `categories_no_self_parent` : CHECK (((parent_id IS NULL) OR (parent_id <> id))); `categories_archived_inactive` : CHECK (((archived_at IS NULL) OR (NOT is_active))); `categories_pkey` : PRIMARY KEY (id); `categories_store_identity` : UNIQUE (id, store_id); `categories_store_code` : UNIQUE (store_id, code); `categories_store_slug` : UNIQUE (store_id, slug); `categories_store_request` : UNIQUE (store_id, request_id); `categories_store_id_fkey` : FOREIGN KEY (store_id) REFERENCES stores(id) ON DELETE RESTRICT; `categories_parent_store` : FOREIGN KEY (parent_id, store_id) REFERENCES categories(id, store_id) ON DELETE RESTRICT

customers

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`store_id`	uuid	não		
`phone`	text	não		
`full_name`	text	não		
`created_at`	timestamptz	não	now()	
`updated_at`	timestamptz	não	now()	

Policies: customers_owner_read SELECT → `private.catalog_store_allowed(store_id, 'customers.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `customers_phone_check` : CHECK ((phone ~ '^\+[1-9][0-9]{7,14}$'::text)); `customers_full_name_check` : CHECK (((char_length(btrim(full_name)) >= 2) AND (char_length(btrim(full_name)) <= 120))); `customers_pkey` : PRIMARY KEY (id); `customers_store_phone` : UNIQUE (store_id, phone); `customers_store_identity` : UNIQUE (id, store_id); `customers_store_id_fkey` : FOREIGN KEY (store_id) REFERENCES stores(id) ON DELETE RESTRICT

expense_categories

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`store_id`	uuid	não		
`name`	text	não		
`classification`	text	não		
`is_active`	bool	não	true	
`request_id`	uuid	não		
`request_payload`	jsonb	não		
`created_at`	timestamptz	não	now()	
`updated_at`	timestamptz	não	now()	

Policies: expense_categories_read SELECT → `private.catalog_store_allowed(store_id, 'expenses.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `expense_categories_name_check` : CHECK (((char_length(btrim(name)) >= 2) AND (char_length(btrim(name)) <= 100))); `expense_categories_classification_check` : CHECK ((classification = ANY (ARRAY['OPERATING'::text, 'INVENTORY_PURCHASE'::text]))); `expense_categories_pkey` : PRIMARY KEY (id); `expense_categories_store_identity` : UNIQUE (id, store_id); `expense_categories_request` : UNIQUE (store_id, request_id); `expense_categories_store_id_fkey` : FOREIGN KEY (store_id) REFERENCES stores(id) ON DELETE RESTRICT

expense_entries

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`store_id`	uuid	não		
`category_id`	uuid	não		
`category_name_snapshot`	text	não		
`classification`	text	não		
`kind`	text	não		
`amount`	numeric	não		
`method`	text	não		
`description`	text	não		
`external_reference`	text	sim		
`occurred_at`	timestamptz	não		
`recorded_at`	timestamptz	não	now()	
`recorded_by`	uuid	não		
`request_id`	uuid	não		
`inventory_movement_id`	uuid	sim		
`reversal_of_id`	uuid	sim		

Policies: expense_entries_read SELECT → `private.catalog_store_allowed(store_id, 'expenses.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `expense_entries_store_id_fkey` : FOREIGN KEY (store_id) REFERENCES stores(id) ON DELETE RESTRICT; `expense_entries_recorded_by_fkey` : FOREIGN KEY (recorded_by) REFERENCES auth.users(id) ON DELETE RESTRICT; `expense_entries_inventory_movement_id_fkey` : FOREIGN KEY (inventory_movement_id) REFERENCES inventory_movements(id) ON DELETE RESTRICT; `expense_entries_reversal_of_id_fkey` : FOREIGN KEY (reversal_of_id) REFERENCES expense_entries(id) ON DELETE RESTRICT; `expense_entries_classification_check` : CHECK ((classification = ANY (ARRAY['OPERATING'::text, 'INVENTORY_PURCHASE'::text]))); `expense_entries_kind_check` : CHECK ((kind = ANY (ARRAY['EXPENSE'::text, 'CORRECTION'::text, 'REFUND'::text]))); `expense_entries_amount_check` : CHECK (((amount > (0)::numeric) AND (amount <> 'NaN'::numeric))); `expense_entries_method_check` : CHECK ((method = ANY (ARRAY['PIX'::text, 'CASH'::text, 'BANK_TRANSFER'::text, 'CARD'::text, 'OTHER'::text]))); `expense_entries_description_check` : CHECK (((char_length(btrim(description)) >= 3) AND (char_length(btrim(description)) <= 500)));

`expense_entries_external_reference_check` : CHECK (((external_reference IS NULL) OR (char_length(external_reference) <= 160))); `expense_entries_reversal` : CHECK (((kind = 'EXPENSE'::text) = (reversal_of_id IS NULL))); `expense_entries_stock_link` : CHECK (((inventory_movement_id IS NULL) OR ((kind = 'EXPENSE'::text) AND (classification = 'INVENTORY_PURCHASE'::text)))); `expense_entries_pkey` : PRIMARY KEY (id); `expense_entries_inventory_movement_id_key` : UNIQUE (inventory_movement_id); `expense_entries_reversal_of_id_key` : UNIQUE (reversal_of_id); `expense_entries_request` : UNIQUE (store_id, request_id); `expense_entries_category_store` : FOREIGN KEY (category_id, store_id) REFERENCES expense_categories(id, store_id) ON DELETE RESTRICT

inventory_balances

Coluna	Tipo	Nulo	Default	Gerada
`product_id`	uuid	não		
`store_id`	uuid	não		
`quantity`	int4	não	0	
`stock_value`	numeric	não	0	
`average_cost`	numeric	sim		GENERATED
`revision`	int8	não	0	
`last_movement_id`	uuid	sim		
`created_at`	timestamptz	não	now()	
`updated_at`	timestamptz	não	now()	

Policies: inventory_balances_read SELECT → `private.catalog_store_allowed(store_id, 'inventory.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `inventory_balances_quantity_check` : CHECK (((quantity >= 0) AND (quantity <= 1000000000))); `inventory_balances_stock_value_check` : CHECK (((stock_value >= (0)::numeric) AND (stock_value <> 'NaN'::numeric))); `inventory_balances_revision_check` : CHECK ((revision >= 0)); `inventory_balances_zero_quantity` : CHECK (((quantity > 0) OR (stock_value = (0)::numeric))); `inventory_balances_pkey` : PRIMARY KEY (product_id); `inventory_balances_product_store` : FOREIGN KEY (product_id, store_id) REFERENCES products(id, store_id) ON DELETE RESTRICT; `inventory_balances_last_movement` : FOREIGN KEY (last_movement_id) REFERENCES inventory_movements(id) ON DELETE RESTRICT

inventory_movements

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`store_id`	uuid	não		
`product_id`	uuid	não		
`product_code`	text	não		
`product_name`	text	não		
`request_id`	uuid	não		
`batch_request_id`	uuid	sim		
`movement_type`	inventory_movement_type	não		
`quantity_delta`	int4	não		
`unit_cost`	numeric	não		
`value_delta`	numeric	não		
`quantity_before`	int4	não		
`quantity_after`	int4	não		
`value_before`	numeric	não		
`value_after`	numeric	não		
`average_cost_before`	numeric	não		

Coluna	Tipo	Nulo	Default	Gerada
`average_cost_after`	numeric	não		
`revision`	int8	não		
`reason`	text	não		
`reference`	text	sim		
`reversal_of_id`	uuid	sim		
`created_by`	uuid	não		
`created_at`	timestamptz	não	now()	

Policies: inventory_movements_read SELECT → `private.catalog_store_allowed(store_id, 'inventory.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `inventory_movements_quantity_delta_check` : CHECK (((quantity_delta <> 0) AND ((quantity_delta >= '-1000000000'::integer) AND (quantity_delta <= 1000000000)))); `inventory_movements_unit_cost_check` : CHECK (((unit_cost >= (0)::numeric) AND (unit_cost <> 'NaN'::numeric))); `inventory_movements_value_delta_check` : CHECK ((value_delta <> 'NaN'::numeric)); `inventory_movements_quantity_before_check` : CHECK ((quantity_before >= 0)); `inventory_movements_quantity_after_check` : CHECK (((quantity_after >= 0) AND (quantity_after <= 1000000000))); `inventory_movements_value_before_check` : CHECK (((value_before >= (0)::numeric) AND (value_before <> 'NaN'::numeric))); `inventory_movements_value_after_check` : CHECK (((value_after >= (0)::numeric) AND (value_after <> 'NaN'::numeric))); `inventory_movements_average_cost_before_check` : CHECK ((average_cost_before >= (0)::numeric)); `inventory_movements_average_cost_after_check` : CHECK ((average_cost_after >= (0)::numeric)); `inventory_movements_revision_check` : CHECK ((revision > 0)); `inventory_movements_reason_check` : CHECK (((char_length(btrim(reason)) >= 3) AND (char_length(btrim(reason)) <= 500))); `inventory_movements_reference_check` : CHECK (((reference IS NULL) OR (char_length(reference) <= 120))); `inventory_movements_quantity` : CHECK (((quantity_after)::bigint = ((quantity_before)::bigint + (quantity_delta)::bigint))); `inventory_movements_value` : CHECK ((value_after = (value_before + value_delta))); `inventory_movements_zero_value` : CHECK ((((quantity_before > 0) OR (value_before = (0)::numeric)) AND ((quantity_after > 0) OR (value_after = (0)::numeric)))); `inventory_movements_sign` : CHECK ((((movement_type = ANY (ARRAY['ENTRY'::inventory_movement_type, 'ADJUSTMENT_IN'::inventory_movement_type])) AND (quantity_delta > 0) AND (value_delta >= (0)::numeric)) OR ((movement_type = ANY (ARRAY['EXIT'::inventory_movement_type, ; `inventory_movements_reversal` : CHECK (((movement_type = 'REVERSAL'::inventory_movement_type) = (reversal_of_id IS NOT NULL))); `inventory_movements_pkey` : PRIMARY KEY (id); `inventory_movements_reversal_of_id_key` : UNIQUE (reversal_of_id); `inventory_movements_request` : UNIQUE (store_id, request_id); `inventory_movements_revision` : UNIQUE (product_id, revision); `inventory_movements_reversal_of_id_fkey` : FOREIGN KEY (reversal_of_id) REFERENCES inventory_movements(id) ON DELETE RESTRICT; `inventory_movements_created_by_fkey` : FOREIGN KEY (created_by) REFERENCES auth.users(id) ON DELETE RESTRICT; `inventory_movements_product_store` : FOREIGN KEY (product_id, store_id) REFERENCES products(id, store_id) ON DELETE RESTRICT

inventory_reservations

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`store_id`	uuid	não		
`product_id`	uuid	não		
`order_id`	uuid	não		
`quantity`	int4	não		
`state`	text	não	'HELD'::text	
`expires_at`	timestamptz	não		
`created_at`	timestamptz	não	now()	

Policies: reservations_owner_read SELECT → `(private.catalog_store_allowed(store_id, 'inventory.read'::text) OR private.catalog_store_allowed(store_id, 'orders.read'::text))`
GRANT UPDATE (authenticated): nenhum
Constraints: `inventory_reservations_quantity_check` : CHECK (((quantity >= 1) AND (quantity <= 999))); `inventory_reservations_state_check` : CHECK ((state = ANY (ARRAY['HELD'::text, 'RELEASED'::text, 'CONSUMED'::text]))); `inventory_reservations_pkey` : PRIMARY KEY (id); `inventory_reservations_order_product` : UNIQUE (order_id, product_id); `inventory_reservations_product_store` : FOREIGN KEY (product_id, store_id) REFERENCES products(id, store_id) ON DELETE RESTRICT; `inventory_reservations_order_store` : FOREIGN KEY (order_id, store_id) REFERENCES orders(id, store_id) ON DELETE RESTRICT

order_items

Coluna	Tipo	Nulo	Default	Gerada
id	uuid	não	gen_random_uuid()	
store_id	uuid	não		
order_id	uuid	não		
product_id	uuid	não		
product_code	text	não		
product_name	text	não		
product_slug	text	não		
image_path	text	sim		
attributes_snapshot	jsonb	não	'[]'::jsonb	
quantity	int4	não		
unit_price	numeric	não		
unit_cost_snapshot	numeric	não		
line_total	numeric	não		

Policies: order_items_owner_read SELECT → `private.catalog_store_allowed(store_id, 'orders.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `order_items_quantity_check` : CHECK ((((quantity >= 1) AND (quantity <= 999))); `order_items_unit_price_check` : CHECK ((unit_price >= (0)::numeric)); `order_items_unit_cost_snapshot_check` : CHECK ((unit_cost_snapshot >= (0)::numeric)); `order_items_check` : CHECK ((line_total = ((quantity)::numeric * unit_price))); `order_items_pkey` : PRIMARY KEY (id); `order_items_product` : UNIQUE (order_id, product_id); `order_items_order_store` : FOREIGN KEY (order_id, store_id) REFERENCES orders(id, store_id) ON DELETE RESTRICT; `order_items_product_store` : FOREIGN KEY (product_id, store_id) REFERENCES products(id, store_id) ON DELETE RESTRICT; `order_items_order_identity` : UNIQUE (id, order_id, store_id)

order_payments

Coluna	Tipo	Nulo	Default	Gerada
id	uuid	não	gen_random_uuid()	
store_id	uuid	não		
order_id	uuid	não		
kind	text	não		
method	text	não		
amount	numeric	não		
external_reference	text	sim		
note	text	não		
confirmed_by	uuid	não		
confirmed_at	timestamptz	não	now()	
request_id	uuid	não		

Policies: order_payments_read SELECT → `private.catalog_store_allowed(store_id, 'orders.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `order_payments_kind_check` : CHECK ((kind = ANY (ARRAY['RECEIPT'::text, 'REFUND'::text]))); `order_payments_method_check` : CHECK ((method = ANY (ARRAY['PIX'::text, 'CASH'::text, 'BANK_TRANSFER'::text, 'CARD'::text, 'OTHER'::text]))); `order_payments_amount_check` : CHECK (((amount >= (0)::numeric) AND (amount <> 'NaN'::numeric))); `order_payments_external_reference_check` : CHECK (((external_reference IS NULL) OR (char_length(external_reference) <= 160))); `order_payments_note_check` : CHECK (((char_length(btrim(note)) >= 3) AND (char_length(btrim(note)) <= 1000))); `order_payments_pkey` : PRIMARY KEY (id); `order_payments_kind` : UNIQUE (order_id, kind); `order_payments_confirmed_by_fkey` : FOREIGN KEY (confirmed_by)

REFERENCES auth.users(id) ON DELETE RESTRICT; `order_payments_order` : FOREIGN KEY (order_id, store_id) REFERENCES orders(id, store_id) ON DELETE RESTRICT

order_return_items

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`store_id`	uuid	não		
`order_id`	uuid	não		
`order_item_id`	uuid	não		
`disposition`	text	não		
`quantity`	int4	não		
`allocated_cost`	numeric	não		
`restocked_value`	numeric	não		
`loss_value`	numeric	não		
`note`	text	não		
`created_at`	timestamptz	não	now()	

Policies: order_return_read SELECT → `private.catalog_store_allowed(store_id, 'orders.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `order_return_items_disposition_check` : CHECK ((disposition = ANY (ARRAY['RESTOCKED'::text, 'DAMAGED'::text]))); `order_return_items_quantity_check` : CHECK ((quantity > 0)); `order_return_items_allocated_cost_check` : CHECK ((allocated_cost >= (0)::numeric)); `order_return_items_restocked_value_check` : CHECK ((restocked_value >= (0)::numeric)); `order_return_items_loss_value_check` : CHECK ((loss_value >= (0)::numeric)); `order_return_items_note_check` : CHECK ((char_length(note) <= 1000)); `order_return_cost` : CHECK ((allocated_cost = (restocked_value + loss_value))); `order_return_items_pkey` : PRIMARY KEY (id); `order_return_unique` : UNIQUE (order_item_id); `order_return_item` : FOREIGN KEY (order_item_id, order_id, store_id) REFERENCES order_items(id, order_id, store_id) ON DELETE RESTRICT

order_status_history

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`store_id`	uuid	não		
`order_id`	uuid	não		
`revision`	int8	não		
`event_type`	text	não		
`status_before`	order_status	sim		
`status_after`	order_status	não		
`note`	text	não		
`actor_id`	uuid	sim		
`request_id`	uuid	não		
`created_at`	timestamptz	não	now()	

Policies: order_history_read SELECT → `private.catalog_store_allowed(store_id, 'orders.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `order_history_revision` : UNIQUE (order_id, revision); `order_status_history_revision_check` : CHECK ((revision >= 0)); `order_status_history_event_type_check` : CHECK ((event_type = ANY (ARRAY['CREATED'::text, 'TERMS_UPDATED'::text, 'PAYMENT_CONFIRMED'::text, 'SHIPPED'::text, 'DELIVERED'::text, 'CANCELLED'::text, 'RETURNED'::text]))); `order_status_history_note_check` : CHECK ((char_length(note) <= 1000)); `order_status_history_pkey` : PRIMARY KEY (id);

`order_status_history_actor_id_fkey` : FOREIGN KEY (actor_id) REFERENCES auth.users(id) ON DELETE RESTRICT;
`order_history_order` : FOREIGN KEY (order_id, store_id) REFERENCES orders(id, store_id) ON DELETE RESTRICT

order_stock_operations

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`store_id`	uuid	não		
`order_id`	uuid	não		
`order_item_id`	uuid	não		
`kind`	text	não		
`movement_id`	uuid	não		
`quantity`	int4	não		
`allocated_value`	numeric	não		
`created_at`	timestamptz	não	now()	

Policies: order_stock_read SELECT → `private.catalog_store_allowed(store_id, 'orders.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `order_stock_operations_kind_check` : CHECK ((kind = ANY (ARRAY['SALE'::text, 'RESTOCK'::text]))); `order_stock_operations_quantity_check` : CHECK ((quantity > 0)); `order_stock_operations_allocated_value_check` : CHECK ((allocated_value >= (0)::numeric)); `order_stock_operations_pkey` : PRIMARY KEY (id); `order_stock_operations_movement_id_key` : UNIQUE (movement_id); `order_stock_kind` : UNIQUE (order_item_id, kind); `order_stock_operations_movement_id_fkey` : FOREIGN KEY (movement_id) REFERENCES inventory_movements(id) ON DELETE RESTRICT; `order_stock_item` : FOREIGN KEY (order_item_id, order_id, store_id) REFERENCES order_items(id, order_id, store_id) ON DELETE RESTRICT

orders

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`store_id`	uuid	não		
`customer_id`	uuid	não		
`code`	text	não		
`status`	order_status	não	'PENDING'::order_status	
`request_id`	uuid	não		
`request_payload`	jsonb	não		
`customer_snapshot`	jsonb	não		
`delivery_address`	jsonb	não		
`store_snapshot`	jsonb	não		
`customer_note`	text	não	''::text	
`payment_method`	text	não	'WHATSAPP'::text	
`subtotal`	numeric	não		
`total`	numeric	não		
`reservation_expires_at`	timestamptz	não		
`created_at`	timestamptz	não	now()	
`updated_at`	timestamptz	não	now()	
`revision`	int8	não	0	
`shipping_amount`	numeric	não	0	

Coluna	Tipo	Nulo	Default	Gerada
`shipping_confirmed`	bool	não	false	
`tracking_code`	text	sim		
`paid_at`	timestamptz	sim		
`shipped_at`	timestamptz	sim		
`delivered_at`	timestamptz	sim		
`cancelled_at`	timestamptz	sim		
`returned_at`	timestamptz	sim		

Policies: orders_owner_read SELECT → `private.catalog_store_allowed(store_id, 'orders.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `orders_customer_note_check` : CHECK ((char_length(customer_note) <= 1000)); `orders_payment_method_check` : CHECK ((payment_method = 'WHATSAPP'::text)); `orders_subtotal_check` : CHECK (((subtotal >= (0)::numeric) AND (subtotal <> 'NaN'::numeric))); `orders_pkey` : PRIMARY KEY (id); `orders_store_identity` : UNIQUE (id, store_id); `orders_store_code` : UNIQUE (store_id, code); `orders_store_request` : UNIQUE (store_id, request_id); `orders_store_id_fkey` : FOREIGN KEY (store_id) REFERENCES stores(id) ON DELETE RESTRICT; `orders_customer_store` : FOREIGN KEY (customer_id, store_id) REFERENCES customers(id, store_id); `orders_revision_check` : CHECK ((revision >= 0)); `orders_shipping_amount_check` : CHECK (((shipping_amount >= (0)::numeric) AND (shipping_amount <> 'NaN'::numeric))); `orders_tracking_code_check` : CHECK (((tracking_code IS NULL) OR (char_length(tracking_code) <= 120))); `orders_total_calculation` : CHECK (((total = (subtotal + shipping_amount)) AND (total >= (0)::numeric) AND (total <> 'NaN'::numeric)))

permissions

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`code`	text	não		
`module`	text	não		
`description`	text	não	''::text	
`is_system`	bool	não	false	
`is_active`	bool	não	true	
`created_at`	timestamptz	não	now()	
`updated_at`	timestamptz	não	now()	

Policies: permissions_sudo ALL → `( SELECT private.can_manage_security() AS can_manage_security)`
GRANT UPDATE (authenticated): `code` , `created_at` , `description` , `id` , `is_active` , `is_system` , `module` , `updated_at`
GRANT INSERT: `code` , `created_at` , `description` , `id` , `is_active` , `is_system` , `module` , `updated_at`
Constraints: `permissions_code_check` : CHECK ((code ~ '^[a-z][a-z0-9_.]{2,99}$'::text)); `permissions_module_check` : CHECK (((char_length(module) >= 1) AND (char_length(module) <= 80))); `permissions_pkey` : PRIMARY KEY (id); `permissions_code_key` : UNIQUE (code)

plans

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`code`	text	não		
`name`	text	não		
`description`	text	não	''::text	
`price`	numeric	não	0	
`currency`	text	não	'BRL'::text	
`is_active`	bool	não	true	
`created_at`	timestamptz	não	now()	

Coluna	Tipo	Nulo	Default	Gerada
updated_at	timestamptz	não	now()	

Policies: plans_update_sudo UPDATE → `(( SELECT private.is_sudo() AS is_sudo) AND ( SELECT private.has_global_permission('plans.manage'::text) AS has_global_permission) AND ( SELECT private.has_global_permission('auth.access'::text) AS has_global_permission))` ; plans_select SELECT → `((is_active AND (SELECT private.has_global_permission('plans.read'::text) AS has_global_permission)) OR ((SELECT private.is_sudo() AS is_sudo) AND (SELECT private.has_global_permission('plans.manage'::text) AS has_global_permission) AND (SELECT private.has_global_permission('auth.access'::text) AS has_global_permission)))`
GRANT UPDATE (authenticated): `description` , `is_active` , `name`
Constraints: `plans_name_check` : CHECK (((char_length(btrim(name)) >= 1) AND (char_length(btrim(name)) <= 100))); `plans_description_check` : CHECK ((char_length(description) <= 1000)); `plans_current_code` : CHECK ((code = 'FREE'::text)); `plans_current_price` : CHECK (((price = (0)::numeric) AND (currency = 'BRL'::text))); `plans_pkey` : PRIMARY KEY (id); `plans_code_key` : UNIQUE (code)

product_attributes

Coluna	Tipo	Nulo	Default	Gerada
id	uuid	não	gen_random_uuid()	
store_id	uuid	não		
product_id	uuid	não		
name	text	não		
name_key	text	sim		GENERATED
value	text	não		
sort_order	int2	não		
created_at	timestamptz	não	now()	
updated_at	timestamptz	não	now()	

Policies: product_attributes_read SELECT → `private.catalog_product_allowed(product_id, 'catalog.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `product_attributes_name_check` : CHECK (((char_length(btrim(name)) >= 1) AND (char_length(btrim(name)) <= 80))); `product_attributes_value_check` : CHECK (((char_length(btrim(value)) >= 1) AND (char_length(btrim(value)) <= 500))); `product_attributes_sort_order_check` : CHECK (((sort_order >= 0) AND (sort_order <= 29))); `product_attributes_pkey` : PRIMARY KEY (id); `product_attributes_name` : UNIQUE (product_id, name_key); `product_attributes_position` : UNIQUE (product_id, sort_order); `product_attributes_product_store` : FOREIGN KEY (product_id, store_id) REFERENCES products(id, store_id) ON DELETE RESTRICT

product_images

Coluna	Tipo	Nulo	Default	Gerada
id	uuid	não	gen_random_uuid()	
store_id	uuid	não		
product_id	uuid	não		
path	text	não		
alt_text	text	não	''::text	
sort_order	int2	não		
created_at	timestamptz	não	now()	
updated_at	timestamptz	não	now()	

Policies: product_images_read SELECT → `private.catalog_product_allowed(product_id, 'catalog.read'::text)`
GRANT UPDATE (authenticated): nenhum
Constraints: `product_images_pkey` : PRIMARY KEY (id); `product_images_path_key` : UNIQUE (path); `product_images_position` : UNIQUE (product_id, sort_order) DEFERRABLE; `product_images_product_store` : FOREIGN KEY (product_id, store_id) REFERENCES

products(id, store_id) ON DELETE RESTRICT; `product_images_alt_text_check` : CHECK ((char_length(alt_text) <= 200)); `product_images_sort_order_check` : CHECK (((sort_order >= 0) AND (sort_order <= 3))); `product_images_path` : CHECK (COALESCE((private.product_media_location(path) = ARRAY[store_id, product_id]), false))

product_stock_balances

Coluna	Tipo	Nulo	Default	Gerada
`product_id`	uuid	sim		
`store_id`	uuid	sim		
`code`	text	sim		
`name`	text	sim		
`status`	product_status	sim		
`category_id`	uuid	sim		
`quantity`	int4	sim		
`stock_value`	numeric	sim		
`average_cost`	numeric	sim		
`revision`	int8	sim		
`last_movement_id`	uuid	sim		
`updated_at`	timestamptz	sim		
`reserved_quantity`	int4	sim		
`available_quantity`	int4	sim		

GRANT UPDATE (authenticated): nenhum

products

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`store_id`	uuid	não		
`category_id`	uuid	sim		
`code`	text	não		
`name`	text	não		
`slug`	text	não		
`description`	text	não	''::text	
`status`	product_status	não	'INACTIVE'::product_status	
`price`	numeric	não		
`cost_price`	numeric	não	0	
`is_featured`	bool	não	false	
`request_id`	uuid	não		
`request_payload`	jsonb	não		
`created_at`	timestamptz	não	now()	
`updated_at`	timestamptz	não	now()	
`archived_at`	timestamptz	sim		

Policies: products_read SELECT → `((archived_at IS NULL) AND private.catalog_store_allowed(store_id, 'catalog.read'::text))`
GRANT UPDATE (authenticated): nenhum

Constraints: `products_store_request` : UNIQUE (store_id, request_id); `products_store_id_fkey` : FOREIGN KEY (store_id) REFERENCES stores(id) ON DELETE RESTRICT; `products_category_store` : FOREIGN KEY (category_id, store_id) REFERENCES categories(id, store_id) ON DELETE RESTRICT; `products_name_check` : CHECK (((char_length(btrim(name)) >= 2) AND (char_length(btrim(name)) <= 160))); `products_slug_check` : CHECK ((((char_length(slug) >= 2) AND (char_length(slug) <= 160)) AND (slug ~ '^[a-z0-9]+(-[a-z0-9]+)*$'::text))); `products_description_check` : CHECK ((char_length(description) <= 10000)); `products_price_check` : CHECK (((price >= (0)::numeric) AND (price <> 'NaN'::numeric))); `products_cost_price_check` : CHECK (((cost_price >= (0)::numeric) AND (cost_price <> 'NaN'::numeric))); `products_archived_inactive` : CHECK (((archived_at IS NULL) OR (status = 'INACTIVE'::product_status))); `products_pkey` : PRIMARY KEY (id); `products_store_identity` : UNIQUE (id, store_id); `products_store_code` : UNIQUE (store_id, code); `products_store_slug` : UNIQUE (store_id, slug)

profiles

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não		
`full_name`	text	não	''::text	
`phone`	text	sim		
`avatar_path`	text	sim		
`is_active`	bool	não	true	
`created_at`	timestamptz	não	now()	
`updated_at`	timestamptz	não	now()	

Policies: profiles_update_self UPDATE → `((id = ( SELECT auth.uid() AS uid)) AND ( SELECT private.has_global_permission('profile.update'::text) AS has_global_permission) AND ( SELECT private.has_global_permission('auth.access'::text) AS has_global_permission))` ; profiles_select SELECT → `((id = ( SELECT auth.uid() AS uid)) OR ( SELECT private.can_manage_security() AS can_manage_security))`
GRANT UPDATE (authenticated): `avatar_path` , `full_name` , `phone`
Constraints: `profiles_full_name_check` : CHECK ((char_length(full_name) <= 120)); `profiles_phone_check` : CHECK (((phone IS NULL) OR (phone ~ '^\+[1-9][0-9]{7,14}$'::text))); `profiles_avatar_path_check` : CHECK (((avatar_path IS NULL) OR (char_length(avatar_path) <= 512))); `profiles_pkey` : PRIMARY KEY (id); `profiles_id_fkey` : FOREIGN KEY (id) REFERENCES auth.users(id) ON DELETE CASCADE

role_permissions

Coluna	Tipo	Nulo	Default	Gerada
`role_id`	uuid	não		
`permission_id`	uuid	não		
`is_enabled`	bool	não	true	
`created_at`	timestamptz	não	now()	
`updated_at`	timestamptz	não	now()	

Policies: role_permissions_sudo ALL → `( SELECT private.can_manage_security() AS can_manage_security)`
GRANT UPDATE (authenticated): `created_at` , `is_enabled` , `permission_id` , `role_id` , `updated_at`
GRANT INSERT: `created_at` , `is_enabled` , `permission_id` , `role_id` , `updated_at`
Constraints: `role_permissions_pkey` : PRIMARY KEY (role_id, permission_id); `role_permissions_role_id_fkey` : FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE; `role_permissions_permission_id_fkey` : FOREIGN KEY (permission_id) REFERENCES permissions(id) ON DELETE CASCADE

roles

Coluna	Tipo	Nulo	Default	Gerada
`id`	uuid	não	gen_random_uuid()	
`code`	text	não		
`name`	text	não		
`description`	text	não	''::text	

Coluna	Tipo	Nulo	Default	Gerada
is_system	bool	não	false	
is_active	bool	não	true	
created_at	timestamptz	não	now()	
updated_at	timestamptz	não	now()	

Policies: roles_sudo ALL → (SELECT private.can_manage_security() AS can_manage_security)
GRANT UPDATE (authenticated): code , created_at , description , id , is_active , is_system , name , updated_at
GRANT INSERT: code , created_at , description , id , is_active , is_system , name , updated_at
Constraints: roles_code_check : CHECK ((code ~ '^[A-Z][A-Z0-9_]{1,49}$'::text)); roles_name_check : CHECK (((char_length(name) >= 1) AND (char_length(name) <= 120))); roles_pkey : PRIMARY KEY (id); roles_code_key : UNIQUE (code)

security_audit_logs

Coluna	Tipo	Nulo	Default	Gerada
id	int8	não		
actor_id	uuid	sim		
table_name	text	não		
operation	text	não		
row_before	jsonb	sim		
row_after	jsonb	sim		
created_at	timestamptz	não	now()	

Policies: security_audit_select SELECT → (SELECT private.can_manage_security() AS can_manage_security)
GRANT UPDATE (authenticated): nenhum
Constraints: security_audit_logs_operation_check : CHECK ((operation = ANY (ARRAY['INSERT'::text, 'UPDATE'::text, 'DELETE'::text]))); security_audit_logs_pkey : PRIMARY KEY (id)

stores

Coluna	Tipo	Nulo	Default	Gerada
id	uuid	não	gen_random_uuid()	
owner_id	uuid	não		
plan_id	uuid	não		
name	text	não		
title	text	não		
subtitle	text	não	''::text	
slug	text	não		
cnpj	text	sim		
email	text	sim		
whatsapp_number	text	não		
logo_path	text	sim		
banner_path	text	sim		
primary_color	text	não	'#6750A4'::text	
secondary_color	text	não	'#03A9F4'::text	
social_links	jsonb	não	'{}'::jsonb	
is_published	bool	não	false	

Coluna	Tipo	Nulo	Default	Gerada
`onboarding_request_id`	uuid	não		
`onboarding_payload`	jsonb	não		
`created_at`	timestamptz	não	now()	
`updated_at`	timestamptz	não	now()	
`archived_at`	timestamptz	sim		

Policies: stores_select SELECT → `((archived_at IS NULL) AND private.has_store_permission(id, 'store.read'::text))`; stores_update UPDATE → `((archived_at IS NULL) AND private.has_store_permission(id, 'store.update'::text))`
GRANT UPDATE (authenticated): `banner_path`, `cnpj`, `email`, `is_published`, `logo_path`, `name`, `primary_color`, `secondary_color`, `slug`, `social_links`, `subtitle`, `title`, `whatsapp_number`
Constraints: `stores_name_check`: CHECK (((char_length(btrim(name)) >= 2) AND (char_length(btrim(name)) <= 120))); `stores_title_check`: CHECK (((char_length(btrim(title)) >= 2) AND (char_length(btrim(title)) <= 160))); `stores_subtitle_check`: CHECK ((char_length(subtitle) <= 300)); `stores_cnpj_check`: CHECK (((cnpj IS NULL) OR ((char_length(cnpj) >= 1) AND (char_length(cnpj) <= 32)))); `stores_email_check`: CHECK (((email IS NULL) OR ((char_length(email) <= 254) AND (email ~ '^[^[:space:]@]+@[^[:space:]@]+\.[^[:space:]@]+$'::text)))); `stores_whatsapp_number_check`: CHECK ((whatsapp_number ~ '^\+[1-9][0-9]{7,14}$'::text)); `stores_primary_color_check`: CHECK ((primary_color ~ '^#[0-9A-Fa-f]{6}$'::text)); `stores_secondary_color_check`: CHECK ((secondary_color ~ '^#[0-9A-Fa-f]{6}$'::text)); `stores_social_links_check`: CHECK (private.valid_social_links(social_links)); `stores_slug_format`: CHECK ((((char_length(slug) >= 3) AND (char_length(slug) <= 63)) AND (slug ~ '^[a-z0-9]+(-[a-z0-9]+)*$'::text) AND (slug <> ALL (ARRAY['admin'::text, 'api'::text, 'app'::text, 'auth'::text, 'login'::text, 'signup'::text, 'dashboard'::text, ; `stores_logo_path`: CHECK (((logo_path IS NULL) OR COALESCE((private.store_id_from_media_path(logo_path) = id), false))); `stores_banner_path`: CHECK (((banner_path IS NULL) OR COALESCE((private.store_id_from_media_path(banner_path) = id), false))); `stores_archived_unpublished`: CHECK (((archived_at IS NULL) OR (NOT is_published))); `stores_pkey`: PRIMARY KEY (id); `stores_slug_key`: UNIQUE (slug); `stores_onboarding_key`: UNIQUE (owner_id, onboarding_request_id); `stores_owner_id_fkey`: FOREIGN KEY (owner_id) REFERENCES auth.users(id) ON DELETE RESTRICT; `stores_plan_id_fkey`: FOREIGN KEY (plan_id) REFERENCES plans(id) ON DELETE RESTRICT

user_platform_roles

Coluna	Tipo	Nulo	Default	Gerada
`user_id`	uuid	não		
`role_code`	text	não		
`created_at`	timestamptz	não	now()	

Policies: platform_roles_select SELECT → `((user_id = ( SELECT auth.uid() AS uid)) OR ( SELECT private.can_manage_security() AS can_manage_security))`
GRANT UPDATE (authenticated): nenhum
Constraints: `platform_role_codes`: CHECK ((role_code = ANY (ARRAY['USER'::text, 'SUDO'::text]))); `user_platform_roles_pkey`: PRIMARY KEY (user_id, role_code); `user_platform_roles_user_id_fkey`: FOREIGN KEY (user_id) REFERENCES auth.users(id) ON DELETE CASCADE; `user_platform_roles_role_code_fkey`: FOREIGN KEY (role_code) REFERENCES roles(code) ON UPDATE RESTRICT ON DELETE RESTRICT

Apêndice C — Permissões, buckets e policies de Storage

Permissão	Módulo	Descrição
`auth.access`	auth	Acessar area autenticada da aplicacao.
`catalog.categories.manage`	catalog	Criar, editar e arquivar categorias/subcategorias.
`catalog.products.manage`	catalog	Criar, editar, arquivar produtos e substituir atributos.
`catalog.products.media`	catalog	Enviar, vincular, ordenar e desvincular imagens de produtos.
`catalog.read`	catalog	Consultar o catalogo privado da loja.
`customers.read`	customers	Consultar contatos privados da loja.
`expenses.categories.manage`	expenses	Criar, renomear e ativar/desativar categorias.
`expenses.read`	expenses	Consultar categorias e lancamentos de despesas.
`expenses.record`	expenses	Declarar despesas pagas e compras de estoque.
`expenses.reverse`	expenses	Corrigir lancamento ou declarar reembolso recebido.
`inventory.adjust`	inventory	Ajustar contagem fisica individual ou em lote.
`inventory.entry`	inventory	Registrar entradas manuais de estoque.
`inventory.exit`	inventory	Registrar saidas manuais de estoque.
`inventory.read`	inventory	Consultar saldo, valor de estoque e historico de movimentos.
`inventory.reverse`	inventory	Reverter o ultimo movimento ainda nao revertido.
`orders.cancel`	orders	Cancelar pendente ou pago ainda nao enviado.
`orders.create`	orders	Habilitar criacao publica de pedidos pelo WhatsApp.
`orders.deliver`	orders	Registrar entrega de pedido enviado.
`orders.pay`	orders	Confirmar recebimento manual e baixar estoque.
`orders.read`	orders	Consultar pedidos privados da loja.
`orders.return`	orders	Receber devolucao total e registrar reembolso manual.
`orders.ship`	orders	Registrar envio de pedido pago.
`orders.terms`	orders	Confirmar frete manual antes do pagamento.
`plans.manage`	plans	SUDO: editar nome, descricao e disponibilidade do plano.
`plans.read`	plans	Consultar plano gratuito disponivel.
`profile.update`	profile	Editar o proprio perfil.
`reports.read`	reports	Consultar ERP gerencial com valores financeiros e custos.
`security.manage`	security	Indicador de acesso ao painel interno de seguranca.
`store.admin`	store	SUDO: administrar lojas de outros proprietarios.
`store.archive`	store	Arquivar a loja.
`store.branding.manage`	store	Enviar, vincular e remover logo/banner.
`store.create`	store	Criar a primeira loja ativa pelo onboarding.
`store.publish`	store	Publicar ou despublicar a loja.
`store.read`	store	Consultar a loja autorizada.

Permissão	Módulo	Descrição
`store.update`	store	Editar dados e personalizacao da loja.
`storefront.read`	storefront	Disponibilizar a vitrine publica da loja.

Buckets

id	público	max bytes	MIME
`store-logos`	True	2097152	image/png, image/jpeg, image/webp
`store-banners`	True	5242880	image/png, image/jpeg, image/webp
`product-images`	True	5242880	image/png, image/jpeg, image/webp

Policies de storage.objects

Policy	Cmd	Expressão
vibestore_product_media_delete	DELETE	`private.can_delete_product_media(bucket_id, name)`
vibestore_product_media_insert	INSERT	`private.can_manage_product_media(bucket_id, name, 'catalog.products.media'::text)`
vibestore_product_media_read	SELECT	`private.can_manage_product_media(bucket_id, name, 'catalog.read'::text)`
vibestore_branding_delete	DELETE	`private.can_delete_store_media(bucket_id, name)`
vibestore_branding_insert	INSERT	`private.can_manage_store_media(bucket_id, name, 'store.branding.manage'::text)`
vibestore_branding_select	SELECT	`private.can_manage_store_media(bucket_id, name, 'store.read'::text)`

Sem policy de UPDATE em storage.objects (uploads imutáveis, `upsert: false`).

Documento gerado a partir do manifesto efetivo do PostgreSQL local. Rotas, arquivos e stores são propostas. Nenhum deploy remoto foi realizado.